

Multimedia Retrieval

University of Basel Course Textbook

Multimedia Retrieval - 2026 - Uni Basel

Preface

This textbook accompanies the Multimedia Retrieval course at the University of Basel. The online edition remains the authoritative source for interactive material and updates.

Contents

Multimedia Retrieval	1
From Ctrl+F to AI Answers	1
The Semantic Gap	1
Closing the Gap	2
How This Book Is Organized	4
What to Expect in Each Chapter	5
What You Will Be Able To Do	5
Further Reading	6
1 - Classical Text Retrieval	7
Retrieval Systems and Flows	9
From Text to Searchable Vectors	13
Boolean Retrieval	21
Vector Space Retrieval	26
Probabilistic Ranking and BM25	30
Summary	38
2 - Performance Evaluation	41
Designing a Retrieval Benchmark	42
Precision and Recall	46
Evaluating Ranked Results	51
Evaluating Graded Relevance	58
Evaluating Text Classifiers	63
Scores, Thresholds, and ROC Curves	69
When Evaluation Misleads	74
Summary	78
3 - Advanced Text Processing	81
Tokenization and Normalization	82
Stemming and Lemmatization	87
Phrases and Compounds	92
Query Understanding	100
Intent Routing and Classification	107
Summary	113

Multimedia Retrieval

About this book

This is the companion textbook for **Multimedia Retrieval**, a Master's-level course at the University of Basel, edition HS26. Course announcements, exercise submissions, and scheduling are managed through [ADAM](#) and [GitHub](#)

From Ctrl+F to AI Answers

Suppose you need to find all books about cats in a library. A simple string search (Ctrl+F for “cat”) scans every page until it finds a match. Even for a modest collection this is impractical, and the result is merely a Boolean filter: it tells you which books contain the string “cat”, but it cannot tell you which ones are *most relevant* to what you are looking for. For popular topics, you are left with a long list of matches, and have to manually browse through the results to find what you wanted.

Retrieval systems solve this problem in two steps. First, they organize information in advance through preprocessing and indexing: extracting features from documents, building data structures that enable fast lookup, and storing statistics that describe each document's content. Second, at query time they apply ranking models that score documents by their relevance to the query rather than simply filtering by presence or absence of a term.

Modern AI systems have made retrieval even more critical. Large language models can in theory process long documents, but doing so is expensive (costs scale with token count) and prone to hallucination when the model must rely on memorized knowledge. In practice, it is far more effective to first retrieve the relevant passages from a document collection and feed only those into the model's context. This pattern, retrieval-augmented generation, depends entirely on the quality of the retrieval step: if the wrong passages are retrieved, the generated answer will be wrong too.

The Semantic Gap

Consider searching for a picture of a cat. Nothing in the raw pixel values tells a machine that an image depicts a cat. The system sees a grid of numbers; the user asks a question about meaning. This mismatch between how humans describe what they want and how machines represent information is called the *semantic gap* (Figure 0.1).

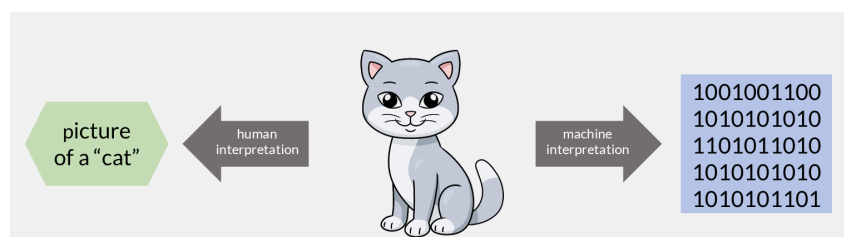


Figure 0.1: A human sees a “cat”; a machine sees a grid of binary values.

The gap exists in text too, though it is narrower. A search for “cat” will match many documents containing that word or its plural “cats”, but it will miss documents that discuss “felines” or mention specific breeds by name without ever using the word “cat”. In images, audio, and video the gap is far wider: there is no surface-level token to match against.

The semantic gap plagued earlier retrieval systems, especially those working with non-text media. Recent breakthroughs in AI, particularly dense embeddings and transformer models, have dramatically narrowed it by learning to map both queries and documents into a shared space where meaning, not surface form,

determines similarity. This book traces the full path from systems that match keywords to systems that understand meaning.

Closing the Gap

The semantic gap is not one problem but a spectrum of problems, and this book attacks them in order of increasing sophistication.

The first step is keyword matching: building an index so that a search for “cat” instantly finds every document containing that word, without scanning the entire collection. This alone closes the gap for queries where the user’s words appear verbatim in the target. But as we saw, many relevant documents use different vocabulary.

The second step is statistical ranking. Instead of treating every keyword match equally, we weight terms by how informative they are and score documents by how well they match the query overall. A document that mentions “cat” once in passing ranks lower than one dedicated to the topic. This narrows the gap further, but only within the space of shared vocabulary.

The third step is semantic matching. We represent both queries and documents as dense vectors in a shared space, trained so that “cat” and “feline” end up near each other. Now the system can find relevant documents even when no words overlap. This is where the gap narrows dramatically for text, and where non-text media (images, audio) first become searchable by meaning.

The final step is retrieval-augmented generation. Rather than returning a ranked list of documents, the system retrieves the most relevant passages and feeds them to a language model that synthesizes a direct answer. The gap between question and answer is closed end-to-end.

Each step builds on the previous one. A RAG pipeline still needs an index (step 1), still benefits from good ranking (step 2), and works best with semantic retrieval (step 3). This is why the book is structured as a cumulative progression rather than a menu of independent topics.

This progression was not designed in one stroke. It accumulated over six decades, as each generation of researchers added a new way to narrow the gap. The optional history below traces that path from the first computerized catalogues to today’s generative systems.

From card catalogues to transformers: six decades of retrieval (optional reading)

The 1960s saw the birth of computerized information retrieval. Calvin Mooers had coined the term “information retrieval” in 1950, but serious research began only in the 1960s. H. P. Luhn created KWIC (Key Word In Context) indexes, IBM developed the STAIRS system, and Gerard Salton began work on SMART at Cornell. Most systems remained experimental because few texts existed in machine-readable form, and retrieval was still largely a human-mediated activity (Figure 0.2).

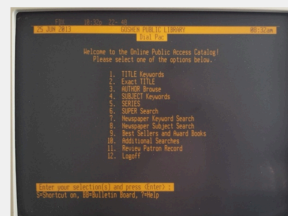


Figure 0.2: Library retrieval in the 1960s: a manual reference desk (left) beside an early terminal-based catalogue (right).

The 1970s brought the first practical systems. Widespread computer typesetting produced the machine-readable text needed for large-scale retrieval. Salton’s vector space model represented documents and queries as vectors whose similarity could be measured mathematically. Robertson and Sparck Jones developed the probabilistic framework that would later lead to BM25. Commercial systems like Lockheed Dialog and MEDLARS demonstrated that automated retrieval could work at scale (Figure 0.3).

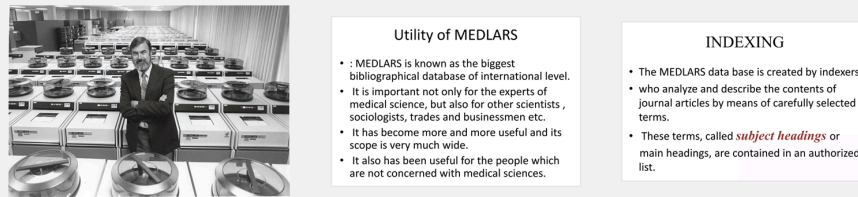


Figure 0.3: The MEDLARS system: an early large-scale bibliographic database running on reel-to-reel tape storage.

The 1980s refined these models. Boolean retrieval became the dominant commercial approach, letting users combine terms with AND, OR, and NOT (Figure 0.4). Robertson formalized the Probability Ranking Principle. The Okapi system at City University London gave its name to the BM25 ranking function. Latent Semantic Indexing appeared as a method to reveal hidden relationships between terms.



How to Search Using Boolean Operators:

Concept	Search Examples	Results
AND	politics AND media children AND poverty "civil war" AND Virginia	 Results will include both terms
OR	"law enforcement" OR police labor OR labour 60s OR sixties	 Results will include one or both terms
NOT	"civil war" NOT American Caribbean NOT Cuba therapy NOT physical	 Excludes results with the terms following NOT

Figure 0.4: Boolean operators (AND, OR, NOT) as set operations that select matching documents.

The 1990s brought the web revolution. Early search engines (Archie, WebCrawler, AltaVista) appeared to help users navigate the new online world (Figure 0.5). The most significant development was PageRank (Page and Brin, 1996), which ranked pages by the authority of incoming links rather than keyword matching alone.

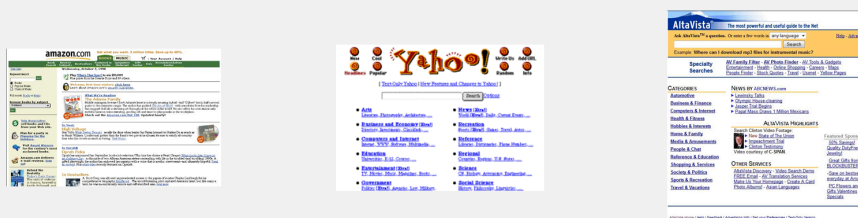


Figure 0.5: Early web interfaces circa 1998-1999, showing keyword search boxes and directory-style navigation.

The 2000s introduced machine learning into ranking. Learning to Rank (LTR) trained supervised models to order results by relevance. BM25 became a standard baseline. Deep learning re-emerged after a quiet period, driven by more powerful GPUs and larger datasets (Figure 0.6).

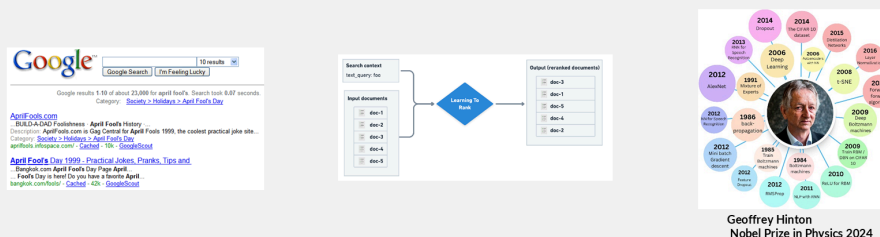


Figure 0.6: The 2000s: Google-era keyword search alongside the rise of Learning-to-Rank.

The 2010s saw deep learning applied directly to retrieval. Neural ranking models replaced handcrafted features with learned representations. Google introduced BERT in 2018, enabling search engines to understand context and word relationships. Transformer architectures fundamentally changed how machines handle sequential data (Figure 0.7).

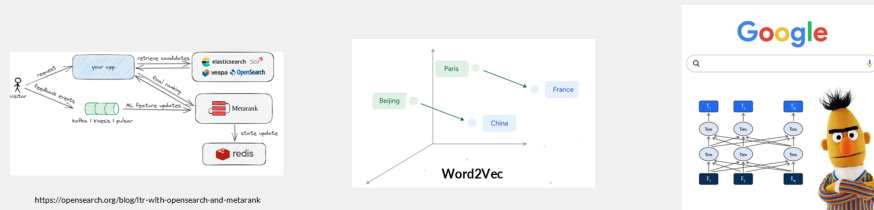


Figure 0.7: Neural ranking in the 2010s: a Learning-to-Rank pipeline (left), an embedding space (center), and contextual models like BERT (right).

The 2020s brought semantic search and vector databases as production infrastructure. Retrieval-Augmented Generation combines retrieval with large language models. Agentic systems can reason about information needs, execute multi-step retrieval, and synthesize results from multiple sources (Figure 0.8).

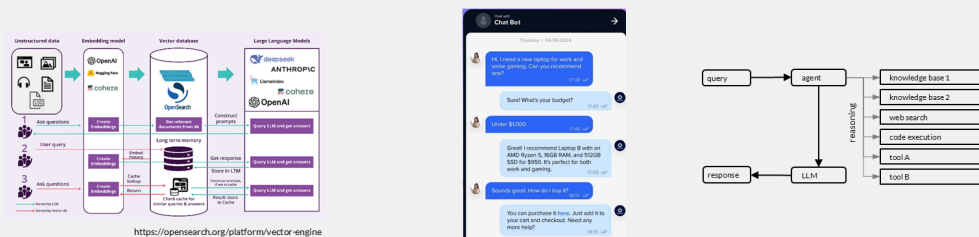


Figure 0.8: Three architectures for knowledge-grounded AI: a RAG pipeline with vector retrieval (left), a conversational interface (center), and an agentic system selecting among tools and knowledge sources (right).

How This Book Is Organized

The book is structured in three parts that build on each other. Each chapter focuses on one retrieval capability and the techniques to implement it. The path is cumulative: later chapters assume you have worked through the earlier ones.

Part I: Foundations (Chapters 1-3)

The first three chapters establish the fundamentals that every retrieval system relies on.

Chapter 1: Classical Text Retrieval introduces the core problem: given a query and a collection of documents, find the relevant ones. We build from Boolean retrieval (exact keyword matching) through TF-IDF weighting to BM25, the statistical ranking model that still powers production search engines today.

Chapter 2: Performance Evaluation asks the question every retrieval engineer must answer: does this system actually work? We cover precision, recall, ranked evaluation metrics (MAP, NDCG), and the experimental methodology for benchmarking retrieval systems.

Chapter 3: Advanced Text Processing looks at what happens before retrieval: how raw text is transformed into features that improve search quality. Tokenization, stemming, compound handling, query understanding, and intent classification.

Part II: Search Systems (Chapters 4-7)

The middle chapters move from individual techniques to complete systems.

Chapter 4: Index for Text Retrieval covers the data structures that make retrieval fast: inverted files, posting lists, compression, and how systems scale from thousands to billions of documents.

Chapter 5: Semantic Search introduces the shift from keyword matching to meaning matching. We trace the path from latent semantic indexing through word embeddings to transformer-based dense retrieval.

Chapter 6: Vector Search addresses the infrastructure challenge: once documents are represented as vectors, how do we find nearest neighbors efficiently among millions of embeddings? Approximate nearest neighbor algorithms, quantization, and vector databases.

Chapter 7: Retrieval-Augmented Generation combines retrieval with large language models. We cover chunking strategies, query transformation, retrieval pipelines, and how to build systems that generate answers grounded in retrieved evidence.

Part III: Advanced Topics (Chapters 8-12)

The final chapters extend retrieval beyond text.

Chapter 8: Web Search adds link analysis (PageRank, HITS) and web-specific ranking signals to the retrieval stack.

Chapter 9: Content Analysis covers structural and metadata-based features for multimedia documents.

Chapter 10: Visual Features applies retrieval to images: color histograms, texture descriptors, shape features, and modern deep visual features.

Chapter 11: Audio Features extends retrieval to audio: perceptual features, musical features, and fingerprinting for music recognition.

Chapter 12: Video Structural Features addresses the temporal dimension: shot detection, motion features, and how to search within video.

What to Expect in Each Chapter

Every chapter starts with a concrete scenario that motivates the problem, then develops the solution through four layers:

1. **Concepts and examples.** We introduce each technique through a running example with real data, so you can see what the method does before we formalize how it works.
2. **Formal foundations.** Key formulas and models are stated precisely, with plain-English intuition alongside the math. You will know both what to compute and why.
3. **Practical implementation.** Code examples show how techniques translate into working systems. Where relevant, we reference production tools (Lucene, FAISS, LangChain) so you can connect theory to practice.
4. **Hands-on notebooks.** Interactive demos let you run the techniques yourself, experiment with parameters, and observe how changes affect retrieval quality.
5. **Quiz questions.** Each chapter includes multiple-choice questions in the [Quiz App](#) to test your understanding and prepare for the exam.

Each chapter ends with a **summary** that recaps the most important concepts, formulas, and insights in condensed form, useful for review and exam preparation. Summaries also point to further reading for those who want to go deeper.

What You Will Be Able To Do

By the end of this book, you will be able to:

- Build a text retrieval system from scratch using an inverted index
- Evaluate retrieval quality with precision, recall, and ranking metrics
- Process raw text into features that improve search quality
- Design a semantic search system using dense embeddings
- Construct a RAG pipeline that answers questions from a document collection
- Apply retrieval techniques to non-text media: images, audio, and video
- Choose the right retrieval architecture for a given problem

We begin with the simplest useful system (keyword search over a small collection) and end with systems that search across media types and generate natural-language answers. Let us start.

Further Reading

These four sources trace the same four-step progression this book follows, from keyword matching to generation. Read them for depth beyond the chapter treatment.

- Manning, C. D., Raghavan, P., & Schütze, H. (2008). **Introduction to Information Retrieval**. Cambridge University Press. [Free online edition](#). The standard reference for classical text retrieval: indexing, Boolean and vector-space models, and evaluation. It underpins Chapters 1 to 4.
- Robertson, S., & Zaragoza, H. (2009). **The Probabilistic Relevance Framework: BM25 and Beyond**. *Foundations and Trends in Information Retrieval*, 3(4), 333-389. [PDF](#). The definitive account of BM25, the statistical ranking model that anchors the second step of the progression and still serves as the standard baseline.
- Lin, J., Nogueira, R., & Yates, A. (2021). **Pretrained Transformers for Text Ranking: BERT and Beyond**. *arXiv preprint*. [arXiv:2010.06467](#). A survey of how transformer models moved retrieval from surface matching to meaning, the third step covered in Chapter 5.
- Lewis, P., Perez, E., Piktus, A., et al. (2020). **Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks**. *Advances in Neural Information Processing Systems (NeurIPS)*. [arXiv:2005.11401](#). The paper that introduced RAG, the final step where retrieval feeds a language model to generate grounded answers, developed in Chapter 7.

1 - Classical Text Retrieval

Every day, billions of queries hit search engines, email clients, and document archives. The core challenge is always the same: given a short query, find the most relevant items in a large collection. This chapter introduces the classical methods that solve this problem for text.

Text retrieval was one of the earliest problems in computer science to receive serious attention. Beginning in the 1950s, researchers like Gerard Salton and Karen Spärck Jones developed methods that worked remarkably well. The reason is fundamental: queries and documents share the same vocabulary. When a user types “climate change policy”, the system can match those exact words in documents without needing to bridge different representations. This directness gave text retrieval a head start over image or audio search, where the gap between raw data and human meaning is far wider.

By the early 2000s, many considered text retrieval a solved problem. The core algorithms were mature, search engines worked well enough, and research attention shifted elsewhere. Recent developments in generative AI have changed the picture. Modern language models depend on precise, up-to-date text passages retrieved from external sources, and suddenly the quality of the retrieval stage matters more than ever. Classical text retrieval is no longer just the backbone of search engines. It is a critical component in modern AI systems.

We begin with Boolean filtering, the simplest approach, and trace the evolution toward ranked retrieval models that estimate how relevant each document is. Along the way, we develop the feature extraction pipeline that transforms raw text into searchable representations.

Figure 1.1 traces this six-decade evolution, from Salton’s SMART system through BM25 to its current role as the fast first-stage retriever in retrieval-augmented generation pipelines. The optional reading below describes each milestone.

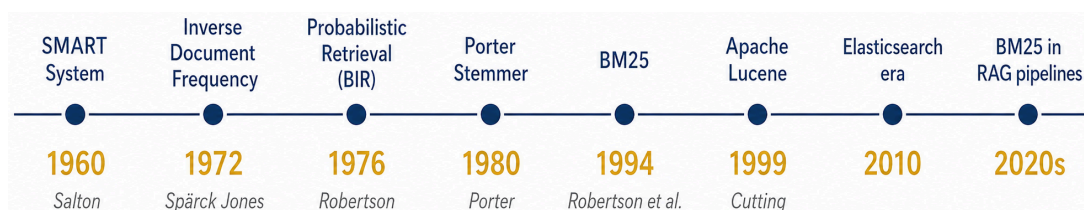


Figure 1.1: Evolution of classical text retrieval: key milestones, methods, and contributors from the 1960s to the present.

The milestones behind modern text search (optional reading)

- **1960 - SMART System (Salton):** Gerard Salton’s SMART system at Cornell was the first to treat documents and queries as vectors in a term space. It established the experimental methodology that the entire IR field still uses today.
- **1972 - Inverse Document Frequency (Spärck Jones):** Karen Spärck Jones proposed weighting terms by how rarely they appear across documents. This single idea remains the foundation of every term-weighting scheme in use, including BM25.
- **1976 - Probabilistic Retrieval / BIR (Robertson):** Stephen Robertson formalized retrieval as a probability estimation problem: how likely is this document to be relevant? This gave term weighting a theoretical grounding beyond heuristics.
- **1980 - Porter Stemmer (Porter):** Martin Porter published a simple suffix-stripping algorithm that reduced English words to common stems. Despite its errors, it became the de facto standard and is still the default stemmer in many search engines.
- **1994 - BM25 (Robertson et al.):** The Okapi BM25 ranking function combined saturating term frequency, document-length normalization, and probabilistic IDF into a single formula. It outperformed all prior models and remains the default in production search systems.

- **1999 - Apache Lucene (Cutting):** Doug Cutting released Lucene as open-source Java, making high-quality full-text search accessible to any developer. It became the engine underneath Solr, Elasticsearch, and OpenSearch.
- **2010 - Elasticsearch era:** Elasticsearch wrapped Lucene in a distributed, JSON-based API. Suddenly, BM25-powered search scaled to billions of documents with minimal configuration.
- **2020s - BM25 in RAG pipelines:** Large language models need retrieved context to answer questions accurately. BM25 serves as the fast first-stage retriever in Retrieval-Augmented Generation, proving that 30-year-old algorithms still earn their place in modern AI systems.

What you'll learn

- Explain how retrieval systems evolved from Boolean filtering to ranked retrieval (retriever-only arrow.r retriever-ranker)
- Describe the feature extraction pipeline: extract, split, tokenize, and summarize
- Compare document representations: set-of-words vs. bag-of-words vs. tf-idf weighted vectors
- Implement and contrast retrieval models: Boolean, Extended Boolean, Vector Space, Probabilistic (BIR), and BM25
- Evaluate term weighting using inverse document frequency and Zipf's law

Prerequisites

- Basic set theory and linear algebra (dot product, vector norms)
- Familiarity with logarithms and probability notation
- The concept of the semantic gap introduced in the opening chapter

Retrieval Systems and Flows

The need to search large text collections has its roots in library science. As described in the opening chapter, librarians and researchers relied on card catalogs, subject indexes, and classification systems for centuries. When collections outgrew manual methods, the first computerized retrieval systems automated what librarians had always done: match a user's query against a catalog of documents. Today, the same principles apply far beyond libraries, from legal databases and patent archives to corporate knowledge bases and web search engines.

Across these applications, a retrieval system must answer three connected questions: what unit should be returned, what work can be completed before a query arrives, and whether matching documents should be filtered or ranked. These decisions determine the system architecture and the flow from source documents to results.

From Filtering to Ranking

Consider a university library with millions of articles and books. A researcher enters terms like “neural plasticity AND memory consolidation”. The simplest architecture retrieves all documents whose catalog entry satisfies this Boolean expression and returns the matching set. This retriever-only approach can evaluate each document independently and return results as soon as it finds them. No scoring or sorting is required. Figure 1.2 illustrates this basic architecture.

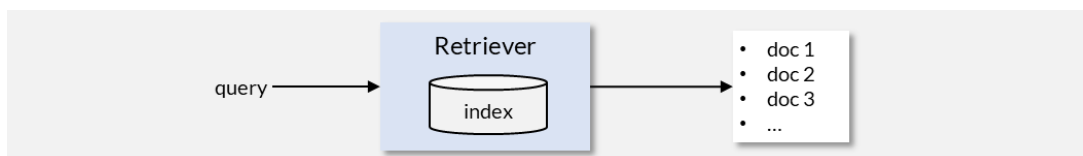


Figure 1.2: Retriever-only architecture: query enters, matching documents come out.

As the library's collection grows, a Boolean query like “neural plasticity” might return hundreds of articles. The researcher needs a way to narrow results without reformulating the query. A post-processing step adds **faceted search**: the ability to filter and sort results by metadata such as publication year, journal, or language. The researcher can toggle these filters while browsing without resubmitting the original search. Faceted search does not change which documents are considered relevant. It simply helps navigate large result sets. Figure 1.3 shows this extended pipeline.

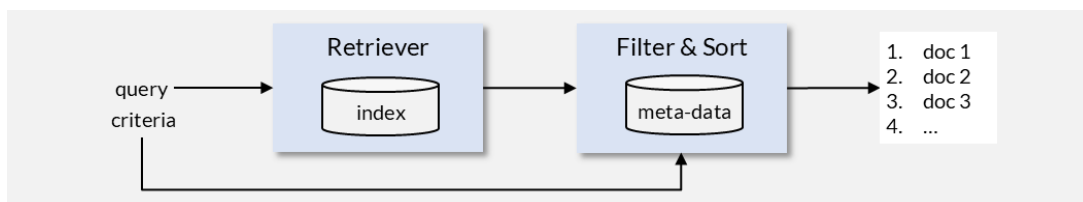


Figure 1.3: Retriever with faceted filtering: candidates are filtered and sorted by metadata.

Even with filters, an unranked list of 200 articles forces the researcher to scan them manually. In the 1970s, the Vector Space and Probabilistic retrieval models introduced a fundamentally different idea: instead of a binary relevant/not-relevant decision, the system estimates a degree of relevance and produces a ranked list. The most promising articles appear first. The architecture becomes a two-stage pipeline: a fast retriever selects candidates, and a ranker scores and orders them (Figure 1.4).

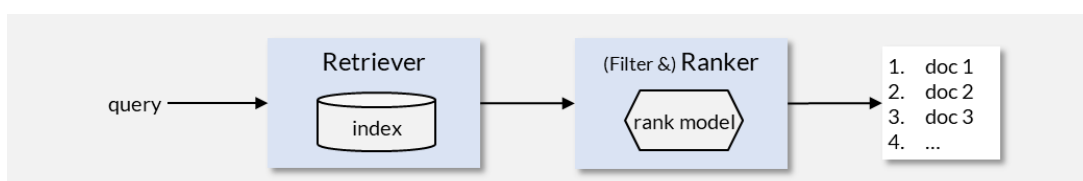


Figure 1.4: Retriever-ranker pipeline: a scoring model orders candidates by estimated relevance.

These three architectural patterns, retriever-only, retriever with faceted filtering, and retriever-ranker, represent the historical progression of text retrieval. The same patterns appear today in systems of all scales, from a researcher querying PubMed to a customer searching an e-commerce catalog.

Documents and Retrieval Granularity

Before we can search a collection, we need to define what a “document” is. In retrieval, a document is the unit that gets indexed, matched against queries, and returned to the user. It consists of three elements:

- **Document ID** - a unique identifier (ideally a UUID) for efficient storage and lookup.
- **Metadata attributes** - descriptive fields (author, publication date, language) used for filtering and faceted search, but not necessarily included in full-text matching.
- **Content attribute(s)** - the text fields indexed for search and ranking.

The critical design decision is: what constitutes a single retrieval unit?

Consider a pharmacology reference book with 800 pages. If we treat the entire book as one document, a search for “ibuprofen dosage” returns “the pharmacology textbook” and the user must search within the book manually. If instead we treat each section or page as a separate document, the system can return “page 47: Ibuprofen - Dosage and Administration” directly.

This decision about retrieval granularity defines the collection structure. Finer granularity gives more specific results but increases the number of entries the system must manage. Coarser granularity reduces index size but forces users to locate relevant passages themselves.

For classical text retrieval, the choice is typically straightforward: emails are individual documents, web pages are individual documents, book chapters or sections become individual documents. The retrieval models in this chapter assume that this splitting has already occurred and each document in the collection is a coherent, self-contained unit. We describe the concrete splitting strategies in the [feature extraction pipeline](#).

Note

Choosing the right retrieval granularity becomes significantly more complex when building systems that feed passages to language models. We revisit chunking strategies, overlapping windows, and hierarchical approaches in the chapter on Retrieval-Augmented Generation.

Offline Phase: Indexing Pipeline

Many search systems scan through data at query time. File search on a local computer, for example, reads through every file on disk whenever the user types a query. This works for a personal laptop with a few thousand files but not for a library with millions of articles. Instead, retrieval is split into two phases: an offline indexing phase that processes documents before any query arrives, and an online querying phase that handles user requests in real time.

The offline phase processes documents through a sequence of stages before any query arrives ([Figure 1.5](#)): (a) ingest raw documents from their source formats, (b) split them into retrieval units at the chosen granularity, (c) identify metadata attributes for filtering, (d) extract and normalize text into feature vectors, and (e) store these vectors in a searchable index.

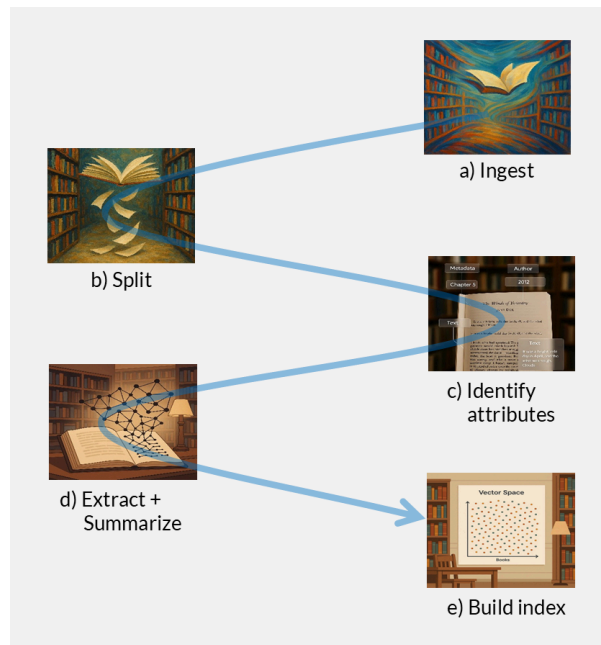


Figure 1.5: Document preprocessing pipeline: ingest (a), split (b), identify attributes (c), extract features (d), build index (e).

Online Phase: Query Processing

When a researcher types “neural plasticity AND memory consolidation” into the library search, the online phase begins. The system processes the query through a symmetric sequence of stages (Figure 1.6): (a) accept the user’s query, (b) tokenize it using the same pipeline that processed documents during indexing, (c) optionally broaden the query through spelling correction or synonym expansion, (d) compare the query’s feature vector against the vectors stored in the index to find similar documents, and (e) rank candidates by estimated relevance and return an ordered result list.

This symmetry between offline and online processing is essential: queries and documents must live in the same feature space for comparison to work.

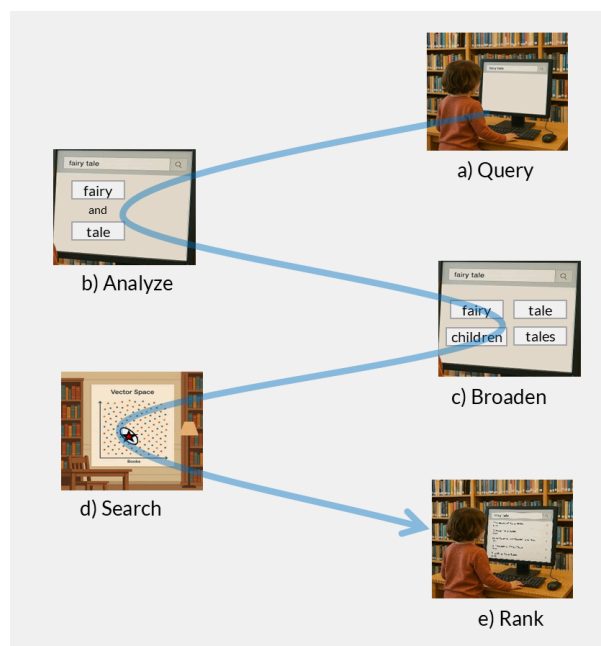


Figure 1.6: Online query pipeline: query (a), tokenize (b), expand (c), search (d), rank (e).

The central challenge in the online phase is relevance ranking: accurately estimating how important a document is given the query. The next sections develop the feature extraction pipeline that creates document representations, followed by the retrieval models that use those representations for scoring.

Hands-on: Explore a Document Collection

Load a document collection, inspect its structure, and understand the raw material that retrieval systems work with.

[Open notebook ->](#)

From Text to Searchable Vectors

Retrieval models cannot operate directly on PDF files, HTML markup, or character streams. They compare structured representations of documents and queries. The feature extraction pipeline shown in Figure 1.7 creates these representations through five conceptual steps: extract text from the source format, split it into retrieval units, tokenize and normalize the text, construct a vocabulary, and summarize each unit as a feature vector. This produces two key artifacts:

- **Index** - high-dimensional feature vectors for every retrieval unit, stored together with source metadata.
- **Vocabulary** - the complete set of normalized terms used for both document and query processing.

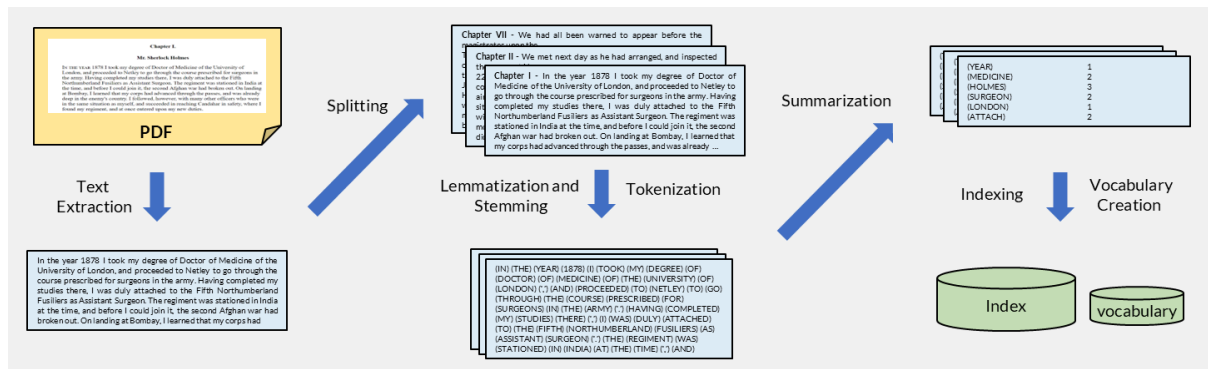


Figure 1.7: Text indexing pipeline: extract, split, tokenize, stem, summarize, then build the index and vocabulary.

Text Extraction

The first step extracts plain text and metadata from the source format. Documents arrive in many formats: PDF, EPUB, plain text, HTML, or office documents (DOCX, PPTX). Each format encodes content differently, mixing actual text with layout instructions, styling, and structural markup. The extraction step strips formatting and control sequences, isolates the character stream, and records metadata attributes (author, title, date) for later use in filtering.

Consider the HTML example in Figure 1.8. The header holds structured metadata (title, keywords) that can enrich the index entry, while the body contains the main text interleaved with markup tags. Extracting useful text from HTML requires parsing the DOM tree and distinguishing content nodes from navigation, scripts, and boilerplate. Although HTML follows a well-defined standard, real-world pages use diverse layouts, making robust extraction (often called scraping) a non-trivial engineering problem.

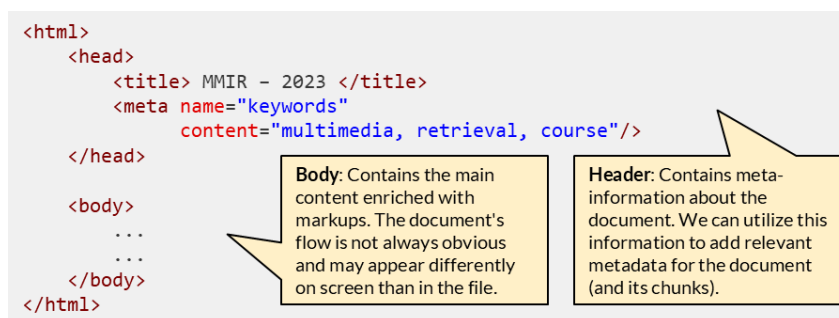


Figure 1.8: HTML document structure: metadata in the header, content in the body.

Source formats differ, so extraction normally relies on format-specific parsers and established libraries. The retrieval pipeline itself only requires that this stage return clean text and useful metadata; the implementation details can be selected for the collection at hand.

Source formats and extraction tools (optional reading)

Different source formats require different extraction strategies:

- **PDF:** Text is stored as positioned glyphs, not logical paragraphs. Extraction must reconstruct reading order from coordinates. Tables and multi-column layouts are particularly challenging.
- **Office documents (DOCX, PPTX):** ZIP archives containing XML. Libraries can parse the XML structure to extract text, but embedded images and charts require separate handling.
- **HTML/XML:** Well-structured but noisy. Useful text must be separated from navigation menus, advertisements, and script blocks.
- **Plain text and Markdown:** Minimal transformation needed, but encoding detection (UTF-8 vs. legacy encodings) and line-break conventions still require attention.

In practice, we rarely build extraction from scratch. Modern extraction pipelines rely on established libraries and frameworks:

- **Apache Tika:** Java-based toolkit from the Apache Lucene ecosystem that auto-detects file types and extracts text and metadata from over 1,000 formats (PDF, DOCX, EPUB, email archives, images via OCR). Commonly used as the ingestion layer in Solr and Elasticsearch pipelines.
- **Unstructured** (unstructured on PyPI): Open-source Python library designed for LLM/RAG pipelines. Handles PDFs, HTML, images, and office documents with built-in chunking support.
- **MarkItDown** (markitdown on PyPI): Open-source Python library from Microsoft that converts PDF, DOCX, PPTX, XLSX, HTML, CSV, JSON, images, and audio into clean Markdown. Lightweight and popular for RAG preprocessing.
- **Trafilatura:** Focused on web content extraction. Strips boilerplate from HTML pages and returns clean article text.
- **PyMuPDF (fitz):** Fast PDF text and layout extraction in Python, preserving reading order and table structure.
- **BeautifulSoup / lxml:** HTML/XML parsers for custom extraction logic when off-the-shelf tools fall short.

The choice of extraction tool depends on the document mix in the collection, the required fidelity (do you need tables? images? footnotes?), and whether the pipeline must run at scale.

At this point, we must also decide on a character encoding for the index. UTF-8 is the dominant standard and handles virtually all scripts. Legacy collections may contain documents in older encodings (Latin-1, Shift-JIS) that require detection and conversion before indexing.

Web documents and link structure

HTML documents carry additional structure beyond text: anchor texts, heading hierarchies, and hyper-link networks. These provide valuable signals for retrieval but require specialized treatment. We cover HTML-specific extraction and link-based relevance (including PageRank) in the chapter on Web Search.

Splitting

As discussed in the previous section, retrieval granularity determines what constitutes a single retrieval unit. Once we have extracted the raw text from a document, we must split it into the units that will be indexed and returned to the user. In classical text retrieval, the splitting strategies are pragmatic and structure-driven:

- **By document boundary:** Each file, email, or web page becomes one retrieval unit. This is the most common approach when documents are naturally self-contained.
- **By structural element:** Books are split at chapter or section headings. Legal texts are split by article or paragraph number. The document's own structure defines the boundaries.
- **By line or log entry:** In observability systems like Elasticsearch, each log line becomes an independent retrieval unit. This enables searching millions of events by timestamp, severity, or content.
- **By fixed page or paragraph count:** When no structural markup is available (e.g., scanned documents), we split at regular intervals such as every N paragraphs or every page.

These approaches share a common property: the splitting decision is made once during indexing and remains fixed. Each resulting unit is treated as an independent document by the retrieval models that follow. [Figure 1.9](#) illustrates this for a novel split into chapter-sized retrieval units.

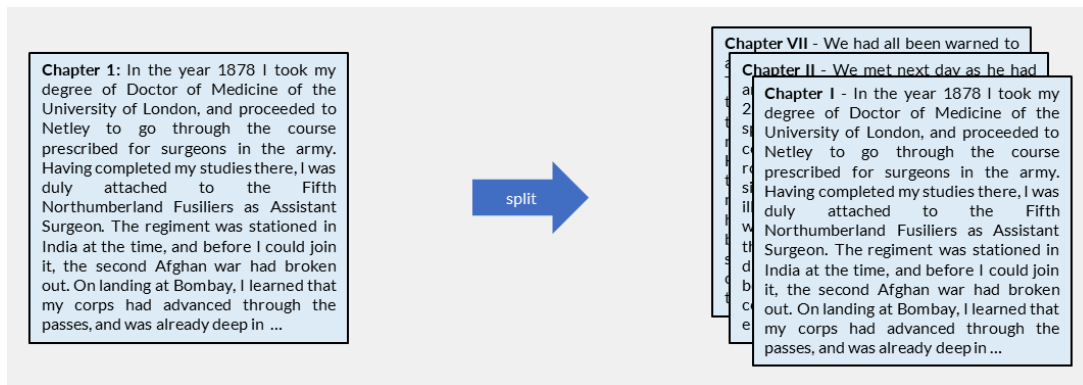


Figure 1.9: A single document divided into multiple smaller retrieval units.

Advanced chunking strategies for RAG

When building systems that feed passages to language models, splitting becomes significantly more nuanced. Overlapping windows, semantic chunking, and hierarchical approaches are covered in the chapter on Retrieval-Augmented Generation.

Tokenization

Tokenization converts the extracted character stream into a sequence of discrete units called tokens. A **token** is an individual element in this sequence: it can be a word, a number, or a punctuation mark. A **term** is the normalized form of a token after processing (stemming, case folding). In classical text retrieval, tokens are typically whole words, and the two concepts are often used interchangeably. In later chapters, we will see that this equivalence breaks down when tokens become subword fragments or multi-word phrases.

For classical retrieval, we split text at whitespace and punctuation boundaries to produce word-level tokens. This requires decisions about edge cases: how to handle hyphenated words (“state-of-the-art”), numbers (“3.14”), abbreviations (“U.S.A.”), and possessives (“Watson’s”). In languages without explicit word boundaries (e.g., Japanese, Chinese), segmentation requires dictionary-based or statistical methods.

The most significant challenge is variation in word forms. Since retrieval models match documents to queries by comparing tokens, two tokens are either identical or unrelated. There is no notion of “almost the same”. A query for “cats” will not match a document containing only “cat” because these are distinct tokens. Similarly, “go”, “goes”, “went”, and “going” are four independent dimensions in the vocabulary despite sharing a meaning. The next subsection addresses this through stemming and lemmatization, which reduce inflected forms to a common token before indexing. Figure 1.10 shows the tokenization step applied to a document collection.



Figure 1.10: Tokenization: raw text split into individual word-level tokens.

Beyond word-level tokens

Subword tokenization (BPE, WordPiece) and n-gram approaches are covered in the chapter on Advanced Text Processing. Word and sentence embeddings, which map tokens to dense vectors, are introduced in the chapter on Semantic Search.

Lemmatization and Stemming

The previous subsection established that retrieval models treat each token as an independent symbol: “cat” and “cats” are unrelated unless we normalize them to the same form. The goal of this pipeline stage is to reduce surface variation so that semantically equivalent word forms map to a single canonical token in the vocabulary.

Two approaches exist:

- **Stemming** applies rule-based suffix stripping to produce a common pseudo-stem. The stem is not necessarily a real word (e.g., “computing”, “computation”, “computer” all reduce to “comput”). Stemming is fast, requires no dictionary, and works well enough for most retrieval tasks. Its errors go in both directions: it can merge unrelated words (“universal” and “university” both stem to “univers”) or fail to merge related ones (“alumnus” and “alumni” remain distinct).
- **Lemmatization** uses a dictionary or morphological analysis to map each word to its actual root form (the lemma). “Better” maps to “good”, “went” maps to “go”. This produces more accurate results but requires language-specific resources and is computationally more expensive. Modern NLP libraries like spaCy provide lemmatization out of the box.

For classical text retrieval, stemming is the standard choice due to its speed and simplicity. The most widely used stemmer for English is the Porter Algorithm, which we examine next.

Beyond inflection: compounds, synonyms, and semantic relations

Some languages form compound words of arbitrary length (e.g., German, Finnish), creating additional matching challenges. Compound decomposition, synonym expansion, and other linguistic transformations are covered in the chapter on Advanced Text Processing.

The Porter Algorithm

The most well-known rule-based stemmer for English is the Porter Algorithm (1980); see [An Algorithm for Suffix Stripping](#) in the Further Reading section of the chapter summary. It applies an ordered sequence of suffix-stripping rules to produce a shared pseudo-stem. For the retrieval pipeline, the important result is that related forms map to the same vocabulary term; the exact rule sequence is secondary.

Consider these examples:

Input	Porter stem	True root
“computing”, “computation”, “computer”	comput	compute
“retrieval”, “retrieving”, “retrieved”	retriev	retrieve
“generalization”, “generalizing”	general	generalize
“relational”	relat	relate
“agreed”, “agreeable”	agre	agree

The stems “comput”, “retriev”, and “agre” are not dictionary words. They exist only to ensure that related forms map to the same token during indexing. When a user queries “computation”, the system stems it to “comput” and matches documents containing “computing” or “computer” because those were also stemmed to “comput”.

This approach is fast (roughly 60 rules in about 400 lines of code), requires no dictionary, and works well enough for most English retrieval tasks. However, it is purely mechanical and makes errors in both directions: over-stemming merges words that should remain distinct (“operate” and “operating” correctly merge, but “operation” and “operational” may conflate with “opera”), while under-stemming fails to merge irregular forms (“be”, “was”, “been” remain three separate tokens).

Note

The internal mechanics of Porter are covered in detail in the chapter on Advanced Text Processing, along with modern alternatives like Snowball and dictionary-based lemmatization.

Vocabulary

After tokenization and stemming, we have a set of normalized tokens across the entire collection. The **vocabulary** is the complete set of distinct terms that appear at least once. Each term becomes a dimension in the feature space used for retrieval. But not all terms are equally useful.

Stop Words

Many terms are grammatically necessary but carry no content. The article “the” appears in virtually every English document but tells us nothing about what a document is about. A search for “the” would return the entire collection, unable to differentiate between relevant and non-relevant results. Figure 1.11 shows that the 50 most frequent terms in a news corpus account for roughly one-third of all occurrences and appear in over 60% of documents.

t_j	$df(t_j)$	$tf(D, t_j)$						
the	100%	6.03%	from	90%	0.44%	when	74%	0.24%
a	99%	2.57%	be	88%	0.46%	their	71%	0.27%
of	99%	2.55%	an	88%	0.38%	his	70%	0.49%
and	99%	2.40%	have	88%	0.45%	had	70%	0.29%
to	99%	2.58%	was	85%	0.65%	been	70%	0.21%
in	99%	2.00%	not	84%	0.38%	all	69%	0.20%
for	98%	0.97%	this	83%	0.35%	which	69%	0.20%
that	96%	1.15%	are	83%	0.45%	will	68%	0.27%
on	96%	0.75%	has	83%	0.40%	out	68%	0.20%
is	95%	0.93%	who	81%	0.37%	up	68%	0.20%
with	95%	0.70%	they	78%	0.37%	if	67%	0.21%
at	93%	0.55%	he	78%	0.70%	than	66%	0.18%
by	92%	0.49%	one	77%	0.26%	were	66%	0.22%
it	92%	0.69%	said	77%	0.70%	would	65%	0.23%
as	91%	0.57%	more	75%	0.26%	can	65%	0.20%
but	91%	0.49%	about	75%	0.27%	new	64%	0.23%
			or	75%	0.31%	there	64%	0.18%

Figure 1.11: Document frequency and collection term frequency for the 50 most frequent terms in a news corpus (~20,000 documents).

Stop word lists for most languages are readily available, for example on [Kaggle \(stop words in 28 languages\)](#). However, eliminating stop words entirely creates problems. Consider the search for “it”: if we remove this term, we lose the ability to find IT books or the novel “It” by Stephen King. The modern approach retains all terms but assigns them different weights based on their discriminating power.

Zipf’s Law

The distribution of term frequencies follows a remarkably regular pattern. Let N be the total number of token occurrences in the collection and M the number of distinct terms. If we rank terms by decreasing frequency, Zipf’s law states that the probability of encountering the term at rank r is:

$$p_r = \frac{c}{r} = \frac{tf(t)}{N}, \quad \text{term } t \text{ with rank}(t) = r \quad (1.1)$$

The constant c depends only on the vocabulary size:

$$c = \frac{1}{\sum_{r=1}^M \frac{1}{r}} \approx \frac{1}{0.5772 + \ln M} \quad (1.2)$$

In other words, a few terms dominate the collection while the vast majority are rare.

For a collection with $M = 5,000$ terms, $c \approx 0.11$; with $M = 100,000$, $c \approx 0.08$. The practical implication is shown in Figure 1.12: terms fall into three zones. The most frequent terms (above the upper cut-off) appear everywhere and cannot discriminate. The rarest terms (below the lower cut-off) are highly specific but unlikely to occur in queries, making them less useful despite their discriminating potential. The terms most useful for distinguishing relevant from non-relevant documents lie in between.

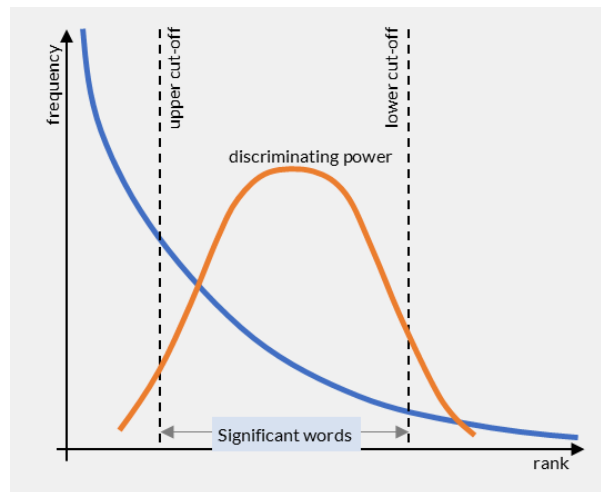


Figure 1.12: Zipf's law and term discrimination: the most useful terms for retrieval fall between the frequency extremes.

Term Discrimination and IDF

The intuition behind term weighting is straightforward: imagine removing a single term from every document in the collection. If documents suddenly become harder to tell apart, that term was helping to discriminate between them. Conversely, removing a term that appears everywhere (like “the”) changes nothing. Terms that appear in a moderate number of documents have the highest discriminating power, as confirmed empirically in Figure 1.13.

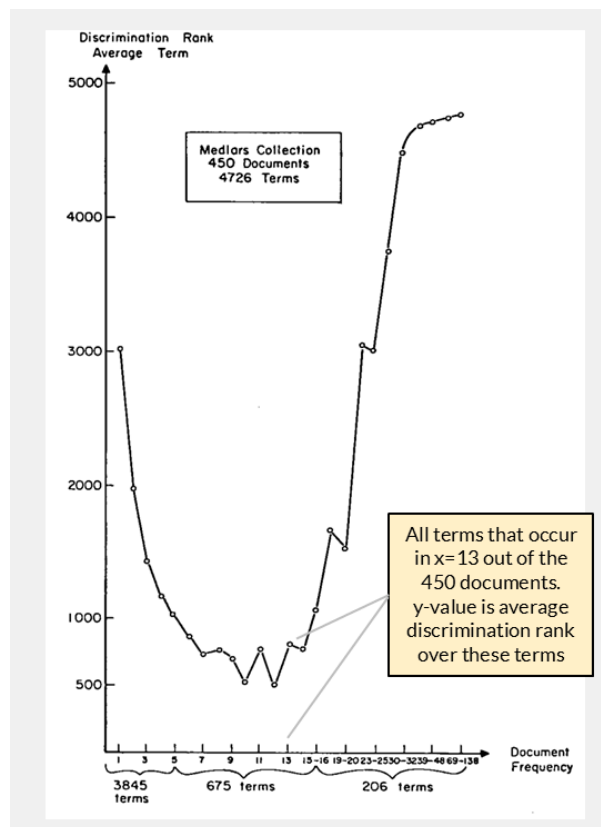


Figure 1.13: Discrimination rank vs. document frequency in the Medlars collection (450 documents). Terms occurring in 9-12 documents have the highest discrimination rank. Source: Salton, Wong & Yang (1975).

Karen Spärck Jones (1972) captured this intuition in a single formula called the **inverse document frequency** (IDF); see [A Statistical Interpretation of Term Specificity and Its Application in Retrieval](#) in the Further Reading section of the chapter summary. It builds on one statistic: the **document frequency** $df(t)$, which counts how many documents contain term t at least once. IDF then weights terms inversely to their commonness:

Key Formula: Inverse Document Frequency

$$\text{idf}(t) = \log \frac{N + 1}{\text{df}(t) + 1} \quad (1.3)$$

Terms that appear in fewer documents receive higher IDF weights. The “+1” in both numerator and denominator is a smoothing term that avoids division by zero (for terms not yet seen) and ensures that a term appearing in every document still receives a small positive weight rather than exactly zero.

Example

Consider a collection of $N = 10,000$ documents. We compute IDF for four terms:

Term	df(t)	$\text{idf}(t) = \log \frac{10,001}{\text{df}+1}$	Interpretation
“the”	9,800	$\log \frac{10,001}{9,801} \approx 0.009$	Appears almost everywhere, nearly zero weight
“retrieval”	50	$\log \frac{10,001}{51} \approx 2.29$	Moderately rare, high weight
“okapi”	3	$\log \frac{10,001}{4} \approx 3.40$	Very rare, highest weight
“algorithm”	1,200	$\log \frac{10,001}{1,201} \approx 0.92$	Common but not ubiquitous, moderate weight

A query “retrieval algorithm” would weight “retrieval” roughly $2.5\times$ more than “algorithm” because it appears in far fewer documents. The stop word “the” contributes almost nothing even if present in the query.

IDF provides the weighting mechanism we need: common terms get low weights, rare terms get high weights. Combined with term frequency, it produces the **tf-idf** weighting that we will use for some of the retrieval models in the next sections. Figure 1.14 compares IDF weights with empirical discrimination power, confirming that IDF closely approximates the true discriminating value of terms.

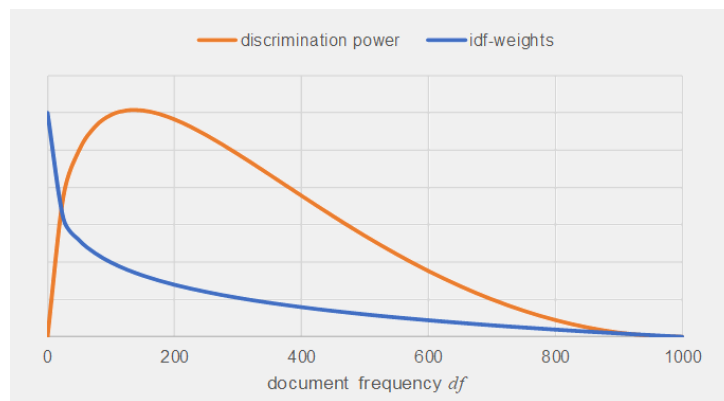


Figure 1.14: IDF weights closely track empirical discrimination power as a function of document frequency ($N = 1,000$).

Document Representation

With the vocabulary established and each term assigned an IDF weight, we can now represent documents as vectors. Each document becomes a point in an M -dimensional space where M is the vocabulary size and each dimension corresponds to one term. Figure 1.15 shows this process: tokenized text is stemmed, stop words are removed, and the remaining terms are counted to produce per-document frequency vectors.

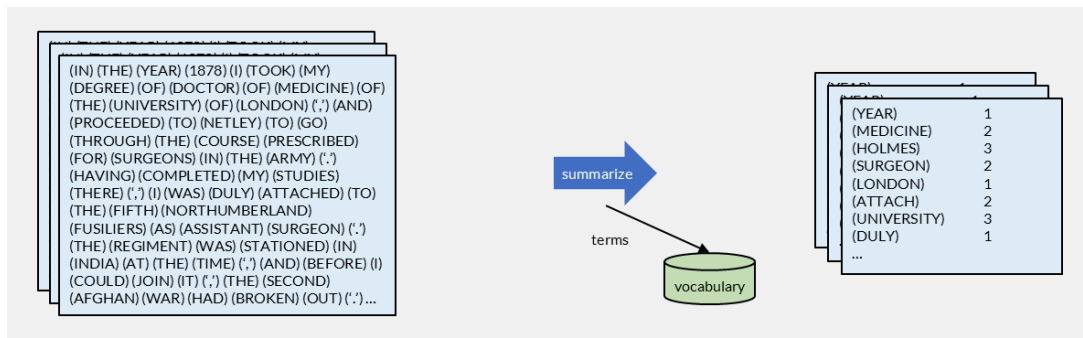


Figure 1.15: From tokenized text to term-frequency vectors: stem, remove stop words, count occurrences per document.

Set-of-Words and Bag-of-Words

Let D_i be the i -th document in a collection of N documents. The feature representation $\mathbf{d}_i \in \mathbb{R}^M$ is a vector where component $d_{i,j}$ describes how term t_j occurs in document D_i . We use $\text{tf}(D_i, t_j)$ to denote the number of occurrences of term t_j in document D_i .

The simplest option is the **set-of-words** model: for each term, record only whether it is present (1) or absent (0). A document mentioning “cat” three times looks identical to one mentioning it once:

$$d_{i,j} = \begin{cases} 1 & \text{if term } t_j \text{ is present in } D_i \\ 0 & \text{otherwise} \end{cases} \quad (1.4)$$

A richer option is the **bag-of-words** model: record how many times each term appears. A document mentioning “cat” five times is represented differently from one mentioning it once:

$$d_{i,j} = \text{tf}(D_i, t_j) \quad (1.5)$$

Both models ignore term proximity and order. A document containing “New York” is represented the same way as one containing “York New”. Despite this limitation, these representations work remarkably well because the presence and frequency of specific terms are strong signals of relevance.

TF-IDF Weighting

The bag-of-words model treats all terms equally: a document with 10 occurrences of “the” scores the same as one with 10 occurrences of “algorithm”. By combining term frequency with IDF, we produce weighted vectors where informative terms contribute more:

Key Formula: tf-idf document representation

$$d_{i,j} = \text{tf}(D_i, t_j) \cdot \text{idf}(t_j) \quad (1.6)$$

A term’s weight grows with how often it occurs in the document and how rare it is across the collection, so terms that are frequent here but uncommon elsewhere dominate the representation. This **tf-idf** weighting is the standard document representation for classical retrieval models.

Sparsity

In practice, a vocabulary can include millions of terms. However, most documents contain only a few hundred or thousand unique terms. The feature vectors are therefore sparsely populated: the vast majority of components are zero. Efficient storage methods like the inverted file exploit this sparsity by recording only the non-zero entries. During retrieval, the system only needs to consider documents that share at least one term with the query.

Hands-on: Feature Extraction

Build a feature extraction pipeline in Python: tokenize text, apply stemming, compute term frequencies and IDF weights.

[Open notebook ->](#)

Boolean Retrieval

The previous section transformed raw text into tokens and term-frequency vectors. A retrieval model takes these representations and decides which documents match a query and, for ranked models, how strongly each document matches. We use the following small fictional library catalogue as a running example. Its wording is deliberately controlled so that the effects of preprocessing, term weighting, document length, and lexical ambiguity remain visible.

ID	Catalogue entry
D_1	The Cat and Dog in the Forest. A cat and a dog begin a woodland adventure.
D_2	Cats of the Woodland. Wild cats begin an adventure beneath ancient trees.
D_3	The Forest Hound. A loyal dog follows a woodland trail through ancient trees.
D_4	Feline Detective. A clever cat solves mysteries with a canine companion.
D_5	Random Forest for Pet Detection. A random forest model classifies cats and dogs in photographs.
D_6	Forests of Search Trees. Algorithms explore a forest of binary trees and graph paths.
D_7	Dog Dog Dog! A dog chases a ball, finds a bone, and wakes the neighbors.
D_8	Forest Forest Forest! A forest guide names trees, flowers, rivers, birds, and hidden ruins.
D_9	Cat Dog Forest.
D_{10}	Cat Dog Forest. A cat meets a dog in a forest beside rivers, mountains, castles, villages, bridges, and caves.
D_{11}	Woodland Companions. A kitten and a puppy share a moonlit adventure among old trees.
D_{12}	The Pet Bakery. A cat, a dog, and a baker make cakes, bread, biscuits, pies, and coffee.

Following [From Text to Searchable Vectors](#), we must define the preprocessing pipeline before applying a retrieval model. For this example, we lowercase the text, split it at punctuation and whitespace, and remove common stop words. We deliberately apply neither stemming nor lemmatization, and we do not expand synonyms. For example:

$D_1 \mapsto [\text{cat}, \text{dog}, \text{forest}, \text{cat}, \text{dog}, \text{begin}, \text{woodland}, \text{adventure}]$

$D_2 \mapsto [\text{cats}, \text{woodland}, \text{wild}, \text{cats}, \text{begin}, \text{adventure}, \text{beneath}, \text{ancient}, \text{trees}]$

These choices expose an important property of lexical matching: terms match by identity, not by visual or semantic similarity. During vocabulary construction, each distinct token receives its own identifier. For example, the vocabulary might map “cat” to token ID 70 and “cats” to token ID 83. A retrieval model therefore treats them as two unrelated dimensions, even though a reader immediately recognizes their relationship. For the same reason, “cat” does not match “feline” or “kitten”, and “dog” does not match “hound”, “canine”, or “puppy”. The reverse problem occurs with “forest”: the same token ID represents both a natural environment and the technical concept in “random forest”, although the meanings differ.

Standard Boolean Model

Consider the query `cat AND dog`. For each document, we test two conditions: the token “cat” is present, and the token “dog” is present. The complete predicate evaluates to true only if both conditions are met. The result set is therefore $\{D_1, D_9, D_{10}, D_{12}\}$. For `cat OR dog`, the predicate evaluates to true if at least one condition is met, producing the larger result set $\{D_1, D_3, D_4, D_7, D_9, D_{10}, D_{12}\}$.

This is the central idea of the Standard Boolean Model: a query is a predicate that maps each document to either true or false. Documents for which the predicate is true enter the result set; all others are excluded. The model filters rather than ranks, so every returned document has the same status regardless of how often or where the terms occur. This makes Boolean retrieval simple to understand, easy to calculate, and straightforward to explain.

Query Language

The query grammar defines two atomic predicates: a term is present, or a term is absent. It then defines two composition rules: AND and OR combine existing predicates into more complex expressions. Formally:

$$Q = t \quad (\text{term } t \text{ must be present}) \quad (1.7)$$

$$Q = \neg t \quad (\text{term } t \text{ must be absent}) \quad (1.8)$$

$$Q = Q_1 \vee Q_2 \quad (\text{at least one sub-query must be true}) \quad (1.9)$$

$$Q = Q_1 \wedge Q_2 \quad (\text{both sub-queries must be true}) \quad (1.10)$$

For example, $(\text{cat} \wedge \text{dog}) \vee \text{hound}$ returns a document if it contains both “cat” and “dog”, or if it contains “hound”. Boolean algebra allows any such expression to be rewritten in disjunctive normal form (DNF):

Key Formula: Boolean Query in DNF

$$Q = \bigvee_{l=1}^L \left(\bigwedge_{k=1}^{K_l} \tau_{l,k} \right) \quad (1.11)$$

with $\tau_{l,k} = t_{j(l,k)}$ or $\tau_{l,k} = \neg t_{j(l,k)}$, where $j(l,k)$ maps to the index of the term used in the query.

Any Boolean query can be rewritten as a disjunction (OR) of conjunctions (AND). This normal form enables systematic evaluation using set operations.

Evaluating a Query

The direct evaluation method follows the initial intuition. For each document D_i , the system checks whether every atomic predicate $\tau_{l,k}$ is true or false and then evaluates the complete expression. For $\text{cat} \wedge \text{dog}$, D_1 produces $\text{true} \wedge \text{true} = \text{true}$, whereas D_4 produces $\text{true} \wedge \text{false} = \text{false}$.

A second method starts from all documents that satisfy each atomic predicate. Let $\mathbb{S}_t$ denote the set of documents containing term t . For the running collection:

$$\mathbb{S}_{\text{cat}} = \{D_1, D_4, D_9, D_{10}, D_{12}\}$$

$$\mathbb{S}_{\text{dog}} = \{D_1, D_3, D_7, D_9, D_{10}, D_{12}\}$$

The documents satisfying $\text{cat} \wedge \text{dog}$ must belong to both sets. The documents satisfying $\text{cat} \vee \text{dog}$ may belong to either set:

$$\mathbb{S}_{\text{cat}} \cap \mathbb{S}_{\text{dog}} = \{D_1, D_9, D_{10}, D_{12}\}$$

$$\mathbb{S}_{\text{cat}} \cup \mathbb{S}_{\text{dog}} = \{D_1, D_3, D_4, D_7, D_9, D_{10}, D_{12}\}$$

More generally, each atomic predicate defines a set of matching documents:

$$\mathbb{S}_{l,k} =$$

The DNF structure translates directly into intersections for AND and unions for OR:

$$\mathbb{Q} = \bigcup_{l=1}^L \bigcap_{k=1}^{K_l} \mathbb{S}_{l,k}$$

An inverted file stores these term-document sets so that the system can combine them without scanning every document. Chapter 4 develops this index structure and its efficient query-processing algorithms.

Limitations of Boolean Retrieval

Suppose the information need is a story involving a cat, a dog, and trees. The strict query $\text{cat} \wedge \text{dog} \wedge \text{trees}$ returns no documents. Several entries are plausible partial matches, but none contains all three exact tokens. For example, D_3 contains “dog” and “trees”, while D_1 contains “cat”, “dog”, “forest”, and “woodland”. Boolean evaluation excludes both because a missing term makes the complete AND predicate false.

Relaxing the query to $\text{cat} \vee \text{dog} \vee \text{trees}$ creates the opposite problem: it returns 11 of the 12 documents. The result includes technical entries such as D_6 because it contains “trees”, even though these are binary

search trees rather than trees in an animal story. Small changes to a Boolean expression can therefore produce abrupt changes in result-set size, from no results to almost the entire collection.

The model also provides no basis for ordering the matches. For `cat AND dog`, D_1 , D_9 , D_{10} , and D_{12} are equivalent Boolean matches. The model ignores that D_1 and D_{10} repeat both query terms, that D_9 consists only of the three topical terms “cat”, “dog”, and “forest”, and that much of D_{12} concerns baking. Term frequency, term importance, document length, and partial evidence do not affect the result.

These limitations establish the questions for the models that follow. The Extended Boolean Model introduces graded matching and ranking while retaining Boolean query structure. Probabilistic and vector-space models provide alternative foundations for term weighting and ranking, while BM25 later combines probabilistic term importance with term-frequency saturation and document-length normalization. We will return to the same collection to see which limitation each model addresses.

Advantages: The model has simple and precise query semantics. Its decisions are easy to calculate and explain, and set operations support fast evaluation over large collections. Boolean predicates also combine naturally with structured metadata filters such as `language = English`.

Disadvantages: Users must express their information need as a Boolean expression and may obtain either too few or too many results. The model neither ranks matches nor represents partial matches. All predicates have equal influence, regardless of term frequency or discriminating power. These properties make the model better suited to exact filtering than to ranking documents by relevance.

Boolean retrieval nevertheless remains an important component of modern text and web search systems. A fast first stage often combines query terms with OR to retrieve a broad candidate set. A second stage then assigns relevance scores and ranks these candidates, allowing the user to inspect the strongest matches first. The next model takes an initial step in this direction by replacing Boolean decisions with graded scores.

Extended Boolean Model

The query `cat AND dog` gave D_1 , D_9 , D_{10} , and D_{12} the same Boolean status. Yet their evidence differs: D_1 and D_{10} contain both terms twice, while D_9 and D_{12} contain them once. Documents such as D_3 and D_4 contain only one query term and are excluded completely. The Extended Boolean Model replaces this hard boundary with a graded score. It can rank exact matches and retain partial matches with lower scores.

What Remains Boolean, and What Changes?

The query language remains the same. Users still express requirements with term predicates, AND, OR, and NOT, and the query retains its tree structure. What changes is the evaluation of that tree. An atomic predicate no longer produces only true or false. It produces a score between 0 and 1 that reflects the strength of the term evidence. Soft versions of AND and OR then combine these scores.

This extension directly addresses two limitations of the Standard Boolean Model. Term frequency and inverse document frequency can distinguish stronger from weaker evidence, while graded operators allow a document to receive a positive score even if it does not satisfy every predicate. The output is therefore a ranked list rather than an unordered set.

Formalizing Graded Evidence

Each document is represented by normalized TF-IDF weights. For term t_j in document D_i , define

$$d_{i,j} = \min\left(1, \frac{\text{tf}(D_i, t_j) \text{idf}(t_j)}{\alpha}\right), \quad 0 \leq d_{i,j} \leq 1,$$

where α is a fixed normalization value shared by the collection. A positive term predicate uses this weight directly; a negative predicate reverses it:

$$\text{sim}(\tau, D_i) =$$

The remaining question is how to combine the atomic scores. The p-norm model, introduced by Salton, Fox, and Wu; see [Extended Boolean Information Retrieval](#) in the Further Reading section of the chapter summary, interprets soft AND as closeness to the ideal point $(1, \dots, 1)$ and soft OR as distance from $(0, \dots, 0)$.

Key Formula: P-Norm Model

$$\text{sim}\left(\bigwedge_{k=1}^K Q_k, D_i\right) = 1 - \sqrt[p]{\frac{\sum_{k=1}^K (1 - \text{sim}(Q_k, D_i))^p}{K}}$$

$$\text{sim}\left(\bigvee_{k=1}^K Q_k, D_i\right) = \sqrt[p]{\frac{\sum_{k=1}^K \text{sim}(Q_k, D_i)^p}{K}}, \quad 1 \leq p < \infty$$

When $p = 1$, both operators average their evidence. As p increases, AND becomes more sensitive to the weakest predicate and OR becomes more sensitive to the strongest predicate. In the limit $p \rightarrow \infty$, they approach minimum and maximum.

Other soft Boolean operators are possible. They differ in how strongly one high or low atomic score influences the combined result.

Alternative soft Boolean operators (optional reading)

The fuzzy set model uses the weakest score for AND and the strongest score for OR:

$$\text{sim}(Q_1 \wedge Q_2, D_i) = \min\{\text{sim}(Q_1, D_i), \text{sim}(Q_2, D_i)\}$$

$$\text{sim}(Q_1 \vee Q_2, D_i) = \max\{\text{sim}(Q_1, D_i), \text{sim}(Q_2, D_i)\}$$

This preserves the strict interpretation of AND and OR most directly, but ignores all scores except the extreme one.

The fuzzy algebraic model combines two operands using a product for AND and a probabilistic sum for OR:

$$\text{sim}(Q_1 \wedge Q_2, D_i) = \text{sim}(Q_1, D_i) \text{sim}(Q_2, D_i)$$

$$\text{sim}(Q_1 \vee Q_2, D_i) = \text{sim}(Q_1, D_i) + \text{sim}(Q_2, D_i) - \text{sim}(Q_1, D_i) \text{sim}(Q_2, D_i)$$

A further family interpolates between the minimum and maximum. For AND, a parameter closer to the minimum emphasizes the weakest condition; for OR, a parameter closer to the maximum emphasizes the strongest condition. These alternatives illustrate that softening Boolean logic requires a modelling choice: Boolean algebra alone does not determine the graded operator.

Evaluating the Running Example

Consider `cat` AND `dog` with $p = 2$. In the 12-document collection, $\text{df}(\text{cat}) = 5$ and $\text{df}(\text{dog}) = 6$, so

$$\text{idf}(\text{cat}) = \ln(12/5) \approx 0.875, \quad \text{idf}(\text{dog}) = \ln(12/6) \approx 0.693.$$

For this example, choose $\alpha = 4 \ln 2 \approx 2.773$, the largest TF-IDF weight among these query terms. We compute the two atomic scores for each candidate and combine them with the p-norm AND formula.

Document	$d_{i, \text{cat}}$	$d_{i, \text{dog}}$	p-norm AND score
D_1	0.632	0.500	0.561
D_{10}	0.632	0.500	0.561
D_7	0.000	1.000	0.293
D_9	0.316	0.250	0.282
D_{12}	0.316	0.250	0.282
D_4	0.316	0.000	0.143
D_3	0.000	0.250	0.116

The model now distinguishes repeated evidence: D_1 and D_{10} rank above D_9 and D_{12} . It also assigns positive scores to the partial matches D_3 , D_4 , and D_7 . Evaluation can still use an inverted file: the union of the query-term postings supplies candidates, after which the system computes and sorts their scores.

Limitations of the Extended Boolean Model

The ranking also exposes the heuristic nature of the method. D_7 contains “dog” four times but no “cat”, yet it narrowly outranks D_9 , which contains both requested terms. A different value of p , another normalization constant, or another soft operator can change this order. The query grammar states what AND and OR mean for true and false values, but it does not uniquely determine how graded evidence should be combined.

The model also inherits the lexical limitations of its representation. It still does not connect “cat” with “cats” or “feline”, and it cannot distinguish the meanings of “forest”. Finally, users must still translate an information need into an explicit Boolean expression. Graded evaluation softens the consequences of that expression but does not remove the burden of formulating it.

Advantages: The model retains the familiar Boolean query structure while adding partial matches and ranked output. TF-IDF weights distinguish terms by occurrence and collection frequency, and inverted files still support efficient candidate retrieval.

Disadvantages: Scores depend on heuristic choices for normalization, operators, and parameters, which can produce counter-intuitive rankings. The model retains lexical matching and explicit Boolean query formulation.

The classical p-norm model is rarely used as the primary ranker in modern search engines. Its central idea remains relevant, however: contemporary systems often combine strict Boolean filters with optional scoring clauses. This preserves explicit query constraints while allowing partial matches to receive different relevance scores. The Vector Space Model in the next section takes the next conceptual step: it replaces explicit Boolean expressions with free-text queries and ranks documents directly from their weighted term vectors.

Vector Space Retrieval

The Vector Space Model emerged from the SMART information retrieval project led by Gerard Salton and colleagues in the 1960s, formalized in Salton, Wong, and Yang's **A Vector Space Model for Automatic Indexing**, listed in the Further Reading section of the chapter summary. SMART introduced many ideas that became central to classical text retrieval: weighted terms, document and query vectors, partial matching, and ranked output. The model had a major influence on both research and practical search systems because it replaced exact logical decisions with simple numerical operations.

Consider the free-text query `cat dog forest`. Boolean OR retrieves every document containing at least one of these terms but cannot order the ten matches. The Vector Space Model uses the same tokenized collection and the same term statistics, but represents the query and documents as vectors. Their similarity becomes a score, allowing the system to place stronger matches first.

What Remains the Same, and What Changes?

As before, preprocessing determines the vocabulary, and each distinct token defines one dimension. Documents remain bag-of-words representations, so word order is ignored and lexical variants such as “cat” and “cats” remain separate. IDF still gives less weight to terms that occur in many documents.

The query changes more fundamentally. Instead of a Boolean expression, it is treated as a short document and processed through the same pipeline. No AND or OR operators are required. A document may receive a positive score when it shares only one query term, and the score determines its position in the ranked result list.

Document and Query Vectors

For a vocabulary of M terms, document D_i becomes the sparse vector $\mathbf{d}_i = (d_{i,1}, \dots, d_{i,M})$ with

$$d_{i,j} = \text{tf}(D_i, t_j) \text{idf}(t_j), \quad 1 \leq j \leq M.$$

The query becomes a vector in the same space:

$$q_j = \text{tf}(Q, t_j) \text{idf}(t_j), \quad 1 \leq j \leq M.$$

Placing the document vectors as columns produces the term-document matrix $\mathbf{A}$. This matrix is conceptually useful, but practical systems store only non-zero entries because each document contains a small fraction of the complete vocabulary.

Inner Product and Cosine Similarity

The inner product, also called the dot product, sums the weighted evidence from terms shared by query and document:

Key Formula: Inner Product

$$\text{sim}_{\text{dot}}(Q, D_i) = \mathbf{q}^\top \mathbf{d}_i = \sum_{j=1}^M q_j d_{i,j} \quad (1.12)$$

Each shared term adds the product of its query and document TF-IDF weights. Rarer terms and higher frequencies raise the score, which has no fixed upper bound.

Only query dimensions contribute because $q_j = 0$ for every non-query term. Repeating a query term increases the score linearly. The score has no fixed upper bound and is often called a retrieval status value rather than a similarity probability. For the full collection,

$$\text{sim}_{\text{dot}}(Q, \mathbb{D}) = \mathbf{A}^\top \mathbf{q}.$$

Cosine similarity divides the same inner product by both vector lengths:

Key Formula: Cosine Similarity

$$\text{sim}_{\cos}(Q, D_i) = \frac{\mathbf{q}^\top \mathbf{d}_i}{|\mathbf{q}| |\mathbf{d}_i|} = \frac{\sum_{j=1}^M q_j d_{i,j}}{\sqrt{\sum_{j=1}^M q_j^2} \sqrt{\sum_{j=1}^M d_{i,j}^2}} \quad (1.13)$$

Dividing by both vector lengths measures the angle between query and document instead of raw overlap, giving a value from 0 (no shared terms) to 1 (identical direction).

Cosine similarity compares vector direction rather than magnitude. For non-negative TF-IDF vectors, its value lies between 0 for no shared terms and 1 for identical directions. Document vectors can be normalized once during indexing. The system then computes cosine similarity as an inner product between unit-length vectors, avoiding repeated norm calculations at query time.

Evaluating a Query

Evaluation begins like Boolean OR. The system looks up the posting list for every query term and takes their union; documents outside this union have score zero. For each candidate, it accumulates $q_j d_{i,j}$ from the matching postings. This sum is already the inner-product score. Cosine evaluation adds one step: divide by the query norm and the document norm stored with the index. Finally, sort candidates by decreasing score.

This procedure avoids constructing the dense matrix $\mathbf{A}$ or comparing the query with every document. Chapter 4 develops inverted files and posting-list traversal in detail.

Running Example

For $Q = \text{"cat dog forest"}$ and $N = 12$, the query terms have

$$\text{idf}(\text{cat}) = \ln(12/5) \approx 0.875,$$

$$\text{idf}(\text{dog}) = \ln(12/6) \approx 0.693,$$

$$\text{idf}(\text{forest}) = \ln(12/7) \approx 0.539.$$

Each query term occurs once, so these values are also the non-zero components of $\mathbf{q}$. Applying the same TF-IDF representation to every document gives the complete ranking evidence below.

Document	TF (cat, dog, forest)	Inner product	Cosine
D_1	(2, 2, 1)	2.784	0.659
D_2	(0, 0, 0)	0.000	0.000
D_3	(0, 1, 1)	0.771	0.102
D_4	(1, 0, 0)	0.766	0.093
D_5	(0, 0, 2)	0.581	0.059
D_6	(0, 0, 1)	0.291	0.034
D_7	(0, 4, 0)	1.922	0.232
D_8	(0, 0, 4)	1.162	0.139
D_9	(1, 1, 1)	1.537	1.000
D_{10}	(2, 2, 2)	3.075	0.342
D_{11}	(0, 0, 0)	0.000	0.000
D_{12}	(1, 1, 0)	1.247	0.137

The inner product ranks $D_{10} > D_1 > D_7 > D_9$. Repeating all query terms makes D_{10} the strongest match, while repeating only “dog” allows D_7 to outrank the compact exact match D_9 . Cosine produces a different order: $D_9 > D_1 > D_{10} > D_7$. The vector for D_9 has exactly the query direction and therefore receives the maximum score of 1. The additional non-query terms in D_{10} increase its norm and reduce its cosine score even though it contains every query term twice.

Geometry of inner product and cosine (optional reading)

For a fixed score threshold γ , the inner product accepts documents satisfying $\mathbf{q}^\top \mathbf{d} \geq \gamma$. The boundary is a hyperplane with the query vector as its normal. Documents farther from the origin in the query direction receive larger scores. Since dimensions with $q_j = 0$ vanish from the sum, the inner product effectively projects each document onto the subspace spanned by the query terms.

Cosine similarity instead defines a hypercone around the query vector. A higher threshold produces a narrower cone, so accepted documents must point in a direction closer to the query. Scaling every component of a document by the same factor does not change its cosine score. The measure therefore rewards similar proportions rather than high absolute frequencies.

This distinction explains D_9 and D_{10} . Both have query-term proportions 1 : 1 : 1, but cosine is measured in the full vocabulary space. The many non-query components of D_{10} contribute to $|\mathbf{d}_{10}|$ even though they contribute nothing to the numerator. They rotate the full document vector away from the query and lower its score.

For a one-term query, the geometry becomes extreme. The inner product primarily ranks documents by the frequency of that term. Cosine rewards vectors concentrated on that one dimension, so additional vocabulary lowers the score. A page containing little except the query term can therefore outrank a richer document that discusses the same topic in broader language.

Limitations of Vector Space Retrieval

TF-IDF weights and both similarity measures are heuristic. They rank effectively in many collections, but neither formula directly estimates relevance. The inner product grows linearly with term frequency, so long documents tend to receive more query-term occurrences and authors can manipulate rankings by repeating selected terms. In the example, four occurrences of “dog” push D_7 above D_9 despite the absence of “cat” and “forest”.

Cosine removes this magnitude bias but introduces different assumptions. It prefers documents whose term-frequency ratios resemble the query and uses every document term in the normalization. Consequently, non-query terms can lower a score, although they provide no negative evidence in the inner-product numerator. This is counter-intuitive when a relevant passage is embedded in a longer document.

The representation also retains the earlier lexical limitations. Terms are independent dimensions, so the model neither connects “cat” with “feline” nor distinguishes the meanings of “forest”. Word order and dependencies such as “random forest” are absent from the bag-of-words vector.

Term independence assumption

The Vector Space Model treats terms as independent dimensions. For example, “New” and “York” contribute separately even when their combination denotes one place. Later chapters introduce phrases, latent representations, and dense embeddings that capture some relationships between terms.

Advantages: The model accepts natural free-text queries, supports partial matches, produces ranked output, and gives discriminating terms more influence through IDF. Sparse vectors and inverted files make evaluation simple and efficient.

Disadvantages: Weighting and similarity are heuristic. Inner products favour high term frequencies and longer documents, while cosine can penalize useful additional content and favours query-like term ratios. Neither measure includes term-frequency saturation, robust length normalization, or semantic relationships.

Modern Applications

Classical TF-IDF vectors with cosine similarity remain useful as transparent baselines and in small or specialized collections. They also support document similarity, clustering, classification, recommendation, and near-duplicate detection where a sparse lexical representation is sufficient. For primary lexical ranking,

production search systems now commonly prefer BM25 because it controls term-frequency growth and document-length effects more carefully.

The geometry has become even more important than the original representation. Modern semantic search encodes text as dense embedding vectors and retrieves neighbours with inner product or cosine similarity, often through approximate nearest-neighbour indexes. Hybrid systems combine this dense vector retrieval with sparse lexical methods such as BM25. Thus, modern systems frequently retain the Vector Space Model's scoring operations while replacing TF-IDF dimensions with learned semantic features.

The Vector Space Model establishes the geometry of ranked retrieval but leaves term-frequency growth and document-length effects to heuristic choices. The next section introduces probabilistic term evidence and develops BM25 as a practical response to these limitations.

Probabilistic Ranking and BM25

The Vector Space Model ranks effectively, but its weighting and similarity measures are heuristic. Probabilistic retrieval asks a more direct question: given query Q and document D_i , how likely is the document to be relevant? The Binary Independence Model (BIR) develops this idea from binary term evidence. BM25 then retains its probabilistic term weighting while adding the term-frequency saturation and document-length normalization missing from BIR and vector-space scoring.

Binary Independence Model

The BIR model uses the same tokenized vocabulary as the previous models, but represents each query and document as a set of terms. It was formalized by Robertson and Spärck Jones; see **Relevance Weighting of Search Terms** in the Further Reading section of the chapter summary. A component is 1 when a term is present and 0 when it is absent. It assumes that terms contribute independently and that non-query terms occur equally often in relevant and non-relevant documents. Under these assumptions, only query terms present in a document affect its rank.

Let R denote relevance and NR non-relevance for query Q . Ranking by the posterior odds

$$\frac{P(R \text{ mid } D_i, Q)}{P(NR \text{ mid } D_i, Q)} \quad (1.14)$$

is equivalent to ranking by the document evidence after query-dependent constants are removed. Define

$$r_j = P(d_{i,j} = 1 \text{ mid } R, Q), \quad n_j = P(d_{i,j} = 1 \text{ mid } NR, Q). \quad (1.15)$$

The first probability measures how often term t_j occurs in relevant documents; the second measures how often it occurs in non-relevant documents. The resulting score is additive, but several probability terms must cancel before we reach that form. The derivation below explains why only query terms present in the document remain.

From posterior odds to the BIR score (optional reading)

Bayes' rule separates the posterior odds into prior odds and document evidence:

$$\frac{P(R \text{ mid } d_i, Q)}{P(NR \text{ mid } d_i, Q)} = \frac{P(R \text{ mid } Q)}{P(NR \text{ mid } Q)} \cdot \frac{P(d_i \text{ mid } R, Q)}{P(d_i \text{ mid } NR, Q)}. \quad (1.16)$$

The prior odds $P(R \text{ mid } Q)/P(NR \text{ mid } Q)$ are the same for every document considered for query Q . They affect the numerical score but not the ranking. We can therefore focus on the likelihood ratio containing the document evidence.

Under the binary representation, each component satisfies $d_{i,j} \in \{0, 1\}$: it is 1 when document D_i contains term t_j and 0 otherwise. Combined with term independence, this makes the probability of the complete document vector a product of Bernoulli probabilities. The exponents select the appropriate case: $r_j^{d_{i,j}}$ contributes r_j when the term is present, while $(1 - r_j)^{1-d_{i,j}}$ contributes $1 - r_j$ when it is absent.

$$P(d_i \text{ mid } R, Q) = \prod_{j=1}^M r_j^{d_{i,j}} (1 - r_j)^{1-d_{i,j}}, \quad (1.17)$$

$$P(d_i \text{ mid } NR, Q) = \prod_{j=1}^M n_j^{d_{i,j}} (1 - n_j)^{1-d_{i,j}}. \quad (1.18)$$

Substituting these products into the odds and taking the logarithm turns multiplication into addition:

$$\log \frac{P(R \text{ mid } d_i, Q)}{P(NR \text{ mid } d_i, Q)} = \log \frac{P(R \text{ mid } Q)}{P(NR \text{ mid } Q)} + \sum_{j=1}^M \left[d_{i,j} \log \frac{r_j}{n_j} + (1 - d_{i,j}) \log \frac{1 - r_j}{1 - n_j} \right]. \quad (1.19)$$

For a term absent from the query, BIR assumes $r_j = n_j$. Both logarithms then become zero, so every non-query term disappears. For a query term, we can rearrange its contribution as

$$\log \frac{1 - r_j}{1 - n_j} + d_{i,j} \log \frac{r_j(1 - n_j)}{n_j(1 - r_j)}. \quad (1.20)$$

The first part depends on the query but not on the document, so it is another constant that does not change the ranking. The second part contributes only when $d_{i,j} = 1$. After removing all ranking constants, the remaining coefficient is

$$c_j = \log \frac{r_j(1 - n_j)}{n_j(1 - r_j)}, \quad (1.21)$$

which produces the additive BIR score below.

Key Formula: BIR Similarity

$$\text{sim}_{\text{BIR}}(Q, D_i) = \sum_{j: q_j=1, d_{i,j}=1} c_j, \quad c_j = \log \frac{r_j(1 - n_j)}{n_j(1 - r_j)} \quad (1.22)$$

A positive c_j means that term t_j is more characteristic of relevant than non-relevant documents. Documents receive the sum of the weights for query terms they contain.

Relevance Feedback

Initial estimates. Before any relevance information is available, the system must produce a first ranking. BIR commonly assumes that every query term has an equal chance of occurring in a relevant document, $r_j = 0.5$. It estimates occurrence in non-relevant documents from the term's document frequency in the complete collection:

$$r_j = 0.5, \quad n_j = \frac{\text{df}(t_j) + 0.5}{N + 1}. \quad (1.23)$$

These initial values produce the first c_j weights and therefore the first result list.

Collecting feedback. The user can now mark retrieved documents as relevant or non-relevant, for example through like and dislike controls. These judgements provide a sample of both classes. Because r_j is the probability that term t_j occurs in a relevant document, we can estimate it by counting how many judged relevant documents contain the term. We estimate n_j in the same way from the judged non-relevant documents.

Updating the estimates. Suppose the user has judged K documents and marked L of them as relevant. Let k_j be the number of judged documents containing t_j , and let l_j be the number that both contain t_j and are relevant. The relevant sample therefore contains l_j occurrences among L documents. The non-relevant sample contains $k_j - l_j$ occurrences among $K - L$ documents. Without smoothing, the corresponding estimates would be

$$r_j \approx \frac{l_j}{L}, \quad n_j \approx \frac{k_j - l_j}{K - L}. \quad (1.24)$$

In practice, BIR adds pseudo-counts so that small samples do not produce probabilities of exactly 0 or 1:

$$r_j = \frac{l_j + 0.5}{L + 1}, \quad n_j = \frac{k_j - l_j + 0.5}{K - L + 1}. \quad (1.25)$$

The updated probabilities produce new c_j weights and a revised ranking. Further rounds of feedback can refine the estimates, although users may be unwilling to judge many results.

Example: Feedback and query expansion

Consider the query `dog forest`. Suppose all 12 documents are judged, and D_1 and D_3 are marked relevant, leaving the other 10 documents as the non-relevant sample. Both relevant documents contain “dog” and “forest”, so feedback first refines the weights of the original query terms:

Term	Relevant documents containing term	Non-relevant documents containing term	r_j	n_j	c_j
“dog”	2	4	0.833	0.409	1.977
“forest”	2	5	0.833	0.500	1.609

With these refined weights, a document containing both “dog” and “forest” scores $1.977 + 1.609 = 3.586$, an improved estimate over the initial, feedback-free weights.

Feedback has a second, independent use: we can compute c_j for terms that never appeared in the query, using the same relevant and non-relevant samples. This tells us which additional terms help separate relevant from non-relevant documents, without requiring the user to reformulate anything.

Term	Relevant documents containing term	Non-relevant documents containing term	r_j	n_j	c_j
“woodland”	2	2	0.833	0.227	2.833
“algorithms”	0	1	0.167	0.136	0.236
“cats”	0	2	0.167	0.227	-0.386

The three terms illustrate three distinct roles:

- **“woodland”** has a high positive c_j : both relevant documents contain it, and only a fifth of the non-relevant documents do. It is a strong candidate for query expansion. Adding it to the query raises D_1 and D_3 to 6.419 and gives lexical variants such as D_2 and D_{11} positive evidence even though they contain neither “dog” nor “forest”.
- **“algorithms”** has a c_j close to zero: it occurs in only one document overall, and that document is not relevant. The sample is too small to say whether the term discriminates at all, so it is best discarded, the same treatment we give to stop words that occur everywhere.
- **“cats”** has a negative c_j : it occurs only in non-relevant documents, both of which are technical distractors about pets and animal classification rather than the woodland adventure the user wants. Note that “cats” is a distinct token from “cat”, which does appear in D_1 ; the tokenizer does not know they share a root, so the two counts are entirely independent. A negative weight can therefore help demote or eliminate documents that share surface vocabulary with the query but belong to the wrong sense.

In practice, an automatic query expansion step keeps terms with a strongly positive c_j , drops terms near zero, and may use strongly negative terms to penalize rather than reward a document.

BIR marks a change in ambition, not only a change in formula. The Vector Space Model ranks by geometric heuristics that happen to work well; BIR instead tries to *explain* relevance from first principles, estimating how likely a document is to be relevant given its terms. It still represents documents as term vectors and still combines evidence with the same OR-like accumulation as VSM: any query term found in the document contributes its weight to the score. What changes is the meaning of that weight. Instead of a heuristic tf-idf product, c_j estimates the independent contribution of each query term toward relevance, grounded in how often the term separates relevant from non-relevant documents.

The restrictive assumptions behind this first version, binary term presence, term independence, and no document-length effect, are not inherent to the probabilistic idea itself. They make BIR tractable, and later probabilistic models relax them one at a time. The 2-Poisson model, for instance, models within-document term frequency directly instead of treating term presence as binary, at the cost of a more complex parameter estimation problem; see Robertson and Walker’s **Some Simple Effective Approximations to the 2-Poisson Model for Probabilistic Weighted Retrieval** in the Further Reading section of the

chapter summary. Probabilistic language models take yet another route, estimating the probability that a document's language model would generate the query. The section below develops BM25, which keeps BIR's additive, per-term structure but approximates the 2-Poisson model's term-frequency behaviour with a simpler saturating function.

The feedback mechanism also has a practical limitation worth mentioning. A user who rates a first result list of 10 documents gives us a sample of size $K = 10$ from which to estimate r_j and n_j , far too small to pin down these probabilities reliably, especially for terms that appear in only one or two of the judged documents. Larger judged samples would improve the estimates, but no user will realistically rate hundreds of documents to make search work. This tension between statistical need and user effort is a recurring theme in relevance feedback, not a flaw specific to BIR.

Okapi BM25

BM25 was developed at London's City University by Stephen Robertson, Karen Spärck Jones, and colleagues as a practical approximation to the 2-Poisson model. It became the robust default for free-text retrieval that BIR itself never was. Robertson and Zaragoza's **The Probabilistic Relevance Framework: BM25 and Beyond**, listed in the Further Reading section of the chapter summary, surveys the complete derivation and its later extensions.

What is the meaning of "Okapi BM25"?

The name comes from Okapi, the experimental retrieval system built at City University London in the 1980s and 1990s where the formula was first implemented, not from any property of the ranking function itself. "BM" stands for "Best Matching", and "25" simply identifies it as the 25th weighting variant the researchers tried.

Three Unresolved Issues

Before introducing the BM25 formula, it helps to see exactly what the two preceding models still get wrong. Each issue below reappears as a concrete design decision in BM25.

Vector-space scoring rewards repetition without limit. Recall the running query `cat dog forest` from the Vector Space Retrieval section. Its inner product already ranks D_7 (four occurrences of "dog", score 2.773 for that term alone) above D_9 (one occurrence of each query term, contributing 0.693 for "dog" toward its combined score of 1.537), even though D_9 is the only document that mentions "cat", "dog", and "forest" together. If a document repeated "dog" twenty times instead of four, its score would climb to 13.863, growing linearly and without bound. A ranking function that rewards raw repetition this directly is vulnerable to keyword stuffing: an author can inflate a document's score simply by repeating a term, regardless of whether the document is actually about that topic.

Raw term frequency ignores what "coverage" means for document length. A term occurring once in a 3-token document (such as D_9) is stronger evidence of relevance than the same term occurring once in a 20-token document, because in the short document, that one occurrence forms a much larger share of the content. TF-IDF and the inner product treat both occurrences identically. Relevance should depend on how much of the document is about the query term, not merely how many times it appears. To reward the same relevance signal, a longer document should need proportionally more occurrences of a term than a shorter one to reach the same score.

Binary presence throws away frequency information, but the idea behind r_j and n_j is worth keeping. BIR's document evidence is either 0 or 1, so it cannot express that D_7 mentions "dog" four times while D_3 mentions it once. At the same time, BIR contributes something valuable: the idea that a term's ranking weight should reflect how well that term distinguishes relevant from non-relevant documents, not just how rare it is in the collection overall. BM25 keeps this probabilistic idea but combines it with a real-valued term-frequency component.

Term-Frequency Saturation

BM25 replaces the raw count in the TF-IDF product with a bounded function of term frequency:

$$\hat{tf}_{k_1} = \frac{tf(k_1 + 1)}{tf + k_1}, \quad k_1 > 0. \quad (1.26)$$

The first occurrence contributes strongly; later occurrences add progressively less; the value never exceeds $k_1 + 1$ no matter how often the term repeats. This directly answers the spamming issue: repeating “dog” twenty times can no longer produce an unbounded score. Parameter k_1 controls how quickly the function saturates. Values between 1 and 2 are common, with $k_1 = 1.2$ used in the running example. Figure 1.16 contrasts linear, square-root, and saturating term-frequency weights.

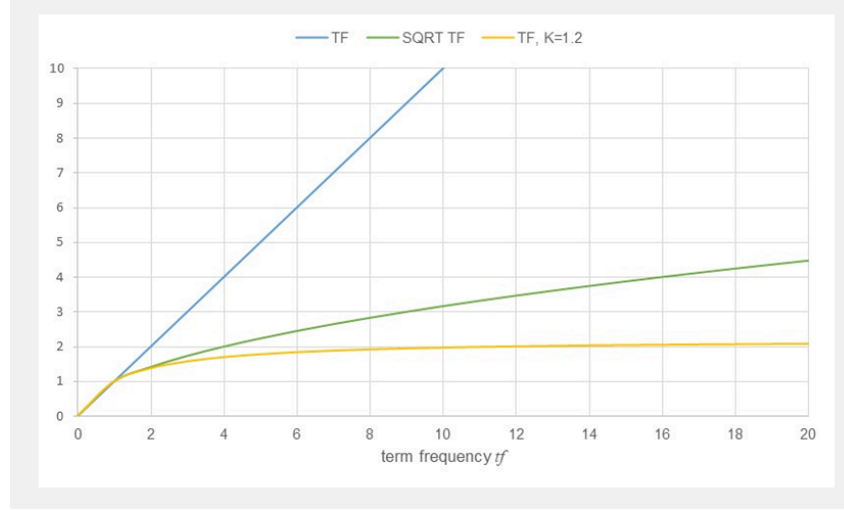


Figure 1.16: Linear, square-root, and BM25-saturated term-frequency weighting.

Document-Length Normalization

Saturation alone does not address the length issue: a document with $tf=1$ in 3 tokens and a document with $tf=1$ in 20 tokens still saturate identically. BM25 makes the saturation point depend on document length by scaling the denominator:

$$\hat{tf}_{k_1,b}(D_i, t) = \frac{tf(D_i, t)(k_1 + 1)}{tf(D_i, t) + k_1 \left(1 - b + b \frac{|D_i|}{\text{avgdl}}\right)}. \quad (1.27)$$

Here $|D_i|$ is the number of processed tokens (=document length) and avgdl is the collection’s average document length. For the running collection, $\text{avgdl} = 9.67$ tokens. A single occurrence of “dog” in a 3-token document (shorter than average) contributes 1.393, while the same single occurrence in a 20-token document (longer than average) contributes only 0.696: the short document needs less repetition to reach the same weight because that occurrence covers a larger share of its content. Parameter $b \in [0, 1]$ controls the strength of this effect: $b = 0$ disables length normalization entirely, while $b = 1$ applies the full length ratio. Figure 1.17 shows how the same term frequency receives more weight in a short document than in a long one.

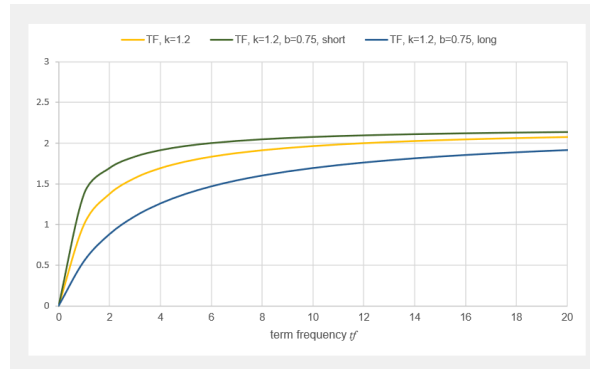


Figure 1.17: BM25 term-frequency saturation for average, short, and long documents.

Probabilistic IDF

The third issue was that BIR's idea of estimating a term's discriminating power from r_j and n_j is worth keeping even though its binary document model is not. Substituting the no-feedback BIR estimates ($r_j = 0.5$ and n_j from document frequency) into the c_j formula from the previous section gives exactly the classical BM25 weight:

$$\text{idf}_{\text{BM25}}(t) = \log \frac{N - \text{df}(t) + 0.5}{\text{df}(t) + 0.5}. \quad (1.28)$$

This value becomes negative when a term appears in more than half of the collection. Many implementations instead use the always-positive Lucene variant

$$\text{idf}_+(t) = \log \left(1 + \frac{N - \text{df}(t) + 0.5}{\text{df}(t) + 0.5} \right) = \log \frac{N + 1}{\text{df}(t) + 0.5}. \quad (1.29)$$

Figure 1.18 compares these variants with the smoothed classical IDF introduced earlier. All encode the same core intuition that common terms provide less evidence, but their numerical values are not interchangeable.

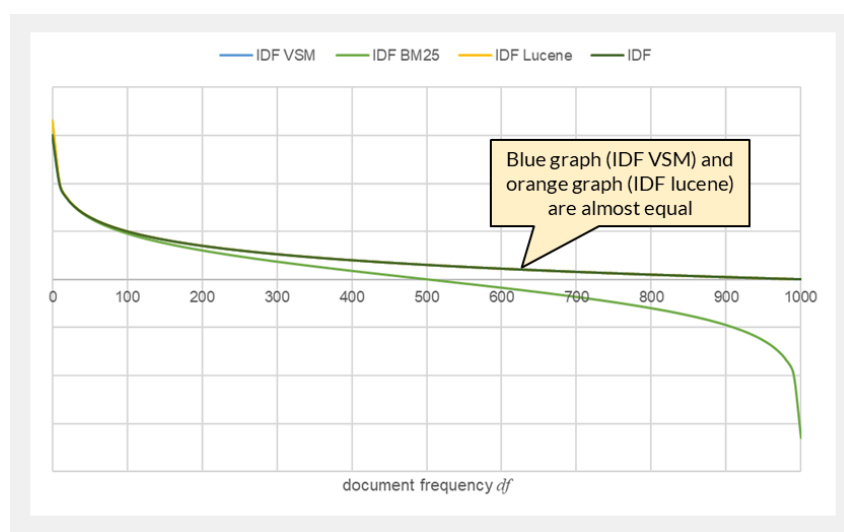


Figure 1.18: Classical, original BM25, and positive Lucene IDF variants.

A negative weight is undesirable for an additive scoring function: it would let a matching query term actively lower a document's score, the opposite of what a match should do, and it complicates summing evidence across query terms of very different commonness. The Lucene weight $\log \frac{N+1}{\text{df}(t)+0.5}$ and the smoothed classical VSM weight $\log \frac{N}{\text{df}(t)}$ are nearly identical: comparing them shows that the Lucene variant is equal to the classical one exactly when $\text{df}(t) = N/2$, slightly below it for rare terms, and slightly above it for common terms, never departing far in either direction. This is exactly the region where the original BM25 IDF turns negative, so Lucene's variant tracks the familiar classical shape almost everywhere while staying non-negative where it matters. Lucene therefore adopts it as a numerically safer stand-in for the same underlying quantity rather than as a conceptually different weighting scheme.

Note

The standard BM25 formula uses the no-feedback estimate for r_j and does not itself perform a feedback step. Nothing prevents combining BM25 with relevance feedback: a system can substitute the feedback-refined r_j and n_j from BIR into the same weight instead of the fixed $r_j = 0.5$. In practice, systems built on Lucene use the fixed no-feedback form as the default.

Complete Formula and Running Example

Using the positive IDF variant gives the practical scoring function used in this example.

Key Formula: BM25

$$\text{sim}_{\text{BM25}}(Q, D_i) = \sum_{t \in Q \cap D_i} \text{idf}_+(t) \frac{\text{tf}(D_i, t)(k_1 + 1)}{\text{tf}(D_i, t) + k_1 \left(1 - b + b \frac{|D_i|}{\text{avgdl}}\right)} \quad (1.30)$$

BM25 sums query-term evidence after applying probabilistic term specificity, diminishing returns for repetition, and document-length normalization.

For $Q = \text{cat dog forest}$, use $k_1 = 1.2$, $b = 0.75$, and the same preprocessing as before. The 12 documents have $\text{avgdl} = 9.67$ tokens. Their positive IDF values are 0.860 for “cat”, 0.693 for “dog”, and 0.550 for “forest”.

Example: BM25 ranking

Document	Length	TF (cat, dog, forest)	BM25 score
D_9	3	(1, 1, 1)	2.930
D_1	8	(2, 2, 1)	2.836
D_{10}	14	(2, 2, 2)	2.568
D_{12}	11	(1, 1, 0)	1.470
D_3	10	(0, 1, 1)	1.226
D_7	10	(0, 4, 0)	1.166
D_4	8	(1, 0, 0)	0.925
D_8	12	(0, 0, 4)	0.894
D_5	11	(0, 0, 2)	0.728
D_6	10	(0, 0, 1)	0.542

Documents D_2 and D_{11} share no exact query term and score zero. The compact exact match D_9 ranks first, resolving the ordering that VSM’s inner product got wrong. Repetition keeps D_1 competitive, but saturation prevents the four occurrences of “dog” in D_7 from dominating the way it did under linear TF-IDF. Although D_{10} repeats all three terms, its greater length lowers its score relative to D_9 : the same occurrences carry less weight once length normalization is applied.

Evaluation follows the same efficient pattern as vector-space retrieval. The system takes the union of the query-term posting lists, accumulates one BM25 contribution per matching term, and sorts candidates by decreasing score. Chapter 4 develops the posting-list algorithms used for this process.

Limitations and Modern Applications

BM25 remains lexical and bag-of-words based. It neither connects “cat” with “feline” nor distinguishes the natural and technical meanings of “forest”. It also assumes that document length should influence relevance in a consistent way, so k_1 and b may require tuning for collections with unusual document types. Despite its probabilistic derivation, a BM25 score is a ranking value, not a calibrated probability of relevance.

Advantages: BM25 combines partial matching, discriminative term weighting, term-frequency saturation, and document-length normalization in an efficient additive score. It is transparent, fast, and difficult to improve upon as a lexical baseline.

Disadvantages: It retains lexical matching and term-independence assumptions, requires parameter choices, and does not directly represent phrase meaning or semantic similarity.

BM25 is the default or standard lexical ranker in systems built on Lucene, including Elasticsearch, Solr, and OpenSearch. It is also widely used as a first-stage retriever in retrieval-augmented generation and in hybrid search, where its exact lexical matches complement dense embedding retrieval.

Hands-on: Retrieval Models

Implement Boolean, TF-IDF, and BM25 retrieval on a small collection and compare their rankings.

[Open notebook ->](#)

Summary

Model Comparison

Property	Boolean	Extended Boolean	Vector Space	BIR	BM25
Query form	Boolean expression	Boolean expression	Free text	Term set	Free text
Ranked output	No	Yes	Yes	Yes	Yes
Partial matches	No	Yes	Yes	Yes	Yes
Term frequency	Ignored	Normalized TF-IDF	Linear TF-IDF	Ignored	Saturating
Document length	Ignored	Indirect normalization	Cosine or none	Ignored	Explicit parameter b
Foundation	Set theory	Soft-logic heuristic	Geometric heuristic	Relevance probability	BIR plus TF and length models
Typical role	Exact filters	Specialized soft constraints	Baseline and vector similarity	Feedback model and foundation	Production lexical ranking
Main weakness	No ranking	Operator choices	Length and repetition effects	Binary evidence and feedback burden	Lexical matching and tuning

The progression is cumulative rather than a sequence of complete replacements. Boolean predicates remain useful for mandatory filters, vector operations remain central to dense retrieval, BIR explains relevance feedback and probabilistic term weights, and BM25 provides the strongest general-purpose lexical ranking baseline.

Key Takeaways

1. Text retrieval represents documents and queries using the same vocabulary, then measures similarity to estimate relevance. This direct word-level matching makes text the least affected modality by the semantic gap.
2. The feature extraction pipeline (Extract $\rightarrow$ Split $\rightarrow$ Tokenize $\rightarrow$ Summarize) transforms raw text into searchable representations. The bag-of-words model records term frequencies; the set-of-words model records only term presence.
3. IDF weighting captures term discrimination power: rare terms that appear in few documents are more informative for distinguishing relevant from non-relevant results.
4. Retrieval models evolved from Boolean (no ranking, exact match) through the Extended Boolean Model (partial matches, simple scoring) to the Vector Space Model (tf-idf, cosine similarity) and Probabilistic Retrieval (BIR model, relevance feedback).
5. BM25 combines the best ideas: saturating term frequency, document-length normalization, and probabilistically grounded IDF. It remains the default baseline in modern search engines (Lucene, Elasticsearch, OpenSearch).

Key Formulas

Key Formula: TF-IDF

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \cdot \log \frac{N}{\text{df}(t)} \quad (1.31)$$

Term frequency $\times$ inverse document frequency. Rewards terms that are frequent in a specific document but rare across the collection.

Key Formula: Cosine Similarity

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{|\mathbf{q}| \cdot |\mathbf{d}|} \quad (1.32)$$

Measures the angle between query and document vectors, normalizing for document length.

Key Formula: BM25

$$\text{sim}_{\text{BM25}}(Q, D) = \sum_{t \in Q \cap D} \log \left(1 + \frac{N - \text{df}(t) + 0.5}{\text{df}(t) + 0.5} \right) \cdot \frac{\text{tf}(t, D) \cdot (k_1 + 1)}{\text{tf}(t, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}} \right)} \quad (1.33)$$

Probabilistic term specificity multiplied by saturating, length-normalized term frequency. Parameter k_1 controls saturation, b controls length normalization, and avgdl is the collection's average document length.

Exam focus

Beyond the formulas above, be able to explain:

- Why the Standard Boolean Model cannot rank documents, and what the Extended Boolean Model changes to enable graded scores.
- Why the inner product favors long, repetitive documents while cosine similarity favors documents whose term proportions match the query, and why cosine can still penalize relevant content.
- Why BIR's binary term-presence assumption is too coarse, and how relevance feedback estimates r_j and n_j .
- Why the classical, BM25, and Lucene IDF variants are not numerically interchangeable, even though they encode the same intuition.
- How BM25's saturation parameter k_1 and length-normalization parameter b each address a specific weakness of TF-IDF scoring.

Self-Check Questions

1. (Understand) Explain in your own words why the Boolean model cannot rank documents, and what the Extended Boolean Model changes to enable ranking.
2. (Analyze) Given a collection of 10,000 documents where the term "retrieval" appears in 50 documents and the term "the" appears in 9,800 documents, calculate the IDF for both terms. Why does this difference matter for relevance scoring?
3. (Analyze) A document is $3\times$ longer than the average document in the collection. How does BM25 (with $b = 0.75$) adjust the effective term frequency compared to an average-length document? What happens when $b = 0$?
4. (Evaluate) An e-commerce search system currently uses Boolean retrieval with faceted filtering. A developer proposes switching to BM25. What are the trade-offs? In which scenarios might Boolean retrieval still be preferable?

Test Your Knowledge

[Take the Chapter 1 Quiz ->](#)

Further Reading

- Spärck Jones, K. (1972). **A Statistical Interpretation of Term Specificity and Its Application in Retrieval**. *Journal of Documentation*, 28(1), 11-21. [PDF](#). Introduced inverse document frequency; foundational for every term-weighting scheme discussed in this chapter, including TF-IDF and BM25.
- Salton, G., Wong, A., & Yang, C. S. (1975). **A Vector Space Model for Automatic Indexing**. *Communications of the ACM*, 18(11), 613-620. [PDF](#). The original paper introducing the Vector Space Model; still readable and surprisingly concise.
- Robertson, S. E., & Spärck Jones, K. (1976). **Relevance Weighting of Search Terms**. *Journal of the American Society for Information Science*, 27(3), 129-146. [Publisher page](#). Formalizes the Binary Independence Model and its r_j/n_j relevance-feedback estimates covered in this chapter.
- Salton, G., Fox, E. A., & Wu, H. (1983). **Extended Boolean Information Retrieval**. *Communications of the ACM*, 26(11), 1022-1036. [Free access](#). Introduces the p-norm model, positioning graded Boolean retrieval between the strict Boolean and vector-space models.
- Porter, M. F. (1980). **An Algorithm for Suffix Stripping**. *Program*, 14(3), 130-137. [Author's page](#). The classic stemming algorithm still used as a baseline in English text processing.
- Robertson, S. E., & Walker, S. (1994). **Some Simple Effective Approximations to the 2-Poisson Model for Probabilistic Weighted Retrieval**. In *Proceedings of the 17th ACM SIGIR Conference*, 232-241. [PDF](#). Derives BM25's term-frequency saturation and length normalization from the 2-Poisson model discussed in the Probabilistic Ranking and BM25 section.
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). **Introduction to Information Retrieval**. Cambridge University Press. [Free online edition](#). The standard graduate textbook; chapters 1 to 6 cover Boolean, VSM, and probabilistic models with rigorous treatment.
- Robertson, S. E., & Zaragoza, H. (2009). **The Probabilistic Relevance Framework: BM25 and Beyond**. *Foundations and Trends in Information Retrieval*, 3(4), 333-389. [PDF](#). Definitive survey of BM25's theoretical foundations and practical extensions.

2 - Performance Evaluation

A student searches a university library for books that provide the computer science foundations needed for a master's degree. One search system returns many useful books mixed with unrelated titles. Another returns fewer useful books, but places them near the top. Which system is better? The answer depends on what the student needs: broad coverage, a clean result set, or the best books in the first few positions.

Performance evaluation turns claims such as “better results” into measurements that can be checked and reproduced. A fair comparison requires more than a metric. It needs a representative collection, realistic information needs, consistent relevance judgments, and a clear definition of success. Without this shared benchmark, a higher score may reflect an easier test rather than a better retrieval method.

Evaluation also applies when a text system assigns labels rather than retrieving documents. A spam filter decides whether an email is unwanted, while an AI assistant may route a query to a knowledge base, web search, calculator, or clarification step. Retrieval evaluates decisions about query-document pairs; classification evaluates labels assigned to text items. Both expose the same fundamental trade-off: finding more desired items often means accepting more incorrect decisions.

What you'll learn

- Design a retrieval benchmark from a document collection, information needs, relevance judgments, and performance goals
- Select and interpret metrics for unordered results, ranked results, and rankings with graded relevance
- Calculate precision, recall, fallout, F-measures, reciprocal rank, average precision, mean average precision, and normalized discounted cumulative gain
- Analyze binary spam filtering and multiclass intent routing with confusion matrices, per-class metrics, and macro and micro averages
- Explain how prevalence, decision thresholds, ROC curves, and AUC affect the interpretation of classifier performance

Prerequisites

- The retrieval architectures and models introduced in [1 - Classical Text Retrieval](#)
- Basic set operations, fractions, arithmetic means, and logarithms
- No prior knowledge of classification algorithms is required

Designing a Retrieval Benchmark

A university library holds 50 books. A student submits one information need: which books give a beginning master's student solid computer science foundations? Fifteen of the 50 books are useful for that need. Two retrieval systems answer the same request, and their first five results look like this.

Rank	System A result	Useful?	System B result	Useful?
1	Computer Organization and Design	yes	Database Systems: The Complete Book	yes
2	Operating System Concepts	yes	Pattern Recognition and Machine Learning	yes
3	The Handmaid's Tale	no	Introduction to Algorithms	yes
4	Narrative of the Life of Frederick Douglass	no	On the Origin of Species	no
5	Towards a New Architecture	no	Introduction to the Theory of Computation	yes

System A does not stop at rank 5. It returns 25 books in total, half the library, and finds 12 of the 15 useful ones. System B returns only 8 books, of which 6 are useful. So System A surfaces more of what exists, and System B wastes less of the reader's attention.

Which system is better? Two students with the same assignment can reasonably disagree. One wants a short list to read end to end and prefers System B. Another wants to be certain that nothing important was missed and prefers System A. Neither is wrong, so before we can measure anything, we have to say what counts as a good result.

What Counts as a Better Result

Relevance is not a property of a book. It is a judgment about whether a book serves one person's need at one moment. "Introduction to Algorithms" is essential to our student and useless to someone looking for a novel for a train journey. Every measurement in this chapter therefore rests on a human decision taken before any system ran.

That subjectivity is unavoidable, but it is containable. Once the need is written down and the judgments are recorded, the counting becomes mechanical: anyone who applies the same procedure to the same lists obtains the same numbers. Evaluation does not remove the subjective step, it isolates it.

What counts as success then varies enormously with the need. At one extreme only the first few results matter. Someone checking when BM25 was published reads two or three entries and stops, so anything below rank 5 might as well not exist. A short and precise list wins here, in the style of System B, and whatever relevant material it leaves behind costs the reader nothing. At the other extreme, missing anything is the real failure. A patent lawyer searching for prior art will work through a long and noisy list rather than risk overlooking one document, so broad coverage wins, in the style of System A, and a very short list would be useless however precise it was.

Many information needs are in between these extremes, including our student's. A foundations reading list has to be broad enough to cover the main areas of the field, yet short enough to work through in a semester. Neither system serves that well: System A buries the useful books among thirteen novels, and System B leaves out nine of the fifteen that would have helped. Web search leans towards the first extreme, biomedical literature search towards the middle, and the lesson is the same in every domain: "better" has no meaning until the intended use is named.

Why Anecdotes Are Not Evidence

Product advertising is full of comparative claims with no shared basis. A vendor states that a search engine is forty percent more accurate, without naming the collection it was tested on, the queries that were used,

who judged the results, or what “accurate” counts. Such a claim cannot be checked, and it is usually easy to construct: pick the queries where your system happens to win, and the number follows.

Retrieval systems invite exactly this failure, because single-example demonstrations are trivially available. For almost any pair of systems, someone can find one query where the first beats the second and another query where the reverse holds. The two result lists at the start of this section are a demonstration, not evidence. They describe one need in a 50-book collection, so they support no general statement about either system.

A scientific comparison replaces the anecdote with a fixed experimental setup. The collection is frozen, the information needs are written down in advance, the relevance judgments are made independently of the systems being compared, and the measurement procedure is stated before any results are seen. Others can then rerun the same experiment and obtain the same numbers. This is what a benchmark provides: not a guarantee that the winner is better for every purpose, but a claim that is reproducible and open to challenge.

Comparable benchmarks let retrieval research accumulate: each generation preserves the fixed collection, information needs, and judgments that make its results comparable with the last. The history below shows how Cranfield, TREC, multilingual campaigns, MS MARCO, and BEIR adapted that shared-evaluation idea to new collections and new failure modes.

From Cranfield to BEIR: how the field learned to compare systems (optional reading)

Shared evaluation is older than most of the methods it evaluates, and the paradigm has been revised roughly once per decade.

The Cranfield experiments, run by Cyril Cleverdon at the College of Aeronautics in Cranfield from the late 1950s into the 1960s, established the idea that retrieval quality can be measured offline. Cleverdon assembled a fixed set of documents, a fixed set of queries, and relevance judgments made by subject experts, then compared indexing methods against that fixed material. The abstraction was radical: rather than observing real users at work, the experiment freezes their needs into reusable test data. Every benchmark since has inherited both the power and the weakness of that move.

The Text REtrieval Conference, launched by the US National Institute of Standards and Technology in 1992, scaled the paradigm to collections far too large for exhaustive judging. TREC introduced pooling: participants submit their runs, the organizers merge the top-ranked documents from all submissions into a pool, and only the pool is judged. It also introduced tracks, so that ad-hoc search, question answering, legal discovery, and web search could each be evaluated with material suited to that task rather than by one universal test.

Comparable efforts followed for other language communities. CLEF began in 2000 in Europe, growing out of TREC’s cross-language work, and NTCIR started in Japan in the late 1990s with a focus on Japanese and other Asian languages. Both extended the paradigm to multilingual and cross-lingual retrieval, where a query in one language must find documents in another.

MS MARCO, released by Microsoft in 2016, changed the economics again. Built from real Bing queries, it was large enough to train neural ranking models rather than merely to test them, which is why it became the standard proving ground for learned retrieval. The price was sparse judgments: with hundreds of thousands of queries, only a small number of documents per query could be labeled.

BEIR, introduced in 2021, responded to a new failure mode. Models tuned on MS MARCO performed impressively on MS MARCO and often disappointed elsewhere. BEIR bundles eighteen heterogeneous collections and evaluates models without task-specific training, measuring whether a method generalizes rather than whether it has learned one benchmark well.

The Four Components of a Benchmark

A benchmark is built from four parts: a document collection, a set of information needs, relevance judgments, and a measurement procedure. Each part answers one question, and a weakness in any of them undermines everything measured on top.

The **document collection** defines the universe the system may search. It can hold news articles, scientific papers, legal filings, product descriptions, or, as in our running example, library records. Two properties matter most. The collection must be stable, because results are only comparable over time if the searched material does not change underneath them. It must also be representative, reflecting the document lengths, content types, and topic mix that the target users really encounter. Well-known examples include the TREC collections, which range from newspaper archives to biomedical abstracts, MS MARCO for web-style passage retrieval, and domain corpora such as PubMed Central and arXiv for scientific literature.

The **information needs** state what the users actually want. It is worth separating the need from the query string: the need is “which books give a beginning master’s student solid computer science foundations”, while the query is whatever short text the user types into the search box. Assessors judge against the need, not the keywords, which is why the need must be written out clearly enough that two people would interpret it the same way. Needs are best derived from real user behaviour such as query logs. When no log exists, large language models can draft candidate needs from samples of the collection, which expands coverage cheaply, though the drafts still require review to stay realistic.

The **relevance judgments** record, for each need, which documents satisfy it. They are the ground truth, so their quality bounds the quality of every conclusion. Human assessors work from written guidelines, receive training on example cases, and are checked for agreement, because vague criteria produce inconsistent labels that no metric can repair. Large language models can pre-screen documents and propose judgments for clear-cut cases, leaving human effort for the borderline ones. Judgments must then be frozen: revising them later silently invalidates every result measured earlier.

The **measurement procedure** turns a result list into a number. It fixes which measure is computed, at which cutoff, and how values are combined across needs. The next three sections develop these measures, starting with results treated as an unordered set, then as a ranking, then as a ranking with graded relevance. What matters here is that the procedure is chosen before the experiment, not after the results are known.

The following table collects the practical rules for each component.

Component	What it provides	Design rule	Common failure
Document collection	The universe that may be searched	Freeze it for the benchmark’s lifetime, make it representative of the target domain, preprocess consistently, and give every document a stable identifier	Built around one feature, so it flatters the methods that exploit that feature
Information needs	What the target users want	Derive needs from real user behaviour, state each one unambiguously, and provide 25 to 50 for a first experiment or 100 and more for a serious comparison	Wording so vague that assessors disagree and the ground truth becomes noisy
Relevance judgments	The ground truth for scoring results	Write assessor guidelines, train assessors, check their agreement, and freeze the labels once published	Judgments revised mid-project, which invalidates all earlier measurements

Component	What it provides	Design rule	Common failure
Measurement procedure	The rule that turns results into numbers	State the measure, the cutoff, and the averaging method before running any experiment	Measure chosen after seeing the results, which turns evaluation into advocacy

Building all four parts is expensive, which is why established benchmarks are reused heavily and why a small, carefully judged collection is often more valuable than a large, carelessly labeled one.

Judging at Scale

One practical obstacle remains. Judging every document for every need is impossible at realistic scale: fifty books can be assessed exhaustively, fifty million cannot. Pooling is the standard response. The organizers collect the documents that participating systems actually retrieved, judge that pool, and treat everything unjudged as not relevant. [Figure 2.1](#) shows the situation this assumes. Relevant documents that no participant retrieved never get judged, so the measured values differ somewhat from what exhaustive judging would produce. For a comparison that matters less than it first appears: every participant is scored against the same judged subset, so the ordering between them survives even though the individual numbers are approximations.

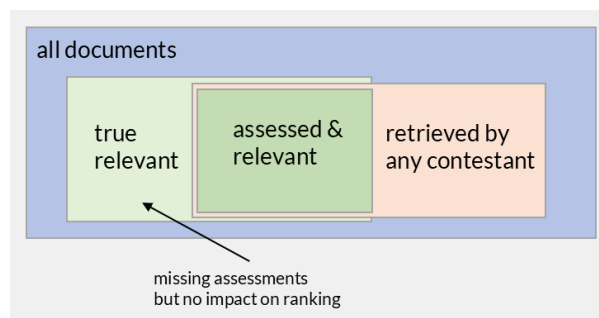


Figure 2.1: Pooled assessment when participating systems retrieve most relevant documents.

This assumption is sound while the pool covers most of the relevant material, and it weakens as the judged fraction shrinks. Pooling is one of several places where a benchmark can mislead rather than inform, and [When Evaluation Misleads](#) returns to all of them once we have the measures needed to describe what gets distorted.

Note

The later sections [Evaluating Text Classifiers](#) and [Scores, Thresholds, and ROC Curves](#) evaluate text classifiers rather than retrieval results, and the ground truth there is a test set rather than a benchmark of the kind described above. Three differences matter. The unit of evaluation is a single text item and its label, not a need-document pair. The labels are normally complete, so pooling does not apply, but class balance and label quality become the central concerns. And a classification dataset is split into training, validation, and test parts, with any decision threshold tuned on the validation part and never on the test part. [Evaluating Text Classifiers](#) develops these points where they are needed.

With a benchmark in place, the remaining question is mechanical: how do we turn a result list into a number that reflects how well the need was served? The next section answers it for results treated as an unordered set, using the same library collection and the same two systems.

Precision and Recall

Boolean retrieval returns a set of documents with no order attached: a document either matches the query or it does not. This section evaluates that kind of result. Ranking comes in [Evaluating Ranked Results](#); until then, a result list is treated purely as a set.

Recall System A and System B from [Designing a Retrieval Benchmark](#), both answering the same need: which books give a beginning master's student solid computer science foundations? System A returned 25 books, System B returned 8. To turn "returned" and "useful" into numbers, every book in the 50-book collection falls into exactly one of four groups, depending on whether the system retrieved it and whether it is actually relevant.

Counting Outcomes

	Relevant	Not relevant
Retrieved	True Positive (TP)	False Positive (FP)
Not retrieved	False Negative (FN)	True Negative (TN)

A true positive is a relevant book the system retrieved: a hit. A false positive is a book the system retrieved that turns out not to be relevant: noise in the result. A false negative is a relevant book the system failed to retrieve: a miss. A true negative is a book the system correctly left out. [Figure 2.2](#) shows this partition as overlapping sets for both systems: the 50-book collection, the 15 relevant books, the retrieved books, and their intersection.

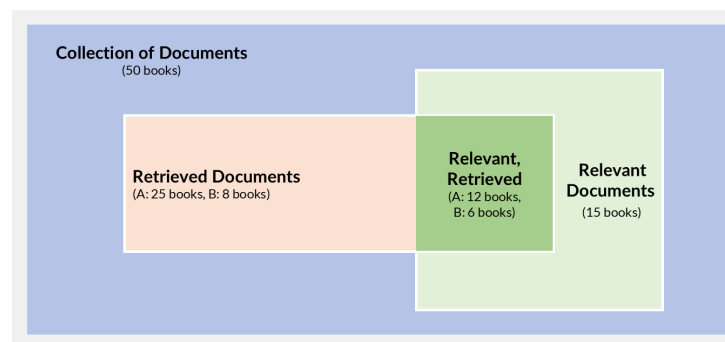


Figure 2.2: Set-theoretic view of retrieval performance for Systems A and B over the 50-book collection.

Counting each book in the collection against the two systems gives:

	TP	FP	FN	TN
System A	12	13	3	22
System B	6	2	9	33

Every row sums to 50, the size of the collection, and every count came directly from checking the 15 known relevant books against each system's result list.

Precision and Recall

The raw counts alone are hard to compare. System A has 12 true positives and System B has 6, but A also retrieved three times as many books and operates in a collection where 15 are relevant. A system retrieving the entire library would produce 15 true positives and look best by that count alone, which shows that an absolute count cannot separate a good system from one that simply returns everything. Ratios solve this: dividing by the appropriate total produces a value between 0 and 1 that is independent of how many books the system returned or how many relevant books the collection contains.

Key Formula: Precision and Recall

$$p = \frac{TP}{TP + FP} \quad r = \frac{TP}{TP + FN} \quad (2.1)$$

Precision is the fraction of retrieved documents that are relevant. Recall is the fraction of relevant documents that were retrieved.

Example: Precision and Recall for System A and B

System A retrieved 25 books and found 12 of the 15 relevant ones:

$$p_A = \frac{12}{12 + 13} = \frac{12}{25} = 48\% \quad r_A = \frac{12}{12 + 3} = \frac{12}{15} = 80\% \quad (2.2)$$

System B retrieved 8 books and found 6 of the 15 relevant ones:

$$p_B = \frac{6}{6 + 2} = \frac{6}{8} = 75\% \quad r_B = \frac{6}{6 + 9} = \frac{6}{15} = 40\% \quad (2.3)$$

This is the trade-off from [Designing a Retrieval Benchmark](#) stated in numbers rather than in words. System B's list is three-quarters useful material; System A's list finds four-fifths of everything useful but buries it among thirteen novels. Whether 48% precision at 80% recall is better than 75% precision at 40% recall still depends on what the user prefers (quick fact check or exhaustive library scan).

A third quantity, fallout, measures the opposite kind of mistake: how much of the irrelevant material got pulled in.

$$f = \frac{FP}{FP + TN} \quad (2.4)$$

Fallout is the fraction of non-relevant documents that the system retrieved. For our two systems, $f_A = 13/35 = 37.1\%$ and $f_B = 2/35 = 5.7\%$. Fallout measures the reading cost imposed on someone who inspects every result: the patent lawyer from [Designing a Retrieval Benchmark](#) needs high recall, because missing a document is expensive, but also wants low fallout, because every irrelevant document in the list costs time and attention. System A delivers recall of 80% but pulls in more than a third of the irrelevant collection; the ideal for an exhaustive search would push recall toward 1 while keeping fallout near 0.

Precision and recall are in tension

Retrieving more documents raises recall, since additional relevant ones may be included, and a system that returns the entire collection reaches perfect recall by construction. The cost is that more non-relevant documents come in as well, which lowers precision. In the other direction, a smaller result set tends to have higher precision, but at the risk of missing many of the relevant documents. In practice, improving one measure typically comes at the expense of the other, which is why both are always reported together.

Balancing the Two

Comparing two numbers side by side, as above, does not say which system wins. The F_β -measure collapses precision and recall into a single value, letting a scenario's priority set the trade-off explicitly.

Key Formula: F-beta Measure

$$F_\beta = \frac{(\beta^2 + 1) \cdot p \cdot r}{\beta^2 \cdot p + r} \quad (2.5)$$

The weighted harmonic mean of precision and recall. $\beta < 1$ favours precision, $\beta > 1$ favours recall, and $\beta = 1$ weighs them equally.

At $\beta = 0$ the formula reduces to precision alone; as $\beta \rightarrow \infty$ it reduces to recall alone. F_1 , the case $\beta = 1$, is the most common default and appears throughout machine learning wherever a single number is needed to compare classifiers.

Example: F-beta for System A and B

Using $p_A = 0.48$, $r_A = 0.80$ and $p_B = 0.75$, $r_B = 0.40$ from above:

β	Weighting	$F_\beta(A)$	$F_\beta(B)$	Winner
0.5	favours precision	0.522	0.638	B
1.0	equal weight	0.600	0.522	A
2.0	favours recall	0.706	0.441	A

The winner changes between $\beta = 0.5$ and $\beta = 1$. Neither system is better in general: a fact-checker who wants a short, clean list would set β below 1 and prefer System B, while a patent lawyer would set β above 1 and prefer System A. F_1 is a convenient default, not a neutral one, since it silently commits to weighing a false positive and a false negative equally.

Averaging Across Needs

One need produces one precision and one recall value. A real benchmark evaluates many needs, and summarising all of them into a single score requires an averaging method. The natural first idea is to compute precision and recall for each need separately and then take the arithmetic mean.

Example: Consider two additional needs alongside the CS-foundations query, answered by the same two systems:

Need	Relevant	System A: retrieved / found	System B: retrieved / found
CS foundations	15	25 / 12	8 / 6
Stage plays	5	6 / 4	4 / 3
Russian novels	1	3 / 0	1 / 1

The straightforward summary is the arithmetic mean of the per-need values:

Key Formula: Macro Averaging

$$p_{\text{macro}} = \frac{1}{N} \sum_{i=1}^N p_i \quad r_{\text{macro}} = \frac{1}{N} \sum_{i=1}^N r_i \quad (2.6)$$

Every need counts equally, regardless of how many relevant documents it has.

Example: Macro Averaging

Per-need recall for both systems:

Need	r_A	r_B
CS foundations	$12/15 = 0.800$	$6/15 = 0.400$
Stage plays	$4/5 = 0.800$	$3/5 = 0.600$
Russian novels	$0/1 = 0.000$	$1/1 = 1.000$

$$r_{\text{macro},A} = \frac{0.800 + 0.800 + 0.000}{3} = 53.3\% \quad (2.7)$$

$$r_{\text{macro},B} = \frac{0.400 + 0.600 + 1.000}{3} = 66.7\% \quad (2.8)$$

System B wins. Yet looking at the table, System A finds more relevant documents on two of the three needs and retrieves 16 of 21 relevant books in total, against B's 10. The single Russian-novels miss, a zero out of one, drags A's average down by 27 percentage points.

The problem is visible: macro-averaging gives each need the same weight $1/N$ in the sum, so a need with one relevant document contributes as much to the average as a need with fifteen. A single zero from the Russian-novels miss lowers the three-need average by a full $1/3$, which is a large negative impact relative to the system's overall performance across 21 relevant documents. This is intentional when every need matters equally, but it can misrepresent a system's aggregate capability.

Micro-averaging addresses this by pooling all outcome counts before computing the ratio. Instead of averaging per-need ratios, it sums the numerators and denominators across needs:

$$p_{\text{micro}} = \frac{\sum_{i=1}^N TP_i}{\sum_{i=1}^N (TP_i + FP_i)} \quad r_{\text{micro}} = \frac{\sum_{i=1}^N TP_i}{\sum_{i=1}^N (TP_i + FN_i)} \quad (2.9)$$

Every document counts equally. Needs with more relevant documents contribute more to the result.

Example: Micro Averaging

Pooling the recall counts across all three needs:

	$\sum TP$	$\sum (TP + FN)$	r_{micro}
System A	$12 + 4 + 0 = 16$	$15 + 5 + 1 = 21$	$16/21 = 76.2\%$
System B	$6 + 3 + 1 = 10$	$15 + 5 + 1 = 21$	$10/21 = 47.6\%$

System A now wins on recall, because the CS-foundations need, where A finds 12 of 15 books, contributes 15 of the 21 relevant documents to the pool and dominates the sum. The single Russian-novels miss adds only one false negative to a denominator of 21 and barely registers.

Same systems, same needs, opposite conclusions on recall: macro says B is better, micro says A is better. The difference reflects a genuine choice about what matters.

Macro averaging is the standard in retrieval evaluation

In retrieval evaluation, the standard practice is macro-averaging: MAP, mean nDCG, and mean MRR all compute a per-topic score and then take the arithmetic mean. Micro-averaging across topics is rarely used, because weighting each document equally would let one large topic dominate all others. The macro/micro choice becomes more relevant in [Evaluating Text Classifiers](#), where classes are often imbalanced and the two methods can disagree sharply. When reporting retrieval results, specify both the measure and the number of topics it was averaged over.

Architecture and the Metrics

[Retrieval Systems and Flows](#) introduced retriever-only, retriever-with-filter, and retriever-ranker architectures. Precision and recall explain why each stage exists and how each is evaluated. The retriever is optimized for recall: it casts a wide net over the collection, accepting low precision because a false positive can still be removed later, while a false negative is lost for good. The ranker or filter is optimized for precision: it reorders or removes candidates so that the top of the list is dense with relevant material. Even a user who cares only about a precise top-ten result depends on the retriever's recall, because the ranker can only surface documents that the retriever found in the first place.

Hands-on: Precision and Recall

Recompute every number in this section from the library collection, then edit a run and watch precision, recall, and F_β respond. [Open notebook ->](#)

Includes pre-run results: you can read through or download and experiment.

Precision, recall, and their averages describe a result set with no notion of order. The next section brings rank back into the picture: whether a system found the right documents matters less if they never appear near the top of the list.

Evaluating Ranked Results

Precision and recall treat a result set as a bag: a document is either in the set or not, and the metrics do not change if the order is shuffled. A ranked retrieval system does more than decide which documents to return; it decides which ones to show first. A user who reads from the top and stops after five items gets a very different experience depending on whether the relevant books are at ranks 1 and 2 or at ranks 16 and 18. Evaluation must account for position, not only membership.

Recall the first five positions from [Designing a Retrieval Benchmark](#):

Rank	System A result	Useful?	System B result	Useful?
1	Computer Organization and Design	yes	Database Systems: The Complete Book	yes
2	Operating System Concepts	yes	Pattern Recognition and Machine Learning	yes
3	The Handmaid's Tale	no	Introduction to Algorithms	yes
4	Narrative of the Life of Frederick Douglass	no	On the Origin of Species	no
5	Towards a New Architecture	no	Introduction to the Theory of Computation	yes

System A retrieves 25 books total (12 of 15 relevant); System B retrieves 8 (6 of 15 relevant). The set-based precision numbers from [Precision and Recall](#) already showed that B is more precise overall, but ranking adds a second dimension: how quickly does the user reach useful material? System A places two relevant books at the top, then three misses; System B delivers four of five.

Precision at Rank k

The simplest rank-aware metric evaluates only the first k positions. Precision at rank k counts how many of the top- k documents are relevant, ignoring everything below.

Key Formula: Precision at k

$$P@k = \frac{|\{D_1, \dots, D_k\} \cap \text{Rel}|}{k} \quad (2.10)$$

The fraction of relevant documents among the first k results.

$P@k$ answers one practical question: if the user reads only the first k items, what proportion is useful? It does not require knowing how many relevant documents exist in the collection, which makes it easy to compute and interpret. It also does not penalize a system for missing relevant documents below rank k : a system that places one relevant book at rank 1 and nothing else achieves $P@1 = 1.0$ regardless of how many relevant books it missed.

Example: $P@k$ for System A and B

k	$P@k$ (A)	$P@k$ (B)
3	$2/3 = 66.7\%$	$3/3 = 100\%$
5	$2/5 = 40.0\%$	$4/5 = 80.0\%$
10	$5/10 = 50.0\%$	$6/10 = 60.0\%^*$

*System B returns only 8 documents. Positions 9 and 10 are empty, counted as non-relevant.

System B dominates at every cutoff. A student who reads the first five results gets four useful books from B but only two from A. $P@k$ captures this user experience directly.

P@k ignores recall

$P@k$ rewards a system for being precise at the top but says nothing about how many relevant documents exist below rank k . Two systems with identical $P@5$ can differ drastically in total recall: one may have surfaced all relevant documents, the other may have missed most of them. Report $P@k$ alongside recall or use metrics that integrate both.

Reciprocal Rank and MRR

Some information needs have a single correct answer, or the user is satisfied as soon as one relevant result appears. A library patron asking “Who wrote Dracula?” needs exactly one fact; seeing it at rank 1 is ideal, at rank 20 is frustrating. The reciprocal rank captures this notion.

Key Formula: Reciprocal Rank and MRR

$$RR = \frac{1}{\text{rank of first relevant document}} \quad MRR(Q) = \frac{1}{|Q|} \sum_{i=1}^{|Q|} RR_i \quad (2.11)$$

The reciprocal of the position where the user first finds what they need, averaged across a set of queries Q .

$RR = 1$ when the first result is relevant; $RR = 0.5$ when the first relevant result is at rank 2; $RR \rightarrow 0$ as the relevant document is pushed further down. If no relevant document appears in the result list at all, $RR = 0$.

Example: MRR for fact lookups

MRR is best demonstrated on fact-lookup queries where exactly one answer exists. Suppose four students each ask the library catalogue a different question, and both systems return a ranked list. The only thing that matters is how far down the student must scroll to find the answer:

Query	Answer	System A: rank	RR_A	System B: rank	RR_B
“Who wrote Dracula?”	Dracula	1	1.000	3	0.333
“A book on operating system design?”	Operating System Concepts	4	0.250	1	1.000
“A nineteenth-century Russian novel?”	Crime and Punishment	6	0.167	2	0.500
“A dystopian novel by Atwood?”	The Handmaid’s Tale	2	0.500	5	0.200

$$MRR_A = \frac{1.000 + 0.250 + 0.167 + 0.500}{4} = 0.479 \quad (2.12)$$

$$MRR_B = \frac{0.333 + 1.000 + 0.500 + 0.200}{4} = 0.508 \quad (2.13)$$

Neither system dominates: A answers one query instantly ($RR = 1$) but buries the Russian novel at rank 6, while B also has one perfect answer but pushes the Atwood question to rank 5. The averages are close, which tells us the two systems surface first answers at comparable speed overall despite different per-query strengths.

MRR is widely used in question answering and known-item search, where one good answer suffices. It ignores what happens after the first relevant document: a system that places five relevant books at ranks 1 through 5 scores the same $RR = 1$ as one that places a single relevant book at rank 1 and nothing else. For needs where multiple relevant documents matter, as in our CS-foundations example with 15 relevant books, MRR is the wrong tool.

The Precision-Recall Curve

$P@k$ evaluates one fixed cutoff. A more complete picture plots precision against recall at every rank where a relevant document appears. As the system returns more documents, recall increases (more relevant documents are found) and precision typically decreases (more non-relevant documents accumulate). Plotting precision against recall produces the precision-recall curve.

At each rank i in the result list, define:

$$P_i = \frac{|\{d_1, \dots, d_i\} \cap \text{Rel}|}{i} \quad R_i = \frac{|\{d_1, \dots, d_i\} \cap \text{Rel}|}{|\text{Rel}|} \quad (2.14)$$

A point (R_i, P_i) is recorded only at ranks where a relevant document is retrieved: those are the only ranks where recall changes. Between two relevant documents, recall stays constant while precision drops (the denominator i grows with no new relevant hit), so these in-between ranks do not add new curve points.

Example: Precision-Recall curve for System A

The relevant documents in System A's ranking appear at ranks 1, 2, 6, 7, 9, 12, 13, 16, 18, 21, 23, and 25. At each of those positions:

Rank	Relevant found	P_i	R_i
1	1	$1/1 = 1.000$	$1/15 = 0.067$
2	2	$2/2 = 1.000$	$2/15 = 0.133$
6	3	$3/6 = 0.500$	$3/15 = 0.200$
7	4	$4/7 = 0.571$	$4/15 = 0.267$
9	5	$5/9 = 0.556$	$5/15 = 0.333$
12	6	$6/12 = 0.500$	$6/15 = 0.400$
13	7	$7/13 = 0.538$	$7/15 = 0.467$
16	8	$8/16 = 0.500$	$8/15 = 0.533$
18	9	$9/18 = 0.500$	$9/15 = 0.600$
21	10	$10/21 = 0.476$	$10/15 = 0.667$
23	11	$11/23 = 0.478$	$11/15 = 0.733$
25	12	$12/25 = 0.480$	$12/15 = 0.800$

The curve starts at $(0.067, 1.0)$ and ends at $(0.800, 0.480)$. After rank 2, three consecutive non-relevant books push precision down before the next relevant book at rank 6 lifts it back to 0.5. The curve never reaches recall 1.0 because System A misses 3 of the 15 relevant books.

Reading the Curve

Figure 2.3 annotates the key regions of a precision-recall curve. The upper-left corner (high precision, low recall) is where a fact-checker operates: a short, clean result list with few mistakes. The lower-right corner (high recall, low precision) is where a patent lawyer operates: an exhaustive search that tolerates noise to avoid missing anything. The diagonal arrow labelled “system efficiency” points toward the ideal corner $(1, 1)$: the closer a curve pushes toward that point, the better the system balances both goals simultaneously.

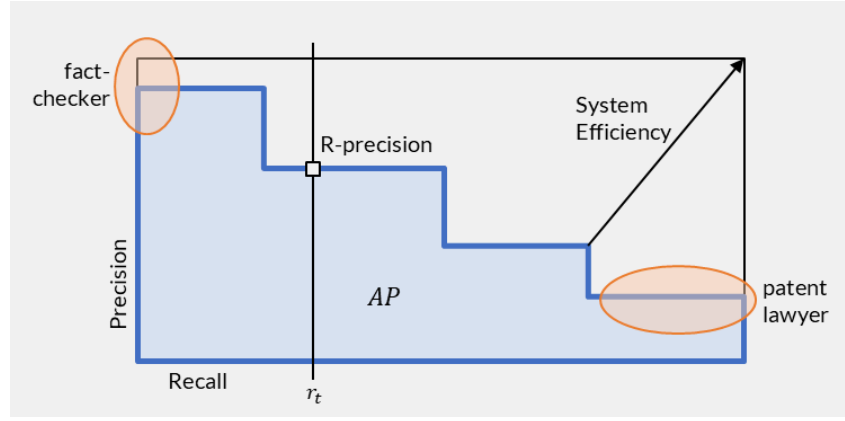


Figure 2.3: Precision-recall curve with key evaluation landmarks: the shaded area is Average Precision (AP), R-precision marks the point where the number of results equals the number of relevant documents, and the two user profiles illustrate the trade-off between precision-oriented and recall-oriented search.

The shaded area under the curve represents Average Precision (AP), which the next subsection defines formally. The figure marks R-precision at rank R , where R is the number of relevant documents. At this cutoff, precision and recall are numerically equal because both divide the number of relevant documents retrieved in the first R positions by R .

Interpolation for Cross-System Comparison

The 12 points in the table above form a smooth, largely decreasing scatter for one system and one need. The need for interpolation arises when we want to compare two systems or average curves across multiple needs: each system produces points at different recall levels, so we cannot directly subtract or average their precisions. Interpolation defines precision at any recall level r as the maximum precision achieved at any recall level $r' \geq r$:

$$P_{\text{interp}}(r) = \max_{r' \geq r} P(r') \quad (2.15)$$

This produces a non-increasing step function evaluated at standard recall levels (0, 0.1, 0.2, ..., 1.0). Two systems evaluated on the same 11 recall points can be compared point by point or averaged across queries. For System A, interpolated precision at recall 0 is 1.0 (the best precision it ever achieves), while at recall 0.9 and 1.0 it is 0 (the system never reaches those recall levels).

R-Precision

R-precision links precision to recall through a single number. If the collection contains R relevant documents for a need, R-precision is the precision after exactly R documents have been retrieved.

$$\text{R-Prec} = P@R = \frac{|\{D_1, \dots, D_R\} \cap \text{Rel}|}{R} \quad (2.16)$$

A perfect system would place all R relevant documents in the first R positions, achieving R-precision of 1.0. The measure adjusts itself to the difficulty of the need: a need with 15 relevant documents (our CS-foundations example) evaluates at cutoff 15, while a need with 2 relevant documents evaluates at cutoff 2.

Example: R-Precision

For the CS-foundations need, $R = 15$. System A places 7 relevant books in its first 15 positions:

$$\text{R-Prec}(A) = \frac{7}{15} = 0.467 \quad (2.17)$$

System B returns only 8 documents total. Positions 9 through 15 contain nothing (treated as non-relevant), so the 6 relevant books it returned are all that count:

$$\text{R-Prec}(B) = \frac{6}{15} = 0.400 \quad (2.18)$$

System A wins here because its longer result list gives it more chances to place relevant books in the first 15 positions (7 out of 25 retrieved), while System B returns only 8 books total, so positions 9 through 15 are empty and count against it.

R-precision is simple and self-adjusting, but it reduces an entire ranking to one point. The metrics that follow evaluate the full curve.

Average Precision

Average precision (AP) collapses the precision-recall curve into a single number. Geometrically, it approximates the area under the precision-recall curve: the shaded region in Figure 2.3. A system whose curve stays high across all recall levels has a large area and a high AP; one whose precision collapses early has a small area and a low AP.

The computation is straightforward: at each rank where a relevant document appears, record the precision at that rank. Then take the mean of those values, using the total number of relevant documents in the collection as the divisor.

Key Formula: Average Precision

$$AP = \frac{1}{|\text{Rel}|} \sum_{k: d_k \in \text{Rel}} P_k \quad (2.19)$$

where P_k is the precision at rank k , and the sum runs only over the ranks at which a relevant document appears. The divisor is the total number of relevant documents in the collection, not the number retrieved.

Dividing by $|\text{Rel}|$ rather than by the number of relevant documents actually found means that missed documents contribute implicit zeros: they never appear in the ranking, so they add nothing to the numerator, but the denominator still counts them. This is what makes AP sensitive to recall, not only to precision at the top.

AP uses the raw (non-interpolated) precision values. Interpolation serves a different purpose: it defines precision at arbitrary recall levels so that curves from different systems can be compared point by point or averaged. AP does not require that step because it sums only at the fixed rank positions where relevant documents actually occur.

Example: Average Precision for System A and B

System A finds 12 of 15 relevant documents. The precision at each relevant rank:

$$AP_A = \frac{1}{15} \left(\frac{1}{1} + \frac{2}{2} + \frac{3}{6} + \frac{4}{7} + \frac{5}{9} + \frac{6}{12} + \frac{7}{13} + \frac{8}{16} + \frac{9}{18} + \frac{10}{21} + \frac{11}{23} + \frac{12}{25} \right) = 0.473 \quad (2.20)$$

The three relevant books that System A missed (ranks would be needed beyond 25, or they were never retrieved) each contribute 0 to the sum, pulling the average down.

System B finds 6 of 15 relevant documents, all placed high:

$$AP_B = \frac{1}{15} \left(\frac{1}{1} + \frac{2}{2} + \frac{3}{3} + \frac{4}{5} + \frac{5}{6} + \frac{6}{8} \right) = 0.359 \quad (2.21)$$

System B's precisions at the positions where it finds relevant documents are higher (it averages 0.897 over its 6 hits vs. A's 0.592 over its 12 hits), but it misses 9 of 15 relevant books. Those 9 zeros in the denominator dominate: $AP_B < AP_A$.

AP rewards both placing relevant documents high (each precision term is larger when fewer non-relevant documents precede it) and finding more relevant documents (more terms contribute to the sum). This makes

it the single best summary of a ranking for a given need: unlike $P@k$ it accounts for the full list, and unlike MRR it values every relevant document, not only the first.

Mean Average Precision

Evaluating a system on one need gives one AP value. A benchmark with multiple needs produces an AP per need, and mean average precision (MAP) averages them.

$$MAP = \frac{1}{|Q|} \sum_{i=1}^{|Q|} AP_i \quad (2.22)$$

MAP is a macro-average: each need contributes equally regardless of how many relevant documents it contains. This is the same averaging strategy discussed in [Precision and Recall](#), and the same caveats apply: a need with one relevant document has as much influence on MAP as a need with fifteen.

Example: MAP across four needs

Using the four needs from our library example:

Need	Relevant	AP_A	AP_B
CS foundations	15	0.473	0.359
Stage plays	5	0.587	0.600
Atwood books	2	0.833	1.000
Russian novels	1	0.000	1.000

$$MAP_A = \frac{0.473 + 0.587 + 0.833 + 0.000}{4} = 0.473 \quad (2.23)$$

$$MAP_B = \frac{0.359 + 0.600 + 1.000 + 1.000}{4} = 0.740 \quad (2.24)$$

System B wins on MAP despite finding far fewer relevant documents on the largest need. Two factors combine: B has perfect AP on two smaller needs, while A's zero on Russian novels costs 0.25 in the average, and A also fails to find one of the five plays in its stage-plays run.

MAP is the standard primary metric in TREC and most retrieval benchmarks. A single MAP value summarizes an entire system's behaviour across all evaluated needs. When two systems are compared, a paired test (such as a paired t -test or Wilcoxon signed-rank test) on the per-need AP values determines whether the difference is statistically significant, not just the result of a few easy or hard queries.

Choosing a Metric

Each metric answers a different question about the ranking:

Metric	Question it answers	Sensitive to
$P@k$	How clean are the first k results?	Precision at a fixed cutoff
MRR	How quickly does the user find one good result?	Position of the first relevant document
R -Prec	How well does the system fill the first R slots?	Balance of precision and recall at a natural cutoff
AP	How good is the overall ranking for one need?	Both precision and recall, position-weighted
MAP	How good is the system across many needs?	Per-need AP, macro-averaged

$P@k$ and MRR suit settings where users inspect only the top few results: web search, question answering, voice assistants. AP and MAP suit settings where the full ranking matters: patent search, systematic

reviews, benchmark comparisons. R-precision provides a single summary that adjusts to the number of relevant documents per need.

All of these metrics treat relevance as binary: a document is relevant or it is not. The next section relaxes this assumption, introducing graded relevance where some documents are more useful than others, and metrics that reward a system for placing highly relevant documents above marginally relevant ones.

Hands-on: Ranked Evaluation

Compute P@k, AP, and MAP from the library rankings, then reorder documents and watch the metrics respond. [Open notebook ->](#)

Includes pre-run results: you can read through or download and experiment.

Evaluating Graded Relevance

All metrics so far treat relevance as binary: a document is relevant or it is not. Average precision, for instance, gives System A's rank-1 result ("Computer Organization and Design") the same credit as System B's rank-1 result ("Database Systems: The Complete Book"), because both are relevant. For the CS-foundations need, we assign three relevance grades: core foundations (grade 3), important specializations (grade 2), and useful peripheral reading (grade 1). "Database Systems" is a grade-3 core text; "Computer Organization and Design" is grade-1 peripheral reading. A metric that sees only "relevant" cannot tell these apart.

Of the 15 relevant books in the collection, 3 are graded as core foundations, 4 as important specializations, and 8 as useful peripheral reading. The complete grade distribution:

Grade	Meaning	Count	Examples
3	Core foundations	3	Database Systems, Pattern Recognition, Data Science from Scratch
2	Important specialization	4	Operating Systems, AI: A Modern Approach, Programming Languages, Theory of Computation
1	Useful peripheral reading	8	Algorithms, Computer Networks, SICP, Cryptography, ...
0	Not relevant	35	Fiction, poetry, drama, general non-fiction

Consider the first five results from each system, now labelled with grades:

Rank	System A	Grade	System B	Grade
1	Computer Organization and Design	1	Database Systems: The Complete Book	3
2	Operating System Concepts	2	Pattern Recognition and Machine Learning	3
3	The Handmaid's Tale	0	Introduction to Algorithms	1
4	Narrative of the Life of Frederick Douglass	0	On the Origin of Species	0
5	Towards a New Architecture	0	Introduction to the Theory of Computation	2

Binary AP cannot distinguish these two top-5 lists beyond counting "2 relevant at top for A, 4 for B". Graded evaluation does more: it rewards B for placing its two highest-value books (both grade 3) in the most visible positions, while A placed a peripheral text (grade 1) at rank 1 and pushed its grade-3 titles to ranks 6, 12, and 18.

A second limitation of AP is subtler. AP rewards a system for placing any relevant document higher, but it does not penalize a system that places relevant documents in a sub-optimal order among themselves. If two relevant documents appear at ranks 3 and 5, AP does not care which of the two is more important. A graded metric can: it assigns more credit when a highly relevant document sits above a marginally relevant one.

Cumulative Gain

The simplest graded metric just sums the relevance grades of the documents returned up to rank k :

Key Formula: Cumulative Gain

$$CG_k = \sum_{i=1}^k rel_i \quad (2.25)$$

The total relevance value accumulated in the first k positions. Higher grades contribute more.

CG_k measures how much total value the user has gathered after reading k results. It is easy to compute but ignores order entirely: swapping a grade-3 book from rank 10 to rank 1 does not change CG_{10} .

Example: Cumulative Gain at k = 5

From the table above:

$$CG_5(A) = 1 + 2 + 0 + 0 + 0 = 3 \quad (2.26)$$

$$CG_5(B) = 3 + 3 + 1 + 0 + 2 = 9 \quad (2.27)$$

System B delivers three times the accumulated value in the first five positions. But if we shuffled B's list to put grade-0 at rank 1 and grade-3 at rank 5, CG_5 would remain 9. Rank does not matter to CG.

Discounted Cumulative Gain

To make position matter, we discount contributions from lower ranks. The idea is that a user reading from the top gets diminishing benefit from each additional position: finding a grade-3 book at rank 1 is more valuable than finding the same book at rank 10, because by rank 10 the user may have already stopped reading, or the earlier results have already partially satisfied the need.

Key Formula: Discounted Cumulative Gain

$$DCG_k = \sum_{i=1}^k \frac{rel_i}{\log_2(i+1)} \quad (2.28)$$

Each relevance grade is divided by a logarithmic discount factor that grows with rank. Rank 1 has no discount ($\log_{22} = 1$); rank 10 discounts by $\log_{211} \approx 3.46$.

The logarithmic discount encodes a model of user patience: the benefit of each additional rank position decreases, but never reaches zero. A document at rank 10 still contributes, just less than the same document at rank 1.

Example: DCG at k = 5

Rank	Discount $\frac{1}{\log_2(i+1)}$	A: grade	A: contribution	B: grade	B: contribution
1	$1/1.000 = 1.000$	1	1.000	3	3.000
2	$1/1.585 = 0.631$	2	1.262	3	1.893
3	$1/2.000 = 0.500$	0	0.000	1	0.500
4	$1/2.322 = 0.431$	0	0.000	0	0.000
5	$1/2.585 = 0.387$	0	0.000	2	0.774

$$DCG_5(A) = 1.000 + 1.262 + 0 + 0 + 0 = 2.262 \quad (2.29)$$

$$DCG_5(B) = 3.000 + 1.893 + 0.500 + 0 + 0.774 = 6.1665 \quad (2.30)$$

The discount makes position visible. Both systems have a grade-2 book in the top 5, but A places it at rank 2 (contribution $2 \times 0.631 = 1.262$) while B places its grade-2 book at rank 5 (contribution $2 \times 0.387 = 0.774$). Same grade, same book quality, but A earns 63% more credit from it by showing it earlier.

Discounting with Binary Relevance

DCG works even when all relevance grades are binary (0 or 1). In that case, it reduces to summing the discount factors at positions where a relevant document appears. This isolates the effect of rank position from the effect of graded relevance, which helps clarify what each component contributes.

Example: DCG@10 with binary relevance

Treating every relevant document as grade 1 and every non-relevant as grade 0:

	System A	System B
Relevant in top 10	ranks 1, 2, 6, 7, 9	ranks 1, 2, 3, 5, 6, 8
DCG_{10} (binary)	$1.000 + 0.631 + 0.356 + 0.333 + 0.301 = 2.6215$	$1.000 + 0.631 + 0.500 + 0.387 + 0.356 + 0.315 = 3.1895$

System B scores higher because it places relevant documents more densely near the top (3 in the first 3 ranks vs. A's 2 in the first 2 ranks, then a gap). With graded relevance, B's lead grows further because its top-ranked documents also carry higher grades.

Normalized DCG

DCG values depend on the number of relevant documents that exist and on the magnitude of the grades, so raw DCG scores cannot be compared across different queries. A query with five grade-3 documents in the collection can produce a much higher DCG than one with two grade-1 documents, regardless of system quality.

Normalization divides the actual DCG by the best possible DCG for that query: the score achieved by an ideal ranking that places documents in descending order of their grades.

Key Formula: Normalized DCG

$$nDCG_k = \frac{DCG_k}{IDCG_k} \quad (2.31)$$

where $IDCG_k$ is the DCG of the ideal ranking: all relevant documents sorted by decreasing grade, placed at ranks 1 through k .

$nDCG$ always falls between 0 and 1. A value of 1 means the system produced the best possible ranking for that query at cutoff k . A value of 0 means no relevant document appeared in the first k positions.

Example: $nDCG@10$ for System A and B

The ideal ranking for the CS-foundations need places the three grade-3 books first, then the four grade-2 books, then grade-1 books:

Rank	Ideal grade	Discount	Contribution
1	3	1.000	3.000
2	3	0.631	1.893
3	3	0.500	1.500
4	2	0.431	0.861
5	2	0.387	0.774
6	2	0.356	0.712
7	2	0.333	0.667
8	1	0.315	0.315
9	1	0.301	0.301
10	1	0.289	0.289

$$IDCG_{10} = 10.3125 \quad (2.32)$$

The actual $DCG@10$ values from above (with graded relevance):

$$nDCG_{10}(A) = \frac{4.5992}{10.3125} = 0.4460 \quad nDCG_{10}(B) = \frac{7.8253}{10.3125} = 0.7588 \quad (2.33)$$

System B achieves 76% of the ideal ranking quality; System A achieves only 45%.

What nDCG Reveals That AP Does Not

The contrast between AP and nDCG on our running example is instructive:

Metric	System A	System B	Winner
AP (binary, full list)	0.473	0.359	A
nDCG@10 (binary)	0.577	0.702	B
nDCG@10 (graded)	0.446	0.759	B

AP favours System A because AP rewards finding many relevant documents (A finds 12 vs. B's 6), and the denominator $|Rel| = 15$ heavily penalizes B's 9 misses. The nDCG metrics tell a different story because they evaluate only the top 10 positions: within that window, B places more relevant, higher-grade material near the top.

Switching from binary to graded nDCG widens B's lead further. Under binary evaluation, A's rank-1 book and B's rank-1 book both score 1.0 (both are "relevant"). Under graded evaluation, B's grade-3 book at rank 1 scores 3.0 while A's grade-1 book scores only 1.0. Graded relevance captures what a user experiences: not all useful results are equally useful.

nDCG and AP answer different questions

AP asks: "How good is the complete ranking at placing relevant documents high and finding all of them?" It integrates precision over the full result list and penalizes missed documents. nDCG@ k asks: "How much value does the user get from reading the first k results?" It evaluates a fixed window, rewards quality ordering within that window, and does not penalize a system for what it fails to retrieve beyond position k .

The right metric depends on the information need. A patent lawyer conducting an exhaustive prior-art search needs high recall and should evaluate with AP or MAP. A web search user or fact-checker who reads a handful of results cares about nDCG@10 or nDCG@5.

Because the logarithmic discount shrinks toward zero, each additional position contributes less to the sum. Beyond a certain depth, nDCG effectively stops changing: a document at rank 50 contributes only $1/\log_2(51) \approx 0.18$ times its grade, while rank 1 contributes the full grade. For this reason, very large values of k add little discriminative power between systems and are rarely used in practice. Standard cutoffs align with how far users actually read: $k = 10$ for a web search results page (the BEIR and MS MARCO benchmarks report nDCG@10), $k = 20$ for TREC Web Track, and $k = 3$ for voice assistants or mobile interfaces.

Practical Notes

nDCG is the standard metric in web search evaluation (where users rarely look past the first page) and in leaderboard-style benchmarks like MS MARCO and BEIR. The cutoff k is chosen to match the application. The grading scale affects what nDCG rewards. Our linear scale (0, 1, 2, 3) treats the jump from grade 0 to grade 1 as equivalent to the jump from grade 2 to grade 3. An exponential gain variant replaces rel_i with $2^{rel_i} - 1$, which amplifies the difference between high and low grades: a grade-3 document contributes $2^3 - 1 = 7$ while a grade-1 document contributes only $2^1 - 1 = 1$.

Example: Exponential gain variant

Applying the $2^{rel_i} - 1$ gain function to our systems (grade 1 gains 1, grade 2 gains 3, and grade 3 gains 7):

	Linear gain $nDCG_{10}$	Exponential gain $nDCG_{10}$
System A	0.446	0.358
System B	0.759	0.804

System B's lead widens under exponential gain. Its three grade-3 books (gain 7 each) earn far more than A's grade-1 and grade-2 books near the top. System A's rank-1 grade-1 book contributes 1 under both gain functions. Its ratio falls because exponential gain makes the three grade-3 books in the ideal ranking much more valuable, while A places only one of them in its top 10 and only at rank 6.

The exponential variant is the default in many evaluation toolkits and benchmark leaderboards. When comparing published nDCG scores, always check whether linear or exponential gain was used; the two are not comparable.

Hands-on: Graded Evaluation

Compute CG, DCG, and nDCG from the library rankings with graded judgments. Swap documents between positions and observe how discounting amplifies or dampens the effect. [Open notebook ->](#)

Includes pre-run results: you can read through or download and experiment.

Evaluating Text Classifiers

Retrieval systems do not exist in isolation. A search engine for images relies on classifiers that detect faces, identify objects, or assign scene categories before any query arrives. An audio retrieval system uses a classifier to segment speech from music. A filtering stage in a retrieval pipeline decides “does this candidate pass the criteria?”: a binary classification decision embedded inside retrieval. These classifiers are components of the retrieval pipeline, and they need their own evaluation. The confusion matrix and its metrics appear every time a system makes a binary or multiclass decision, whether that decision is the final answer or a preprocessing step that feeds into ranking.

The underlying evaluation question is familiar: how often does the system make the right decision? Precision and recall still apply. What changes is the framing. There is no “retrieved set” to measure against a collection; instead, the system assigns exactly one label to each item in a test set, and we compare those predictions against ground-truth labels. Three properties make classification evaluation distinct from retrieval evaluation:

1. **No ranking.** The system outputs a label, not a position. There is no notion of “rank 3 is better than rank 10”.
2. **Prevalence matters.** The proportion of positive items in the test set affects how we interpret every metric. A rare disease affects 1% of patients; a common spam folder holds 40% junk.
3. **Error costs are often asymmetric.** A false positive in spam filtering deletes a legitimate email; a false negative lets spam through. These are not equally bad.

The Confusion Matrix

The confusion matrix organizes every prediction the system made into a 2×2 table for binary classification. Figure 2.4 shows this arrangement. Each item in the test set falls into exactly one cell, depending on the system’s prediction and the true label.

		Actual Condition (as observed)	
		Positive (P)	Negative (N)
Predicted Condition (as computed)	True (T)	True Positive (TP)	False Positive (FP)
	False (F)	False Negative (FN)	True Negative (TN)

Figure 2.4: The binary confusion matrix. Rows represent predictions, columns represent actual conditions. Green cells are correct predictions; orange cells are errors.

The row sums give the number of items the system labelled positive ($T = TP + FP$) and negative ($F = FN + TN$). The column sums give the actual counts: $P = TP + FN$ positive items and $N = FP + TN$ negative items in the test set. The total population is $P + N$.

From these four counts, we derive the same metrics introduced for retrieval, plus several that become important only in classification:

Key Formula: Classification Metrics

$$\text{Precision (PPV)} = \frac{TP}{TP + FP} \quad \text{Recall (TPR)} = \frac{TP}{TP + FN} \quad (2.34)$$

$$\text{Specificity (TNR)} = \frac{TN}{TN + FP} \quad \text{Accuracy} = \frac{TP + TN}{P + N} \quad (2.35)$$

Precision asks: of all items the system called positive, how many truly are? Recall asks: of all truly positive items, how many did the system find? Specificity asks: of all truly negative items, how many did the system correctly leave alone? Accuracy asks: what fraction of all decisions is correct?

Precision and recall are identical to the retrieval definitions from [Precision and Recall](#), just viewed from a different angle: “retrieved” becomes “predicted positive”, and “relevant” becomes “actually positive”. Specificity is new; in retrieval we called its complement “fallout” ($FPR = 1 - \text{Specificity}$). Accuracy had no retrieval equivalent because retrieval operates on a ranked list, not a fixed yes/no decision for every item.

Figure 2.5 summarizes how the four outcome counts produce the classification metrics and their complementary rates.

		Actual Condition (as observed)			
		Population	Positive (P)	Negative (N)	
Predicted Condition (as computed)	True (T)	True Positive (TP)	False Positive (FP)	Positive Predictive Value (PPV), Precision	False Discovery Rate (FDR)
	False (F)	False Negative (FN)	True Negative (TN)	False Omission Rate (FOR)	Negative Predictive Value (NPV)
		True Positive Rate (TPR), Sensitivity, Recall, Hit Rate	False Positive Rate (FPR), Fall-Out	Accuracy (ACC)	
		False Negative Rate (FNR), Miss Rate	True Negative Rate (TNR), Specificity	Error Rate (ERR), Misclassification Rate	

$TPR = \frac{TP}{P}$	$FPR = \frac{FP}{N}$	$PPV = \frac{TP}{T}$	$FDR = \frac{FP}{T}$	$PPV + FDR = 1$
$FNR = \frac{FN}{P}$	$TNR = \frac{TN}{N}$	$FOR = \frac{FN}{F}$	$NPV = \frac{TN}{F}$	$FOR + NPV = 1$
$TPR + FNR = 1$	$FPR + TNR = 1$			

$ACC = \frac{TP + TN}{P + N}$	$ERR = \frac{FP + FN}{P + N}$	$ACC + ERR = 1$
-------------------------------	-------------------------------	-----------------

Figure 2.5: The confusion matrix and its derived metrics. Column-normalized rates (TPR, FPR, FNR, TNR), row-normalized predictive values (PPV, NPV), and overall accuracy, with all complementary pairs summing to one.

Example: Spam Filtering

A university email server processes 1000 messages in one day. Of these, 400 are spam ($P = 400$) and 600 are legitimate ($N = 600$). The spam filter makes the following decisions:

	Actual Spam (400)	Actual Legitimate (600)
Predicted Spam	TP = 360	FP = 30
Predicted Legitimate	FN = 40	TN = 570

Example: Spam filter metrics

$$\text{Precision} = \frac{360}{360 + 30} = \frac{360}{390} = 92.3\% \quad (2.36)$$

$$\text{Recall} = \frac{360}{360 + 40} = \frac{360}{400} = 90.0\% \quad (2.37)$$

$$\text{Specificity} = \frac{570}{570 + 30} = \frac{570}{600} = 95.0\% \quad (2.38)$$

$$\text{Accuracy} = \frac{360 + 570}{1000} = 93.0\% \quad (2.39)$$

All four numbers look healthy: the filter catches 90% of spam, and 92% of what it flags is genuinely spam. But consider the 30 false positives: those are 30 legitimate emails moved to the junk folder. If one of them is an acceptance letter for a grant proposal, the cost of that single FP dwarfs the annoyance of 40 spam messages reaching the inbox.

Which error is worse depends on the application

In spam filtering, a false positive (legitimate email deleted) is typically worse than a false negative (spam reaches inbox). The user can ignore spam in the inbox; they cannot read an email they never saw. This asymmetry means that optimizing for high recall (catching all spam) at the cost of precision (more legitimate emails lost) is the wrong trade-off. A spam filter should favour high precision and high specificity, accepting that some spam slips through.

When Accuracy Misleads

Accuracy counts all correct decisions equally. When the two classes are roughly balanced (as in the spam example with 400 vs. 600), accuracy gives a fair summary. When one class dominates, accuracy becomes misleading.

		Actual Condition (as observed)		
		Positive ($P = 30$)	Negative ($N = 2000$)	
Predicted Condition (as computed)	True (200)	True Positive ($TP = 20$)	False Positive ($FP = 180$)	$PPV = \frac{20}{200} = 10\%$
	False (1830)	False Negative ($FN = 10$)	True Negative ($TN = 1820$)	$NPV = \frac{1820}{1830} = 99\%$
		$TPR = \frac{20}{30} = 67\%$	$TNR = \frac{1820}{2000} = 91\%$	$ACC = \frac{1840}{2030} = 91\%$

Figure 2.6: Confusion matrix for an imbalanced dataset ($P = 30$, $N = 2000$). High accuracy (91%) masks low precision (10%) and moderate recall (67%).

Consider a screening test for a rare disease. In a population of 2030 people, 30 have the disease ($P = 30$) and 2000 do not ($N = 2000$). Figure 2.6 shows the test outcomes: $TP = 20$, $FP = 180$, $FN = 10$, $TN = 1820$.

Example: The accuracy paradox

$$\text{Accuracy} = \frac{20 + 1820}{2030} = \frac{1840}{2030} = 90.6\% \quad (2.40)$$

$$\text{Precision} = \frac{20}{200} = 10\% \quad \text{Recall} = \frac{20}{30} = 66.7\% \quad (2.41)$$

$$\text{Specificity} = \frac{1820}{2000} = 91\% \quad \text{NPV} = \frac{1820}{1830} = 99.5\% \quad (2.42)$$

The test is 91% accurate, yet only 10% of positive results are correct. Of every 10 people told they might be sick, 9 are healthy. A trivial baseline that always predicts “healthy” achieves $2000/2030 = 98.5\%$ accuracy, which is better than the actual test.

The problem is prevalence: only $30/2030 = 1.5\%$ of the population is positive. With so few positives, even a small false-positive rate ($FPR = 180/2000 = 9\%$) produces many more false positives than true positives in absolute terms. Accuracy is dominated by the 2000 true negatives and hides the fact that the test is nearly useless for confirming the disease.

When to distrust accuracy

If prevalence is below 10% or above 90%, accuracy alone is uninformative. Report precision, recall, and specificity separately. The F_1 -score (or F_β with a chosen β) combines precision and recall without being inflated by true negatives, which makes it a better single-number summary for imbalanced problems.

Asymmetric Error Costs

The spam and screening examples already show that FP and FN carry different costs. A third scenario makes this even starker: biometric face unlock on a smartphone.

The system decides whether the face in front of the camera matches the phone's owner. Two errors are possible:

- **False Positive:** an impostor's face is accepted. The phone unlocks for a stranger. This is a security breach.
- **False Negative:** the owner's face is rejected. The owner must retry or use a PIN. This is a minor inconvenience.

The costs are radically different: a single FP compromises all data on the device, while a FN costs five seconds. The system must therefore achieve an extremely low false-positive rate (FPR), even if this means a noticeable false-negative rate (FNR).

Example: Biometric face unlock

A face-unlock system is tested on 10,000 unlock attempts: 9,000 by the legitimate owner and 1,000 by impostors.

	Owner (9000)	Impostor (1000)
Unlocked	TP = 8820	FP = 2
Rejected	FN = 180	TN = 998

$$\text{FPR} = \frac{2}{1000} = 0.2\% \quad \text{FNR} = \frac{180}{9000} = 2.0\% \quad (2.43)$$

$$\text{Precision} = \frac{8820}{8820 + 2} = 99.98\% \quad \text{Recall} = \frac{8820}{8820 + 180} = 98.0\% \quad (2.44)$$

The system accepts a 2% owner-rejection rate to keep impostor acceptance at 0.2%. This trade-off is deliberate: the security requirement demands $\text{FPR} < 0.5\%$ regardless of the cost to FNR.

The choice of which error to minimize is not a mathematical question; it is a design decision that precedes evaluation. The metrics merely reveal whether the system meets the requirement. In the next section, we explore how moving a decision threshold trades FPR against FNR continuously, and how ROC curves visualize this trade-off.

Multiclass Classification

Binary classification assigns one of two labels. Many tasks assign one of K labels: an image classifier distinguishes "indoor", "outdoor", and "portrait"; an AI assistant routes user queries to the correct handler. The confusion matrix generalizes to a $K \times K$ table where rows are predicted classes and columns are actual classes.

Example: Intent routing

An AI assistant classifies each user query into one of four intents: `knowledge_base` (answer from stored documents), `web_search` (look up current information), `calculator` (compute a numerical answer), or `clarify` (ask the user for more detail). Evaluated on 200 queries, the resulting matrix in Figure 2.7 makes the uneven per-class performance visible:

		Actual Class			
		KB (80)	Web (60)	Calculator (40)	Clarify (20)
Predicted Class	Queries (200)				
	KB (92)	70	8	2	12
	Web (60)	4	48	6	2
	Calculator (36)	2	3	30	1
	Clarify (12)	4	1	2	5

Figure 2.7: Confusion matrix for an intent-routing classifier on 200 queries. Diagonal cells (green) are correct; off-diagonal cells (orange/red) are misrouted queries.

Overall accuracy: $(70 + 48 + 30 + 5)/200 = 76.5\%$. The system routes three out of four queries correctly, but the per-class story is far less uniform.

Per-Class Metrics via One-vs-Rest

To compute precision, recall, and specificity for one class, collapse the $K \times K$ matrix into a binary view: “belongs to class C ” vs. “does not belong to class C ”. Figure 2.8 shows this collapse for two classes from the intent-routing example.

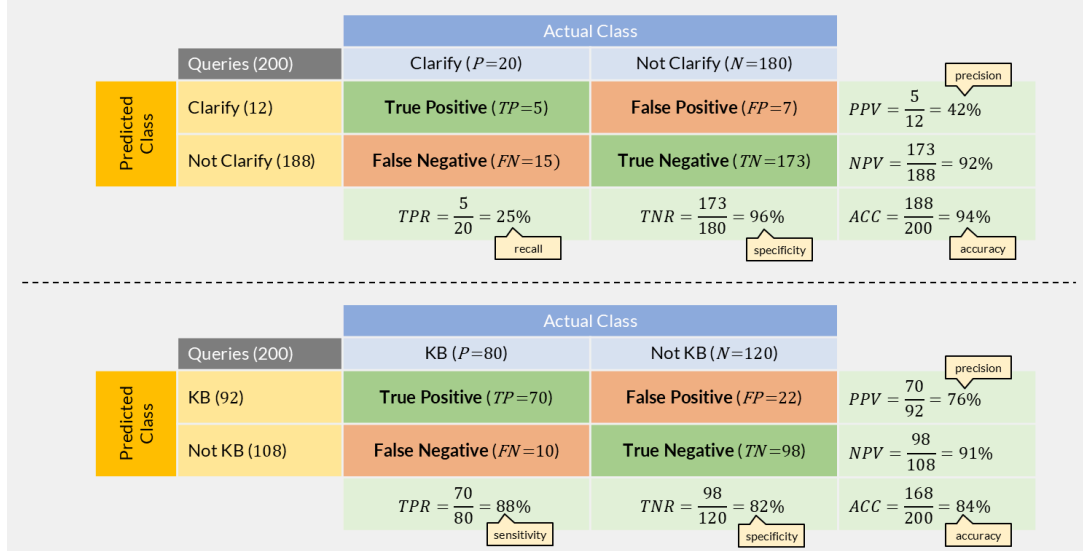


Figure 2.8: One-vs-rest collapse of the intent-routing confusion matrix for the `clarify` class (top) and the `knowledge_base` class (bottom), with derived metrics.

For the `clarify` class (the rare intent with 20 actual queries), the system correctly identifies only one in four queries that actually need clarification. Most of the rest (12 out of 15 missed) get misrouted to `knowledge_base`, where the system attempts to answer a question it does not understand. This is the most dangerous failure mode: the user gets a confident but wrong answer instead of a helpful follow-up question.

For the dominant `knowledge_base` class (80 of 200 queries), the system routes most KB queries correctly (high recall) but also absorbs queries that belong elsewhere (22 false positives), diluting precision. The high recall on the dominant class is what inflates the accuracy of the 4x4 confusion matrix to 76.5%.

Aggregating Across Classes

With K classes, we have K precision values and K recall values. Summarizing them into one number uses the same macro/micro distinction from Precision and Recall:

Macro-averaging computes the metric per class, then takes the arithmetic mean:

$$\text{Precision}_{\text{macro}} = \frac{1}{K} \sum_{k=1}^K \text{Precision}_k \quad (2.45)$$

Each class contributes equally, regardless of size. For our intent router:

Class	Precision	Recall
knowledge_base	0.761	0.875
web_search	0.800	0.800
calculator	0.833	0.750
clarify	0.417	0.250

$$\text{Precision}_{\text{macro}} = \frac{0.761 + 0.800 + 0.833 + 0.417}{4} = 0.703 \quad (2.46)$$

$$\text{Recall}_{\text{macro}} = \frac{0.875 + 0.800 + 0.750 + 0.250}{4} = 0.669 \quad (2.47)$$

Micro-averaging pools TP, FP, and FN across all classes before computing the ratio:

$$\text{Precision}_{\text{micro}} = \frac{\sum_k TP_k}{\sum_k (TP_k + FP_k)} = \frac{70 + 48 + 30 + 5}{92 + 60 + 36 + 12} = \frac{153}{200} = 0.765 \quad (2.48)$$

$$\text{Recall}_{\text{micro}} = \frac{\sum_k TP_k}{\sum_k (TP_k + FN_k)} = \frac{70 + 48 + 30 + 5}{80 + 60 + 40 + 20} = \frac{153}{200} = 0.765 \quad (2.49)$$

Both denominators equal the total number of queries (200): the precision denominator sums all row totals, the recall denominator sums all column totals, and in a single-label setting these are the same. The numerator is always the sum of the diagonal (correct predictions). So micro-precision, micro-recall, and accuracy all collapse to the same value: diagonal sum divided by total.

This explains why accuracy is the dominant single-number metric in classification: it is the micro-averaged precision and recall. In retrieval, micro-averaging played a secondary role because the macro view (one score per query, then average) better matched evaluation practice. In classification, the micro view is the natural default, and accuracy is its name.

Macro-averaging tells the opposite story. It gives each class equal weight: mean precision drops to 70.3% and mean recall to 66.9%, pulled down by *clarify*'s dismal 25% recall. If every intent matters equally (and it should, because a misrouted *clarify* query produces a wrong answer), macro-averaging is the right summary.

Accuracy hides per-class failures

Accuracy (= micro-average) is the standard headline metric for classification, and it naturally weights each class by its frequency: being good at the most common task counts more. This is often what we want, since the common case dominates user experience. However, when class sizes differ substantially, accuracy can mask catastrophic failures on rare but important classes. In our example, a 25% recall on *clarify* is invisible in the 76.5% accuracy number. Supplement accuracy with macro-averaged metrics or per-class breakdowns whenever a rare class carries disproportionate cost (a misrouted *clarify* query produces a wrong answer; a missed fraud case costs millions).

Hands-on: Classification Evaluation

Build confusion matrices for spam filtering and multiclass scenarios, compute per-class metrics, and observe how prevalence affects accuracy. [Open notebook ->](#)

Includes pre-run results: you can read through or download and experiment.

All metrics in this section assume a fixed decision: the system outputs a label and we count outcomes. But most classifiers internally produce a continuous score (a probability, a distance, a similarity), and the label emerges only after applying a threshold. Moving that threshold changes the balance between FP and FN. The next section explores this continuous perspective through score distributions, threshold selection, and ROC curves.

Scores, Thresholds, and ROC Curves

The previous section evaluated classifiers after the decision was already made: the system assigned a label, and we counted outcomes. But most classifiers do not jump directly to a label. They produce a continuous score, a confidence value between 0 and 1, and the label emerges only after comparing that score to a threshold. The spam filter used later in this section assigns 0.90 to obvious junk, 0.10 to the legitimate email “Lunch plans?”, and 0.40 to the ambiguous spam message “Claim your reward points”. At threshold 0.5, the latter message stays in the inbox; at threshold 0.4, it moves to junk.

Different thresholds produce different confusion matrices, different precision/recall values, and different TPR/FPR trade-offs. A single classifier is not one point in metric space; it is a family of points, one for each threshold. This section explores how to visualize that family and choose a single operating point.

Score Distributions and the Threshold

A binary classifier assigns a score $s(x) \in [0, 1]$ to each item x . Items from the positive class tend to receive higher scores; items from the negative class tend to receive lower scores. If the two score distributions were perfectly separated (all positives above some value, all negatives below), any threshold between them would classify perfectly. In practice, the distributions overlap. Figure 2.9 visualizes how that overlap creates unavoidable false positives and false negatives.

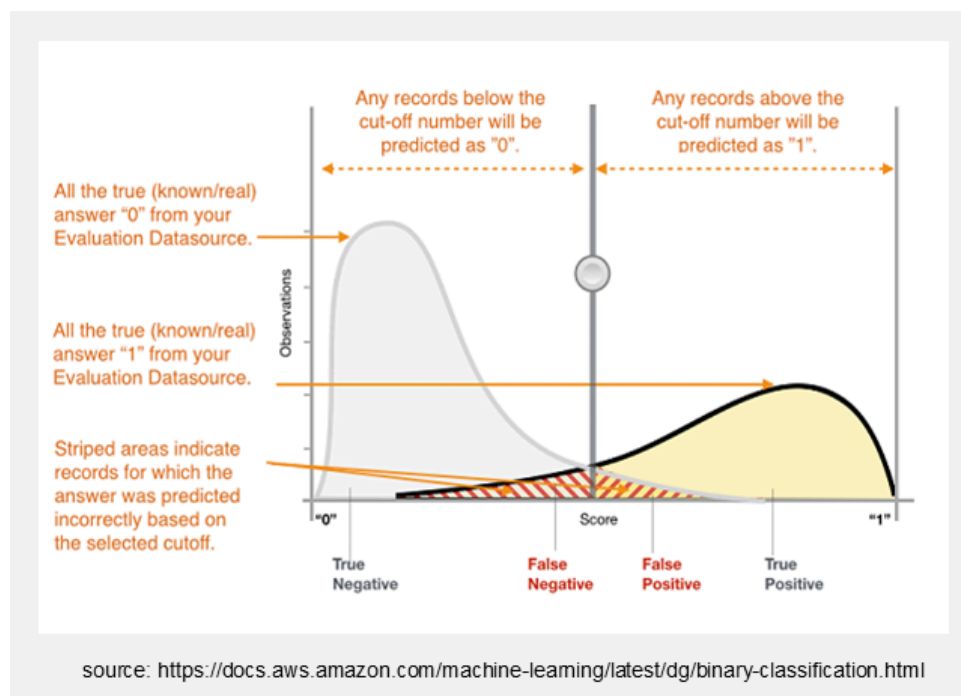


Figure 2.9: Score distributions for the negative class (grey) and positive class (yellow) with a decision threshold. The overlap region produces classification errors regardless of where the threshold is placed.

The threshold T partitions the score axis into two regions: items with $s(x) \geq T$ are predicted positive, items with $s(x) < T$ are predicted negative. Moving the threshold changes the balance between true positives, false positives, true negatives, and false negatives:

- **Shifting T to the left** (lower threshold): more items cross into the “predicted positive” region. TPR increases (fewer false negatives), but FPR also increases (more false positives). The system becomes more sensitive but less specific.
- **Shifting T to the right** (higher threshold): fewer items are predicted positive. FPR decreases (fewer false positives), but TPR also decreases (more false negatives). The system becomes more specific but less sensitive.

This is the fundamental trade-off underlying every threshold-based classifier. No single threshold is universally best; the right choice depends on the error costs of the application.

Formalizing the Threshold as Areas Under the Distributions

Let $f_n(x)$ denote the score distribution of the negative class and $f_p(x)$ the score distribution of the positive class. Each classification rate corresponds to an area under one of these curves, partitioned by the threshold T . Figure 2.10 shows the four resulting areas:

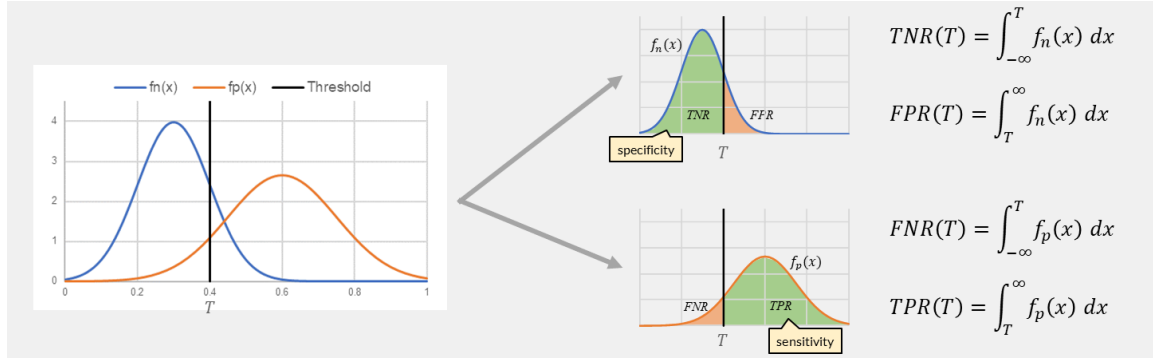


Figure 2.10: Decomposition of the score distributions into the four classification rates. TNR and FPR partition the negative-class curve $f_n(x)$; FNR and TPR partition the positive-class curve $f_p(x)$.

Key Formula: Classification Rates as Integrals

$$TPR(T) = \int_T^{\infty} f_p(x) dx \quad FNR(T) = \int_{-\infty}^T f_p(x) dx \quad (2.50)$$

$$TNR(T) = \int_{-\infty}^T f_n(x) dx \quad FPR(T) = \int_T^{\infty} f_n(x) dx \quad (2.51)$$

Each pair partitions one distribution: $TPR + FNR = 1$ and $TNR + FPR = 1$. Moving T shifts the boundary between the two areas in each pair.

The integrals make the trade-off precise. Shifting T to the left expands the integration range for TPR (the area under f_p to the right of T grows), but simultaneously expands the integration range for FPR (the area under f_n to the right of T also grows). No threshold eliminates the overlap region; it can only choose how to distribute the overlap errors between false positives and false negatives.

The ROC Curve

Instead of evaluating one threshold, the ROC (Receiver Operating Characteristic) curve evaluates all of them simultaneously. It plots $TPR(T)$ on the vertical axis against $FPR(T)$ on the horizontal axis, sweeping T from high to low. The result is a curve from $(0, 0)$ to $(1, 1)$ that shows the full trade-off landscape.

Figure 2.11 shows 20 emails (10 spam, 10 legitimate) sorted by descending spam score. At each score, we set the threshold there and count cumulative TP, FP, FN, TN. The right panel plots the resulting ROC curve.

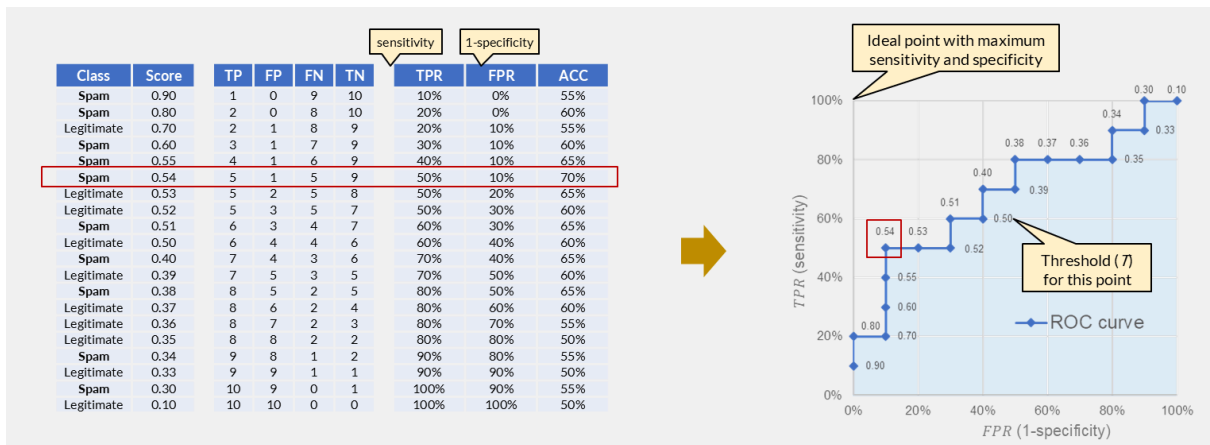


Figure 2.11: ROC curve construction from a ranked list of 20 emails. Each row sets the threshold at that score; the curve traces TPR vs. FPR as the threshold sweeps from high to low. The highlighted point ($T = 0.54$) has the highest accuracy but is not necessarily the best operating point.

Example: ROC curve construction for a spam filter

The highlighted row at score 0.54 gives $TPR = 50\%$, $FPR = 10\%$, accuracy = 70%. This is the highest-accuracy threshold. But is it the best choice for a spam filter?

- At threshold 0.54: half the spam is caught, and only 10% of legitimate emails are falsely flagged.
- At threshold 0.38: $TPR = 80\%$, $FPR = 50\%$. Most spam is caught, but half the legitimate mail goes to junk.
- At threshold 0.80: $TPR = 20\%$, $FPR = 0\%$. No legitimate email is lost, but 80% of spam reaches the inbox.

For a spam filter, a false positive (legitimate email deleted) is far more costly than a false negative (spam reaching the inbox). The right operating point is not the one that maximizes accuracy but the one that keeps FPR very low while maintaining acceptable TPR. Figure 2.11 shows that no threshold achieves both goals well: the classifier's score separation is simply too weak. This is the point where evaluation tells us to go back and improve the model itself rather than searching for a better threshold on a mediocre curve.

Key Landmarks

Point	Meaning
(0, 0)	Threshold so high that nothing is predicted positive (always predict negative)
(1, 1)	Threshold so low that everything is predicted positive (always predict positive)
(0, 1)	Perfect classifier: all positives detected, no false positives
Diagonal	Random classifier: $TPR = FPR$ at every threshold (no discrimination)

A curve that hugs the upper-left corner represents a strong classifier: it achieves high TPR before FPR rises appreciably. A curve that follows the diagonal adds no information beyond random guessing.

Reading the ROC Space

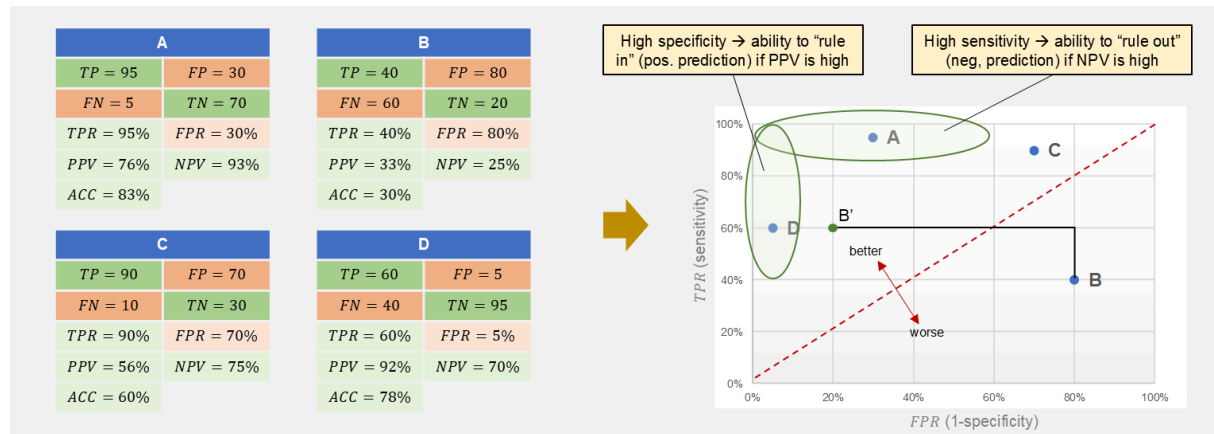


Figure 2.12: Four classifiers (A-D) mapped from their confusion matrices to ROC space. The upper-left region represents high sensitivity with low false-positive rate; the diagonal represents random performance.

Figure 2.12 places four classifiers at different operating points:

- **Classifier A** (TPR = 95%, FPR = 30%): high sensitivity, moderate false-positive rate. If a negative prediction from A is almost certainly correct (high NPV), A can “rule out” the condition: a negative result is trustworthy.
- **Classifier B** (TPR = 40%, FPR = 80%): below the diagonal. This classifier is worse than random. Flipping its predictions gives B' (TPR = 60%, FPR = 20%), which is above the diagonal and usable.
- **Classifier C** (TPR = 90%, FPR = 70%): high sensitivity but very high false-positive rate. It catches most positives but at enormous cost in false alarms.
- **Classifier D** (TPR = 60%, FPR = 5%): high specificity, moderate sensitivity. If a positive prediction from D is almost certainly correct (high PPV), D can “rule in” the condition: a positive result is trustworthy.

The biometric face-unlock from [Evaluating Text Classifiers](#) would appear as a point near the left edge of ROC space: $FPR = 0.2\%$, $TPR = 98\%$. The requirement “FPR below 0.5%” constrains the operating point to a narrow vertical strip on the left side of the plot.

AUC: Area Under the ROC Curve

The ROC curve shows performance across all thresholds. Summarizing it into a single number gives the Area Under the Curve (AUC).

Key Formula: AUC

$$AUC = \int_0^1 TPR(FPR) d(FPR) \quad (2.52)$$

Geometrically: the area under the ROC curve. Probabilistically: the probability that a randomly chosen positive item receives a higher score than a randomly chosen negative item. The equivalence holds because constructing the ROC curve is the same as asking, for each negative item (which produces one ΔFPR step to the right), what fraction of positives score higher (the current TPR height). Summing height times width over all negative items gives both the area and the Wilcoxon-Mann-Whitney U-statistic, which is exactly $P(s(x_+) > s(x_-))$ (Hanley and McNeil, 1982).

AUC	Interpretation
1.0	Perfect separation: every positive scores higher than every negative
0.5	Random: scores carry no discriminative information
< 0.5	Worse than random (flip predictions to improve)

Example: AUC interpretation

The AUC can be computed from the step-function ROC curve by summing rectangles: each time a negative item is encountered (FPR steps right by $1/N$), the rectangle's height is the current TPR. For the 20-email spam filter in [Figure 2.11](#), the ROC curve hugs the left edge for the first two spam emails (TPR rises to 20% with FPR still at 0%), then begins stepping right. The resulting AUC is approximately 0.68.

This means: if we pick one random spam email and one random legitimate email, there is a 68% chance the spam email receives the higher score. The imperfect separation arises because several spam and legitimate emails have scores in the 0.30-0.55 range where the distributions overlap heavily.

AUC is useful for comparing classifiers without committing to a threshold: a system with higher AUC has better score separation overall. However, AUC does not say which operating point the system will use in production, and two classifiers with the same AUC can have very different curves (one may be better at low FPR, the other at high TPR).

AUC summarizes but does not prescribe

A classifier with $AUC = 0.95$ is not necessarily better than one with $AUC = 0.90$ for a specific application. If the requirement is $FPR < 0.1\%$ (as in biometric unlock), the only relevant part of the curve is the far-left edge. A system with $AUC = 0.90$ that achieves $TPR = 97\%$ at $FPR = 0.1\%$ is better for that use case than a system with $AUC = 0.95$ that only reaches $TPR = 85\%$ at the same FPR constraint. Always examine the curve at the operating region that matters.

Choosing an Operating Point

The ROC curve shows what is possible. Selecting one point on it requires a decision criterion from outside the mathematics:

Maximize accuracy. Compute $(TP + TN)/(P + N)$ at each threshold and pick the maximum. This works when classes are balanced and error costs are symmetric. In the spam example from [Figure 2.11](#), threshold 0.54 gives $TP = 5$, $TN = 9$, $accuracy = 14/20 = 70\%$, the highest value in the table. But as we discussed, the spam filter should rather optimize for a low FPR while maintaining acceptable TPR, not simply maximize accuracy.

Constrained optimization. Fix one rate and optimize the other. The face-unlock requirement " $FPR \leq 0.2\%$ " defines a vertical line on the ROC plot; the operating point is the highest TPR value at or to the left of that line. This is the standard approach when one error type has a hard cost limit.

Youden's J-statistic. Maximize $J = TPR - FPR$, which is the vertical distance from the diagonal (random classifier). The point with the highest J is geometrically closest to the upper-left corner and balances sensitivity against specificity without committing to a cost model.

Equal error rate (EER). Find the threshold where $FPR = FNR$ (equivalently, $FPR = 1 - TPR$). This is the point where the ROC curve crosses the anti-diagonal from $(0, 1)$ to $(1, 0)$. EER is commonly reported in biometric and speaker-verification systems as a single-number summary that does not require choosing which error is worse.

In all cases, threshold selection should be performed on a validation set separate from the test set used to report final metrics. Optimizing the threshold on the test set inflates reported performance.

Hands-on: Thresholds and ROC

Sweep the decision threshold across the spam-filter scores and watch the confusion matrix, TPR, FPR, and ROC curve update in real time. Compute AUC and find the threshold that maximizes accuracy vs. the one that satisfies a FPR constraint. [Open notebook ->](#)

Includes pre-run results: you can read through or download and experiment.

When Evaluation Misleads

Two results arrive on the same afternoon. A retrieval group reports that their new ranker raises nDCG@10 on a public benchmark from 0.41 to 0.43. A colleague reports that their spam filter reaches 99.2% accuracy on a held-out test set. Both numbers were computed correctly with the measures developed in the previous sections. Both might still be worthless.

Now that the measures are in hand, we can look at what they do not tell us. Every number in this chapter rests on a chain of decisions: which documents were collected, which needs were written down, who judged them, how the labels were split, and which measure was reported. A weakness anywhere in that chain reaches the final score, and the score itself never reveals it. Reading evaluation results critically means knowing where the chain tends to break.

Incomplete Ground Truth

Recall the pooling arrangement from [Designing a Retrieval Benchmark](#): at realistic scale, only the documents that participating systems actually retrieved get judged, and everything unjudged counts as not relevant. That convention is what makes large benchmarks affordable, and it is also the first place scores go wrong.

In the **dense case** (TREC-style pooling), the benchmark assessed all documents that participating systems submitted in a given year. The judgments are thorough for those systems. But when a new algorithm is evaluated against the same reused collection, it may retrieve relevant documents that no original participant found. Those documents were never assessed and count as non-relevant. The new system is penalized for being original: it found material the pool missed, and the benchmark cannot reward it. A careless evaluator might even conclude the new system is worse, when in fact it discovered answers nobody had seen before.

In the **sparse case** (MS MARCO-style benchmarks with millions of queries), each query has only one or a few assessed relevant documents. [Figure 2.13](#) shows this situation. Any document that equally qualifies as a correct answer but was never selected for assessment penalizes rather than rewards the system that retrieves it. The bias is systematic: retrieval systems trained on such benchmarks tend to replicate what was selected for assessment rather than what is relevant in a broader sense.

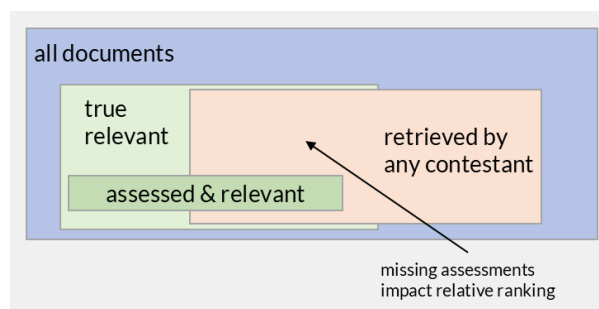


Figure 2.13: Sparse assessment: only a fraction of genuinely relevant documents were ever judged. Systems retrieving unjudged relevant material are penalized.

Recall suffers most, and in a direction that depends on who ran the experiment. Precision at rank 10 needs only the top ten documents judged, which pooling usually delivers. Recall needs the total amount of relevant material in the collection, and that is exactly what incomplete judging never establishes. For the systems that built the pool, recall comes out too high, because the divisor counts only the relevant documents somebody found. For a system evaluated later against the reused collection, the error can run the other way, since its own relevant results may be unjudged and scored as misses.

Unjudged is not the same as not relevant

Every metric in this chapter treats an unjudged document as non-relevant, because the arithmetic needs a label and there is none. That is a convention forced by cost, not a finding about the document. When comparing two systems on a sparsely judged benchmark, a lower score can mean “retrieved different documents” rather than “retrieved worse documents”.

Noisy Ground Truth

Even judged labels are opinions. Assessors work from written guidelines and still disagree on borderline cases, so a benchmark encodes one panel's reading of each need. This is tolerable for comparison, since every system is scored against the same labels, but it puts a floor under how small a difference we can meaningfully interpret. A gap of one point in a measure whose ground truth two qualified assessors would not fully reproduce is not a result.

Classification test sets have the same problem in a different shape. Their labels are usually complete, so pooling bias does not apply, but completeness is not correctness. Whoever labelled the messages had to decide whether an aggressive marketing newsletter counts as spam, and different labellers draw that line differently. A classifier that learned one labeller's line will look better than one that learned another's, independently of which is more useful.

Rare classes make this worse because small counts are unstable. In an intent router with four routes, a test set may contain hundreds of knowledge-base queries and a dozen calculator queries. Per-class recall for the calculator route then moves by eight percentage points when a single example changes label, and a macro-averaged score built from such classes inherits that noise. Before trusting a per-class number, check how many test items it was computed from.

Optimizing Against the Test

A benchmark that stays fixed for years stops being a test and becomes a target. Recall how a TREC-style evaluation runs: participants receive the collection and the topics, run their systems on their own machines, and submit ranked lists that the organizers judge afterwards. Holding the topics is what makes the risk possible. A research group can read the topics, notice that several hinge on ambiguous words, add a hand-built list of expansion terms, adjust the stop word list, add exceptions to the stemmer, and keep every change that raises the score. Each step on its own looks like sound engineering. The result is a system that has absorbed the properties of this topic set and this collection: which words happen to be ambiguous here, and which documents happen to be relevant. The score rises and the method does not transfer. No dishonesty is required, because the selection pressure alone is enough.

Issuing a new topic set for each cycle limits the damage while a benchmark is live, and TREC does this. The cost is comparability, since results measured on this year's topics cannot be placed directly against last year's. The concentrated risk appears after a cycle closes, when the collection, its topics, and its judgments become a reusable test collection that papers measure against for years. Armstrong, Moffat, Webber, and Zobel examined a decade of published ad-hoc retrieval results obtained that way in **Improvements That Don't Add Up**. Dozens of papers reported gains and many claimed statistical significance, yet the improvements did not accumulate into visible progress, and the baselines were often weaker than the median system from the original TREC cycle. Reported improvement and real improvement had come apart.

Classification has a sharper and more mechanical version of the same failure. A decision threshold tuned on the test set produces a score that cannot be trusted, because the threshold was chosen with knowledge of the answers. This is why the validation split exists, and why the threshold selection in **Scores, Thresholds, and ROC Curves** must happen on validation data. The extreme case is contamination: if the test items and their labels were part of a model's training data, the model can score well by recalling an answer rather than by solving the task. Large language models make this problem acute, because they are trained on web-scale corpora that include publicly available exam questions. When GPT-4 is evaluated on the SAT or the MMLU benchmark, the questions and their correct answers were likely present somewhere in the training data. Researchers have shown that GPT-4 can guess missing answer options in MMLU questions with a 57% exact-match rate, far above chance, which strongly suggests memorization rather than reasoning (**Investigating Data Contamination in Modern Benchmarks for Large Language Models**, 2023). A high score on a contaminated benchmark tells us the model saw the answers before, not that it can generalize to unseen problems.

Withholding the labels is a partial defence. MS MARCO keeps the labels of its evaluation set private and scores submissions centrally, so participants cannot tune against them directly. The leaderboard's own designers describe the limit in **Fostering Competition While Plugging Leaks**: every submission returns

a score, and every score leaks a little information about the hidden set. Enough attempts turn a blind test into a slowly revealed one, which is why the official metric carries a deliberate artifact meant to frustrate reverse-engineering.

A genuinely blind evaluation reverses the direction in which data travels. Instead of sending the topics to the participant, the participant sends the system to the evaluator: code or a container executed against topics it has never seen, with judgments made only after all results are collected. This is known as evaluation as a service, or software submission. **TIRA** implements it by keeping the test data and ground truth out of public reach and running each submitted system in a sandbox that prevents data leaks. The protocol costs the organizers considerably more effort, which is why it remains the exception. One caveat survives even then: a blind topic set shows that a system never saw these needs, but for a model pretrained on web text it cannot show that the documents themselves were absent from training.

A benchmark score is not a claim about generalization

A score describes how a system performs on one collection, one topic set, and one panel's judgments, usually under a protocol that let the system's authors read the topics first. Whether the method transfers elsewhere is a separate claim needing separate evidence: a different collection, a topic set the authors never inspected, or a blind evaluation.

The Test Set Is Not Production

Benchmarks are frozen so that results stay reproducible, and freezing is what makes them drift away from the world they were drawn from. Vocabulary shifts, topics appear that the collection predates, document formats change. The stability that makes a benchmark useful for comparison steadily reduces how much it says about current traffic.

For classification the mismatch is measurable, and prevalence is the clearest case. Suppose a spam filter detects 90% of spam and correctly passes 98% of legitimate mail. On a balanced test set of 500 spam and 500 legitimate messages, it produces 450 true positives and 10 false positives, so precision is $450/460 \approx 97.8\%$. Deploy the same filter, unchanged, on a mailbox where only 5% of messages are spam. Out of 500 spam and 9,500 legitimate messages it still catches 450 and still misclassifies 2% of legitimate mail, but 2% of 9,500 is 190 false positives, so precision falls to $450/640 \approx 70.3\%$.

Nothing about the classifier changed. Recall and specificity are identical in both settings. Only the class balance moved, and precision, the measure the user actually experiences, dropped by 27 percentage points. A test set whose prevalence does not match deployment therefore reports a precision that deployment will not reproduce.

One Number Is Not a Result

A single score on a single collection is weak evidence, for a reason that has nothing to do with retrieval. Per-need scores vary enormously: the same system can reach average precision above 0.8 on one need and near zero on another. An average over 50 needs conceals that spread, so two systems differing by a point in the mean may be indistinguishable once the variation between needs is taken into account. Reporting the spread, and testing whether an observed difference exceeds it, matters as much as the measure itself.

The problem compounds across a community. When hundreds of groups tune against one public benchmark, the leading entry is partly the winner of a large search over method variants, and some of its margin is the luck of that search rather than a property of the method. This is the same multiple-comparisons effect that makes a single significance test misleading when many hypotheses were tried, and it is one mechanism behind the missing accumulation that Armstrong and colleagues documented.

Effectiveness Is Not the Only Axis

Finally, relevance is one dimension of quality among several. A production system must also answer quickly, sustain many concurrent requests, and stay affordable, and these goals pull against each other. Approximate nearest-neighbour methods make this trade-off explicit: they deliberately accept slightly

worse results in exchange for large gains in speed and cost, and that exchange is often the right decision. A benchmark reporting only effectiveness cannot express it, which is why operational targets such as latency, throughput, and cost per query belong in the performance goals next to the retrieval measures.

None of this makes evaluation futile. It makes evaluation a claim with a scope. The following questions establish that scope for any reported score:

- How complete are the judgments, and could a better system have been penalized for retrieving unjudged documents?
- Who produced the labels, and how much would a second panel agree?
- Did the authors see the test data before reporting, and was any threshold or hyperparameter chosen using it?
- How closely do the collection and the class balance match the setting where the system will run?
- How large is the difference relative to the variation across needs, and how many alternatives were tried before this one was reported?
- What does the score cost in latency and compute?

A result that survives these questions is worth acting on.

Summary

Metric Lookup Table

Metric	Formula	What it rewards	When to use
Precision	$\frac{TP}{TP+FP}$	Clean results (few false positives)	Any binary decision; especially when FP is costly
Recall	$\frac{TP}{TP+FN}$	Completeness (few misses)	Exhaustive search; patent, legal, medical
F_β	$\frac{(\beta^2+1) \cdot p \cdot r}{\beta^2 \cdot p + r}$	Balances precision and recall with tuneable weight	Single-number summary; $\beta = 1$ default
$P@k$	Precision in top k	Clean top-of-list	Web search, short result pages
MRR	Mean of $1/\text{rank}_{\text{first relevant}}$	First relevant result high	QA, known-item search
R-Precision	$p@R$	Balance at a natural cutoff	Self-adjusting single-point summary
AP	Mean of p_k at relevant positions, divided by $\$$	Rel	$\$$
MAP	Mean of per-query AP	System-level ranking quality	Benchmark comparison (TREC standard)
$nDCG@k$	$DCG_k / IDCG_k$	High-grade documents placed early	Web search, graded relevance, user experience
Accuracy	$\frac{TP+TN}{P+N}$	Overall correct decisions	Balanced classification; equals micro-precision and micro-recall
Specificity (TNR)	$\frac{TN}{TN+FP}$	Few false alarms	Medical tests, biometric security
AUC	Area under ROC curve	Score separation across all thresholds	Comparing classifiers without committing to a threshold

Key Takeaways

1. Evaluation requires a benchmark: a collection, information needs, relevance judgments, and a performance goal. Without any component, a number is not interpretable.
2. Precision and recall are in tension. Improving one typically costs the other, and the F_β -measure makes the trade-off explicit.
3. Ranked evaluation adds position: AP and nDCG reward systems that place relevant (or high-grade) material where users will see it.
4. Graded relevance distinguishes “somewhat useful” from “essential”. nDCG captures what binary metrics cannot: the quality of the relevant documents, not only their presence.
5. Classification evaluation reuses precision and recall but adds accuracy, specificity, and the confusion matrix. Accuracy equals micro-precision equals micro-recall in single-label settings.
6. Prevalence distorts accuracy: when one class dominates, a trivial baseline can score higher than a real classifier. Always check class balance before trusting accuracy.
7. A continuous score becomes a binary decision only after choosing a threshold. The ROC curve shows all possible trade-offs; AUC summarizes overall score separation.
8. No benchmark score is a claim about generalization. Incomplete judgments, data contamination, prevalence mismatch, and overfitting to static test sets all limit what a number can mean.

Key Formulas

$$p = \frac{TP}{TP + FP} \quad r = \frac{TP}{TP + FN} \quad (2.53)$$

Precision and recall: the fundamental pair from which all other metrics derive.

$$AP = \frac{1}{|\text{Rel}|} \sum_{k: d_k \in \text{Rel}} p_k \quad (2.54)$$

Average precision: area under the precision-recall curve, the standard per-query measure.

$$nDCG_k = \frac{\sum_{i=1}^k \frac{rel_i}{\log_2(i+1)}}{IDCG_k} \quad (2.55)$$

Normalized discounted cumulative gain: the standard for graded relevance evaluation.

$$AUC = P(s(x_+) > s(x_-)) \quad (2.56)$$

Area under the ROC curve: the probability that a random positive scores higher than a random negative.

Exam focus

- The difference between precision and recall, and why both are always reported together
- How AP integrates both precision and recall in one number, and why missed documents hurt AP
- Why nDCG and AP can disagree on which system is better (top-k user experience vs. full-list completeness)
- Why accuracy misleads under class imbalance, and how prevalence affects precision
- How moving a decision threshold trades TPR against FPR, and what the ROC curve visualizes
- The difference between macro-averaging (each query/class counts equally) and micro-averaging (each item counts equally)

Self-Check Questions

1. (Understand) A system achieves 95% accuracy on a dataset where 95% of items belong to class A. Is this system useful? What additional metric would reveal the problem?
2. (Analyze) System X has MAP = 0.60 and System Y has MAP = 0.55, both measured on 50 queries. Under what circumstances might Y actually be the better system?
3. (Evaluate) A face-unlock system reports AUC = 0.99 and a spam filter reports AUC = 0.99. Are they equally good? What additional information do you need to judge each for its intended use?
4. (Apply) Given a ranked list of 10 documents where relevant documents appear at positions 1, 4, and 7 (out of 8 relevant in the collection), compute $P@5$, AP, and explain why AP is less than the average of the three precision values.
5. (Analyze) Explain why macro-averaged recall across 4 information needs can disagree with micro-averaged recall, and give a concrete scenario where the disagreement is large.

Test Your Knowledge

[Take the Chapter 2 Quiz](#) 

Further Reading

- Manning, C. D., Raghavan, P., & Schütze, H. (2008). **Introduction to information retrieval**, Chapter 8: Evaluation in information retrieval. Cambridge University Press. [Read online](#). A concise foundation for precision, recall, MAP, and benchmark design.
- Järvelin, K., & Kekäläinen, J. (2002). **Cumulated gain-based evaluation of IR techniques**. *ACM Transactions on Information Systems*, 20(4), 422–446. [ACM record](#). The paper that introduced DCG and nDCG for graded relevance evaluation.
- Fawcett, T. (2006). **An introduction to ROC analysis**. *Pattern Recognition Letters*, 27(8), 861–874. [Read the paper](#). A practical guide to ROC curves, AUC, and threshold selection for binary classifiers.

- Armstrong, T. G., Moffat, A., Webber, W., & Zobel, J. (2009). **Improvements that don't add up: Ad-hoc retrieval results since 1998**. In *Proceedings of CIKM 2009* (pp. 601–610). [ACM record](#). An influential warning that static benchmarks can conceal a lack of cumulative progress.
- [TREC \(Text REtrieval Conference\)](#). NIST's continuing evaluation campaigns provide the pooled collections, shared topics, and comparative methodology behind much of this chapter's retrieval evaluation practice.

3 - Advanced Text Processing

A user searches a bookshop's website for "car repair" and gets nothing useful. The catalog holds hundreds of matching titles, but their spines read "automobile repair", "auto repair", "Cars" (the Pixar film), or "car maintenance". The search index stores each of these as a separate token. "Car" and "automobile" never meet.

A different user asks "Who won the F1 race on the weekend?" and gets nothing useful either. The answer sits in a race-results page that reads "Verstappen took the win at Silverstone on Sunday, 6 July". That page shares almost no tokens with the query. "Who" does not appear in it; "weekend" does not appear either (the report says Sunday and gives a date); even "race" is absent because the article calls it a "Grand Prix". The user asked a question and expected a fact back. The retriever received a bag of five keywords and matched them against pages that also contain the word "weekend".

A third user types "Bücher von Goethe" and again gets unrelated results. Nothing in the pipeline noticed that "Bücher" is German for "books", that "Goethe" is a person's name, or that the sentence is a request for works by an author. All the structure the user provided was there in plain text and none of it was used.

The three failures come from the same source. Classical retrieval treats text as a bag of exact tokens: one surface form is one term and a query is a bag of query terms. That model works for keyword searches over clean text and breaks in two directions. On the document side, one concept has many surface forms and recall collapses. On the query side, natural-language input has structure, meaning, and intent that do not map to tokens in the answer at all, and the retriever ignores every layer of that information.

Advanced text processing addresses both. This chapter builds the classical toolbox that turns raw strings into features a retrieval and routing system can actually use: tokenization, normalization, sentence segmentation, stop words, stemming and lemmatization, phrase and compound detection, part-of-speech tagging, named entity recognition, and simple classifiers for language and intent. Together they close the recall gap on the document side and extract enough structure from queries that a retrieval system can route them, filter them, and rank the results.

The three failure examples do not all fully resolve in this chapter. The bookshop's "car repair" query and the German "Bücher von Goethe" query are handled end to end here. The F1 question is harder: the classical pipeline can turn it into a targeted news-search query with a date filter, but delivering "Verstappen won" as a two-word answer rather than a list of pages is the work of semantic search and retrieval-augmented generation in later chapters. The F1 case will follow us through the book.

What you'll learn

- Tokenize and normalize text for retrieval, including case folding, Unicode and accent handling, sentence segmentation, and language detection
- Compare stemming and lemmatization strategies and choose the right one for a given retrieval scenario
- Detect meaningful multi-word phrases and split compound words with pointwise mutual information, likelihood ratios, and log-frequency scoring
- Extract structure from free-text queries with part-of-speech tagging and named entity recognition
- Turn extracted features into decisions with a Naive Bayes classifier, applied to language detection and query intent routing

Prerequisites

- The retrieval architectures and text-processing basics introduced in [1 - Classical Text Retrieval](#)
- The evaluation framework from [2 - Performance Evaluation](#), used to compare classifiers in the last section
- Basic probability and Bayes' rule, at the level covered in the ML foundations appendix

Tokenization and Normalization

The chapter opener showed a bookshop returning nothing for “car repair” because “automobile repair” is stored under a different token. That failure begins at the very first stage of the retrieval pipeline: how raw text is split into tokens and normalized before any matching happens. This section walks through that stage, starting with the whitespace-and-punctuation splitting from [1 - Classical Text Retrieval](#) and ending with a rule-based language detector that decides which downstream pipeline to invoke.

The naive tokenizer and where it breaks

Chapter 1 split text at whitespace and punctuation boundaries. The simplest rendering of that approach is a one-line tokenizer:

```
def word_tokenize(text: str) -> list[str]:
    text = re.sub(r'[\W-]+', ' ', text)
    return [t for t in text.split(' ') if t]
```

It substitutes any non-word character with a space, then splits on whitespace. It works on well-behaved English prose. Give it a sentence that exercises real-world text and it fails in several distinct ways at once:

I buy my parents' 10% of U.K. startup for \$1.4 billion. Dr. Watson's cat called Mrs. Hersley and it was w.r.o.n.g., more to come ...

Running the naive tokenizer produces:

```
['I', 'buy', 'my', 'parents', '10', 'of', 'U', 'K', 'startup',
 'for', '1', '4', 'billion', 'Dr', 'Watson', 's', 'cat', 'called',
 'Mrs', 'Hersley', 'and', 'it', 'was', 'w', 'r', 'o', 'n', 'g',
 'more', 'to', 'come']
```

Four systematic problems appear:

- **Possessives are handled inconsistently by shape.** parents' (trailing apostrophe only) becomes just parents: the apostrophe and the following space collapse into one space and the apostrophe leaves no trace. Watson's (apostrophe followed by s) becomes two tokens, Watson and a stray s, because the apostrophe becomes a space that splits the two letters apart. For retrieval, folding “Watson's” into “Watson” is often desirable (users search for the name, not the possessive), so the stray s is dropped in a subsequent cleanup step and both possessive forms end up equivalent. For syntactic analysis this is harmful, because the link between “Watson” and “cat” is broken and the shape-dependent behaviour is easy to overlook.
- **Numbers, currencies, and percentages fall apart.** 10% becomes 10, \$1.4 billion becomes 1, 4, billion. Small integers survive; anything realistic does not.
- **Abbreviations disintegrate.** U.K., Dr., Mrs., and the artificial w.r.o.n.g. all break at their internal periods. Searching for U.K. is impossible against this index.
- **Punctuation vanishes.** Fine for retrieval, but downstream sentence analysis loses its cues.

For a retrieval-only pipeline over English, this behaviour is not fatal: users rarely search for “U.K.” with the period, and possessive stripping helps recall. But the distinctions are gone for good, and any task that needs sentence structure or numeric precision needs a better tokenizer up front.

Modern tokenization with nltk and spaCy

Both nltk and spaCy ship word tokenizers built from rules combined with abbreviation and exception lists learned from text corpora. Running each on the demonstration sentence:

```
import nltk
import spacy

sentence = ("I buy my parents' 10% of U.K. startup for $1.4 billion. ")
```

```

"Dr. Watson's cat called Mrs. Hersley and it was w.r.o.n.g., more to
come ...")

nltk.word_tokenize(sentence)
# ['I', 'buy', 'my', 'parents', "'", '10', '%', 'of', 'U.K.', 'startup', 'for',
# '$', '1.4', 'billion', '.', 'Dr.', 'Watson', "'s", 'cat', 'called', 'Mrs.',
# 'Hersley', 'and', 'it', 'was', 'w.r.o.n.g.', ',', 'more', 'to', 'come', '...']

[t.text for t in spacy.load('en_core_web_sm')(sentence)]
# ['I', 'buy', 'my', 'parents', "'", '10', '%', 'of', 'U.K.', 'startup', 'for',
# '$', '1.4', 'billion', '.', 'Dr.', 'Watson', "'s", 'cat', 'called', 'Mrs.',
# 'Hersley', 'and', 'it', 'was', 'w.r.o.n.g.', '.', ',', 'more', 'to', 'come', '...']

```

The two outputs agree on every token except one: nltk treats w.r.o.n.g. as a single abbreviation, spaCy splits the trailing period as sentence-terminating punctuation. On real text the two libraries usually agree; artificial or novel abbreviations are where they diverge.

Compared with the naive regex, the trained tokenizers fix several of the earlier failures but not all of them, and the two possessive shapes still behave differently:

- **Possessives.** parents' produces parents and a bare '. Watson's produces Watson and 's. Both tokenizers keep the apostrophe visible; only the 's form carries a marker that distinguishes it from any other apostrophe. Retrieval pipelines usually drop both apostrophe tokens after this step; syntactic pipelines keep them because they carry grammatical information.
- **Numbers.** Decimals stay together (1.4), which fixes the naive regex's worst behaviour. But the currency and percent symbols split off as their own tokens: \$1.4 becomes ['\$ ', '1.4'] and 10% becomes ['10 ', '%']. A downstream step that wants "\$1.4 billion" as one money entity needs to recombine, which is the job of the named entity recognizer covered later in the chapter.
- **Abbreviations.** U.K., Dr., and Mrs. are kept whole with their trailing periods, because both libraries carry curated abbreviation lists. w.r.o.n.g. is the divergent case: nltk keeps it whole, spaCy splits its final period. Real abbreviations rarely differ between the two libraries.
- **Punctuation.** ., ,, and ... are preserved as their own tokens. Retrieval pipelines filter them out; syntactic pipelines keep them.

For retrieval, we still want a leaner token list than the tokenizer alone provides. The standard cleanup after tokenization is:

1. Drop single-letter tokens and pure punctuation tokens.
2. Drop numbers and currency tokens unless the collection is numeric (a scientific-paper index would keep them).
3. Drop possessive tokens (" 's") once the possessive information has been used.

The chapter's demo notebook composes these three cleanup steps on top of a regex tokenizer and compares the intermediate outputs side by side.

Case, Unicode, and accents

A tokenizer separates words. Normalization decides which surface forms of the same word collapse to the same token. Consider three variants of one city name:

Zurich, Zürich, ZURICH

They refer to the same city. To match a query for Zurich against a document that spells the city Zürich, the index has to normalize both spellings to a shared form before storing them. Three transforms handle almost all cases in European text:

- **Case folding** applies `text.lower()`, or the locale-aware equivalent for Turkish İ/ı. The three variants collapse to `zurich`, `zürich`, `zurich`.

- **Unicode normalization** applies `unicodedata.normalize('NFKC', text)` to canonicalise characters that have multiple valid encodings, such as the ligature `fi` (one codepoint) unfolding to `fi` (two ASCII letters).
- **Accent folding** strips diacritics after normalization, mapping `ü` -> `u`, `é` -> `e`, `ß` -> `ss`. The library `unidecode` does this for arbitrary Unicode; `nltk`'s stemmers apply narrower per-language rules.

After all three, the three variants above become `zurich`. Users can type any spelling and match any document.

Accent folding is not free

Aggressive accent folding trades recall for precision. In German, “`schon`” (already) and “`schön`” (beautiful) have different meanings; folding both to “`schon`” makes them collide in the index. In French, “`où`” (where) and “`ou`” (or) become the same token. Whether to fold depends on the retrieval scenario: web search over cross-language content typically folds, while a legal-document index over a single language does not.

In practice, modern text-processing stacks apply Unicode normalization by default because it has no downside, case-fold when the retrieval scenario is case-insensitive (most web search), and fold accents only when the user population cannot easily type them.

Sentence segmentation

Some tasks need whole sentences as input, not tokens. Part-of-speech taggers work sentence by sentence because word ambiguity resolves in context. RAG systems (see the retrieval-augmented generation chapter) build larger chunks by grouping consecutive sentences until they hit a size limit. Both need a reliable answer to the question: where does one sentence end and the next begin?

Splitting on `.` is the naive approach and fails on the same abbreviations we saw earlier. In “`Dr. Watson's cat called Mrs. Hersley.`”, the periods after `Dr` and `Mrs` are not sentence boundaries; only the trailing period is. Multi-line quoted dialogue, ellipses, and decimal numbers add more edge cases.

Two libraries handle this well:

- **Punkt** (NLTK) is an unsupervised sentence segmenter that learns abbreviations and sentence-start patterns from a training corpus. NLTK ships pre-trained models for English and several other languages. It is fast and accurate on newspaper-style prose; it struggles on narrative dialogue where punctuation appears both inside and outside quotation marks.
- **spaCy** segments sentences as a side effect of its full linguistic pipeline. It is heavier than Punkt but handles complex sentence structures more consistently and supports many languages out of the box.

```
from nltk.tokenize import sent_tokenize
sent_tokenize("Dr. Watson's cat called Mrs. Hersley. She was Egyptian.")
# ["Dr. Watson's cat called Mrs. Hersley.", "She was Egyptian."]
```

Sentence segmentation is one of the few pieces of a classical pipeline that stayed useful into the LLM era. Every RAG system built on top of embeddings still needs to decide where its chunks begin and end, and sentence boundaries are the most reliable low-cost signal for that decision.

Word boundaries are not always spaces

Not every writing system uses spaces to separate words, and not every “word” in a modern document is meant to be a single token. Both cases break the “split on whitespace” assumption at the root.

Scriptio continua languages, including Chinese, Japanese, Thai, and classical Greek, do not use spaces at all. A single sentence is a continuous string of characters:

莎拉波娃现在居住在美国东南部的佛罗里达。

莎拉波娃	现在	居住	在	美国	东南部	的	佛罗里达
Sharapova	now	lives	in	US	southeast	in	Florida

Code identifiers pack multiple words into a single token by convention: `QueryParser`, `assertEquals`, `word_tokenize`. A code-search index that stores only whole identifiers cannot answer a query for `parser`.

Spoken-language transcripts come from a continuous phoneme stream. Speakers do not pause between words; the pauses that exist mark breath, hesitation, or emphasis, and rarely align with word boundaries. Even the pause at a sentence end is unreliable in fluent speech. The segmenter has to infer word boundaries from the phoneme sequence itself, aided by a pronunciation dictionary and a language model, and non-native pronunciation or unclear articulation shifts where those boundaries land.

There are two ways to break these continuous streams into useful tokens:

- **Dictionary lookup with a probabilistic tie-breaker.** The dictionary lists every candidate word starting at the current position, and a trained model (historically an HMM, today usually a neural network) picks the most likely segmentation. This works well when the vocabulary is closed and the domain is stable. It breaks on names, brand terms, and loanwords.
- **Sub-word tokens used directly for retrieval.** Instead of reconstructing words, we index overlapping character n-grams. For languages that *do* have whitespace, a common choice is trigrams with a # boundary marker indicating that a trigram starts at a word boundary (Cavnar & Trenkle 1994):

This course teaches multimedia retrieval.

```
#Thi his #cou our urs rse #tea eac ach che hes #mul ult lti tim ime med edi dia ...
```

For scriptio-continua text where no spaces exist, the # markers are simply omitted and the entire character stream is shingled into overlapping trigrams. Either way, queries are encoded identically:

```
Q = "teach multtimedia"
#tea eac ach #mul ult ltt tti tim ime med edi dia
```

Even with a different inflected form (“teaches” versus “teach”) and a typo (“multtimedia”), 10 of the 12 query trigrams match trigrams from the document. A retrieval model that supports partial matching with a proximity bonus finds the passage without any explicit stemming or spell-check.

This workaround for unsegmentable text turned out to be a foundational idea. Byte Pair Encoding (BPE) and WordPiece take the same principle (match parts of words when whole-word matching fails) and learn the token vocabulary from a corpus instead of fixing it to trigrams. Both power today’s large language models, and the semantic-search chapter covers them in detail.

Language detection

Both indexing documents and understanding queries depend on knowing what language the input is in. A German document benefits from Snowball’s German rules and the German stop-word list; running it through the English pipeline strips accents but leaves compounds and inflections untouched. A query like “Bücher von Goethe” from the chapter opener needs to be recognized as German before the retrieval system can restrict results to German-language books or apply German-specific normalization.

For long documents, a small number of hand-crafted rules is enough:

- **Alphabet diversity** looks at which scripts appear. Latin, Cyrillic, Greek, Arabic, Devanagari, Thai, and CJK (Chinese, Japanese, Korean) are easy to tell apart even from a single character.
- **Character diversity** narrows down within a script. ä, ö, ü are strong signals for German (and Turkish, Estonian, Finnish); ç for French, Portuguese, or Turkish; ñ for Spanish; ß almost exclusively for German.
- **Stop-word counts** track how many of the top hundred function words of each candidate language appear. English “the”, “of”, “and”; German “der”, “die”, “und”, “ist”; French “le”, “la”, “de”, “et”. Whichever language’s stop-word list matches most terms wins.
- **Vocabulary counts** extend the stop-word idea to a longer list of frequent content words.

For document-side indexing this is usually enough: a paragraph or a page provides many opportunities for the rules to fire, and the winning language emerges quickly. Once the language is known, the pipeline branches: a German document goes to the German Snowball stemmer, an English document to the English WordNet lemmatizer, and so on.

Short queries break the rules, though. “Mein computer” mixes German with an English loanword; “pain” is a French word for bread and an English word for suffering. Neither has enough evidence for the rules to fire reliably. That regime needs a statistical classifier over character n-grams, developed with Naive Bayes in [Intent Routing and Classification](#).

Where this sits in modern retrieval

Tokenization and normalization are the least glamorous part of the retrieval stack and still the largest source of quality issues in production. Modern lexical retrievers like Lucene, Solr, and Elasticsearch expose the whole pipeline as a chain of configurable analyzers: a tokenizer, a lowercase filter, a Unicode-normalization filter, a stop-word filter, a stemmer, sometimes a compound splitter. Dense retrieval sidesteps most of these choices by learning them implicitly in the embedding model, but hybrid stacks (BM25 combined with dense retrieval) still need the lexical side, and the lexical side still starts with these steps. Getting them wrong at ingestion time is expensive to fix later because the index has to be rebuilt.

Hands-on: Tokenization and Normalization

Run the naive regex tokenizer, the nltk word tokenizer, and the spaCy tokenizer on the “Dr. Watson” sentence. Normalize the results with case folding and accent stripping, and observe the effect on a small BM25 index.

[Open notebook ->](#)

Includes pre-run results; you can read through or download and experiment.

The next section takes tokens as given and looks at the second half of the recall gap: collapsing “car”, “cars”, “carrying”, and “carried” so a document mentioning any of them matches a query for any other.

Stemming and Lemmatization

The bookshop catalog stores documents word by word. A user searches “car repair” and misses titles like “cars”, “repairing cars”, and “repaired vehicles” because each surface form is a distinct token in the index. Tokenization from the previous section fixed splitting; it did not fix the fact that one root word has many written shapes. This section closes that gap. First it decides which tokens are worth indexing at all, then it collapses inflected surface forms of the same word onto a common stem or lemma.

Stop words: what to keep out

Half a dozen very frequent words dominate any English text. “The”, “of”, “and”, “to”, “is” together account for roughly a quarter of tokens in typical prose. We already saw in [From Text to Searchable Vectors](#) that these high-frequency terms appear in over 60% of documents yet carry almost no content signal. A **stop-word list** is the standard shortcut: a fixed set of high-frequency function words to discard before indexing and querying. English stop-word lists usually contain 100-200 words. The filtering logic is straightforward:

```
STOPWORDS = {'a', 'an', 'the', 'and', 'or', 'in', 'of', 'to', ...}

def remove_stopwords(tokens: list[str], stopwords: set = None) -> list[str]:
    sw = stopwords if stopwords is not None else STOPWORDS
    return [t for t in tokens if t not in sw]
```

Every major language has its own list. `nltk.corpus.stopwords.words('german')` returns 232 German function words; French, Spanish, Italian, and the other main European languages are provided by both `nltk` and `spaCy`.

Dropping stop words works well most of the time and fails visibly the rest of the time. The cost is a small class of queries where the stop word is content-bearing:

- A search for **Stephen King’s “It”** finds nothing if the tokenizer drops “it” as a stop word. The novel’s title is the stop word.
- A search for **“IT security”** loses “IT” (information technology) for the same reason. The remaining “security” retrieves every document about locks, guards, and stock exchanges.
- **“To be or not to be”** consists entirely of stop words. Every word disappears.
- **“The Who”** (the band) has both tokens on typical stop lists.

BM25 already handles high-frequency words gracefully through two mechanisms that earlier bag-of-words models lacked: inverse document frequency drives the weight of corpus-wide common terms toward zero, and term-frequency saturation prevents any single term from dominating a document’s score no matter how often it repeats. Together these make aggressive stop-word removal less necessary. The modern default is to keep stop words in the index (storage is cheap) but skip them when generating phrase collocations, computing term-based similarities, or building compact features for a classifier.

Never strip stop words from a phrase query

Dropping stop words before phrase matching turns “The Who” into an empty query and “not guilty” into “guilty”. If the retrieval scenario supports phrase or entity queries, the stop-word filter must be aware of the phrase context or must not run on those tokens at all.

Stemming: rule-based reduction

Once stop words are out, we still have “car”, “cars”, “carrying”, “carried”, “carrier”. They all point to the same concept, but a bag-of-words index sees five distinct tokens. **Stemming** applies a set of ordered rules that strip suffixes off inflected forms so that variants of one root end up as the same token.

The three widely-used English rule-based stemmers are Porter, Lancaster, and Snowball. All three take a token, apply an ordered sequence of suffix-stripping rules, and return a stem. The stem does not need to be a real English word; it only needs to be the same for all variants of the same lemma.

The Porter algorithm

The Porter algorithm is the classical English stemmer, published by Martin Porter in 1980. Chapter 1 introduced it as a black box; here we look at the internal structure.

Porter defines a **vowel** as A, E, I, O, U, and additionally Y when the preceding character is a consonant (so “Y” is a vowel in “rhythm” but a consonant in “yellow”). All other characters are consonants. Let C be a sequence of consonants and V a sequence of vowels. Every English word can be written in the form:

$$[C](VC)^m[V] \quad (3.1)$$

where the exponent m is the **measure** of the word. TREE has $m = 0$ (consonant sequence TR followed by vowel sequence EE, no VC pair in between), TROUBLE has $m = 1$, and TROUBLES has $m = 2$. Porter uses m to prevent over-stemming: rules that would strip “ATE” from “OPERATE” are gated behind $m \geq 1$ so they do not also strip the “ATE” from “MATE”.

The algorithm applies five steps in sequence, containing roughly 60 suffix-replacement rules in total. The entire implementation fits in about 400 lines of Python (or the equivalent in Java/C). Each step targets a different layer of English morphology:

Step	Goal	Example rules	Example
1a	Strip plurals	SSES arrow.r SS, IES arrow.r I, S arrow.r $\emptyset$	caresses arrow.r caress, ponies arrow.r poni, cats arrow.r cat
1b	Strip past tense / progressive	$(m > 0)$ EED arrow.r EE, (v) ED arrow.r $\emptyset$, (v) ING arrow.r $\emptyset$	feed arrow.r feed, plas- tered arrow.r plaster, motoring arrow.r motor
2	Simplify derivational suffixes	$(m > 0)$ ATIONAL arrow.r ATE, $(m > 0)$ IZER arrow.r IZE	relational arrow.r relate, digi- tizer arrow.r digitize
3	Continue simplification	$(m > 0)$ ICATE arrow.r IC, $(m > 0)$ ALIZE arrow.r AL	triplicate arrow.r triplic, for- malize arrow.r formal
4	Remove residual suffixes	$(m > 1)$ ION arrow.r $\emptyset$, $(m > 1)$ ISM arrow.r $\emptyset$	adoption arrow.r adopt, platon- ism arrow.r platon
5	Clean up trailing letters	$(m > 1)$ E arrow.r $\emptyset$, double consonant arrow.r single	rate arrow.r rate, controll arrow.r control

Within each step, rules are tried in order and only the first match fires. The conditions in parentheses prevent rules from applying to stems that are too short: $(m > 0)$ requires at least one vowel-consonant pair in the remaining stem, and (v) requires at least one vowel anywhere. This is what keeps Step 1b from stripping “ED” off a two-letter word.

The full rule set stems on the order of 95% of English text correctly. Porter’s original paper (Porter, 1980) lists every rule; the `nltk.PorterStemmer` implementation is faithful to that specification.

```
import nltk
porter = nltk.PorterStemmer()

for w in ['car', 'cars', 'carrying', 'carried', 'carrier']:
    print(w, '->', porter.stem(w))

# car      -> car
# cars     -> car
# carrying -> carri
# carried  -> carri
# carrier  -> carrier
```

Note that “carrying” and “carried” collapse to `carri` (not to `carry`), and “carrier” is left intact. The stems are not linguistically pretty, but they map three variants of the verb “carry” onto the same token, which is what the retrieval index needs.

Lancaster and Snowball

Lancaster (Paice, 1990) is a more aggressive stemmer for English. It applies iterative rule-based suffix removal until no rule fires, producing shorter stems than Porter. Speed is its main advantage: on average it does fewer rule scans per word. Its main risk is collisions: unrelated words can reduce to the same short stem.

```
lancaster = nltk.LancasterStemmer()
for w in ['one', 'only', 'organization', 'organize']:
    print(w, '->', lancaster.stem(w))

# one          -> on
# only         -> on
# organization -> org
# organize     -> org
```

“one” and “only” both stem to “on”, so a document about “the only book on Rome” collides with one about “book number one”. For a scenario that favors recall the collisions are usually acceptable; for a name search or a precise query they are a problem.

Lancaster over-stems short words

Lancaster reduces short words to two- or three-letter stems that collide across unrelated meanings (“one” and “only” both become “on”). Use Lancaster when speed dominates and the collection is large enough that a few token collisions do not affect ranking. Prefer Porter or Snowball on short-document collections where each collision matters.

Snowball is Martin Porter’s own multi-language framework, published later, that generalizes the Porter design. It supplies stemmers for around 20 European languages including English, German, French, Spanish, Italian, Dutch, and Russian. On English, Snowball is essentially a revised Porter with a handful of small corrections; on other languages, it is often the only rule-based option available.

```
snowball_en = nltk.SnowballStemmer("english")
snowball_de = nltk.SnowballStemmer("german")

snowball_de.stem("Wagen"), snowball_de.stem("Wägen") # ('wag', 'wag')
snowball_de.stem("Reparatur"), snowball_de.stem("reparieren") # ('reparatur', 'repar')
```

The German stemmer folds umlauts (Wägen -> Wagen -> wag) and reduces “reparieren” to repar, which does not match reparatur. Rule-based stemmers approximate the linguistic stem but do not compute it: for German, the noun “Reparatur” and the verb “reparieren” share a root but have different stems under any pure suffix-stripping rule set.

Lemmatization: dictionary-based reduction

Rule-based stemming is fast and needs no external data, but it produces linguistically incorrect stems and cannot handle strong inflections. English “go” and “went” share no letters at all, so no suffix rule can map them together. **Lemmatization** takes a different approach: look the word up in a dictionary and return its recorded base form.

The three components of a dictionary-based lemmatizer are:

1. A **rule set** for regular inflections (-s, -ed, -ing, plurals with -ies).
2. An **exception list** for irregular forms: went -> go, ate -> eat, mice -> mouse, children -> child.
3. A **dictionary** of all base forms in the language, used to check whether a candidate form is valid.

The algorithm proceeds in order:

1. Take the current token and its part-of-speech (POS) tag (verb, noun, adjective, ...). POS tagging is covered later in this chapter; what matters here is that “meeting” as a noun lemmatizes to meeting, while “meeting” as a verb lemmatizes to meet.
2. If the token is in the dictionary as-is, it is already a base form. Return it.
3. If the token is in the exception list for its POS, return the base form listed there.
4. Apply the regular-inflection rules for its POS, checking the dictionary after each rule. Return the first match.
5. If no rule produces a dictionary word, return the token unchanged. This handles names, misspellings, and loanwords.

WordNetLemmatizer in nltk and the built-in lemmatizer in spaCy are the two standard implementations. Their exception lists cover several thousand irregular forms each.

```
wn = nltk.WordNetLemmatizer()
wn.lemmatize('carried', 'v')    # 'carry'
wn.lemmatize('carrier', 'n')    # 'carrier'
wn.lemmatize('mice', 'n')       # 'mouse'
wn.lemmatize('went', 'v')       # 'go'
wn.lemmatize('better', 'a')     # 'good'
```

Note the last case: “better” lemmatizes to “good” because the exception list encodes the adjective’s comparative form. No rule-based stemmer can do this.

spaCy runs a full linguistic pipeline that assigns POS tags automatically, so the lemmatizer is applied without the caller having to pass a tag:

```
nlp = spacy.load('en_core_web_sm')
for token in nlp("She carried the mice yesterday and went home"):
    print(token.text, '->', token.lemma_)

# She      -> she
# carried  -> carry
# the      -> the
# mice     -> mouse
# yesterday -> yesterday
# and      -> and
# went     -> go
# home     -> home
```

For retrieval, the practical difference between rule-based stemming and dictionary lemmatization is that a lemmatizer gives you the linguistic base form and a stemmer gives you an arbitrary reduction that just happens to be consistent within the class. Both work for matching a query to a document. Lemmatization is heavier because it needs POS tagging plus a dictionary lookup; stemming is a handful of string operations.

Multi-lingual comparison

Every stemmer and lemmatizer described above assumes one language. Running an English stemmer on French text strips accents and collapses “les” to “l” without doing anything useful. The right choice depends on the language of the input, which is what the language detector from the previous section is for.

Snowball is the go-to rule-based option outside English because it covers 20 languages under a single framework. spaCy is the go-to lemmatizer for the same set. On the same French sentence, the two diverge on inflected verb forms:

Input:	"Nous avons aperçu les châteaux"
Snowball:	['nous', 'avons', 'aperçu', 'le', 'château']
spaCy:	['nous', 'avoir', 'apercevoir', 'le', 'château']

Snowball preserves accents in the French output (visible on aperçu and château) but it does not recognize inflected verb forms: avons becomes avion by simple suffix stripping rather than mapping to avoir, and

aperçu stays unchanged rather than lemmatizing to the infinitive apercevoir. spaCy's French lemmatizer handles both because it has the verb tables. Both correctly reduce the plural châteaux to château while keeping the circumflex. On entity-heavy queries the difference is small; on prose retrieval where verbs carry the topic, it moves the recall needle.

In German, nouns inflect for case, gender, and number, and German also forms diminutives with “-chen” and “-lein”. The two tools handle these differently:

Input:	"die Häuser und ihre Häuschen"
Snowball:	['die', 'haus', 'und', 'ihr', 'hausch']
spaCy:	['der', 'Haus', 'und', 'ihr', 'Häuschen']

Both reduce the plural “Häuser” to the singular (“haus” for Snowball, which also lowercases; “Haus” for spaCy). They part ways on the diminutive “Häuschen”: Snowball over-stems it to “hausch”, a token that matches neither “Haus” nor any real word, while spaCy keeps “Häuschen” as its own lemma. For queries against German text, spaCy's dictionary lemmatizer avoids these spurious stems, though the small models are weak on strong verbs and do not always map inflected verb forms to a common infinitive.

Neither approach handles compound splitting, which is the concern of the next section: “Wolkenkratzer” (skyscraper) stays as a single token under both stemmers.

Where this sits in modern retrieval

Rule-based stemming and dictionary lemmatization are still the default text-processing configuration for lexical retrievers. Lucene, Solr, and Elasticsearch ship analyzers for every major language that combine tokenization, stop-word filtering, and stemming or lemmatization in the same order this section presented them. Stemming is the option chosen when performance matters or when no dictionary is available; lemmatization is preferred when accuracy on strongly-inflected languages or on strong-verb English matters more than throughput.

Dense retrieval learns these normalizations implicitly during embedding training. A well-trained embedding model puts “carried”, “carrying”, and “carrier” near each other in vector space without an explicit stemming step. This does not make the lexical stack obsolete: hybrid retrieval combines BM25 (which needs stemming or lemmatization to keep recall high) with dense retrieval (which does not), and the two together consistently outperform either alone. The classical stemmers of this section are still on the query path of every serious production search stack in 2025.

Hands-on: Stemming and Lemmatization

Compare Porter, Lancaster, Snowball, WordNet, and spaCy on English, German, and French text. See how each stemmer treats strong verbs, plurals, and compounds.

[Open notebook ->](#)

Includes pre-run results; you can read through or download and experiment.

The next section takes stemmed and lemmatized tokens as given and looks at what happens when meaning crosses word boundaries: “New York” is not “New” plus “York”, and “Wolkenkratzer” is not “Wolke” plus “kratzer”.

Phrases and Compounds

Tokenization decides what a token is; stemming and lemmatization decide which tokens are equivalent. Both operate at the level of one word. This section handles the two cases where a one-word view of meaning breaks down. Phrases pack multiple words into one concept: “New York” is not “New” plus “York”, and a query for “New” retrieves the wrong documents. Compounds do the opposite: “Bücherregal” (bookcase) is one German token built from “Bücher” (books) and “Regal” (shelf). A shopper on an e-commerce site who searches for “Regal” will not see the bookcase listings until we split the compound into its parts. Both cases need explicit handling before the retrieval model sees the tokens.

Bi-grams, tri-grams, phrases

The naive bag-of-words model treats a document as a multiset of independent tokens. “New York City”, “Salt Lake City”, and “prime minister” become bags of three or two unrelated tokens. Searching for “New York” as two separate terms still retrieves every document that mentions the city, so recall is not the problem. Precision is: the query also matches documents where “new” and “York” occur unrelated (“a new book from York University”), and a plain bag-of-words index has no way to tell those apart from documents where the two words sit side by side. Without a proximity or phrase mechanism, the genuine “New York” documents cannot be ranked above the incidental co-occurrences. Indexing “New York” as a single token `new_york` restores precision and gives the ranker a clean signal.

This generalises to n-grams: a bi-gram joins two adjacent words, a tri-gram three, and so on. Indexing the bi-grams and tri-grams of “the city of New York City” produces

```
the, city, of, new_york_city, new_york, york_city, city
```

with `_` marking a phrase token. We keep the original single-word tokens and emit every overlapping n-gram rather than only the longest one. This is deliberate: a query for “New York” must still match even though this document only ever contains the longer “New York City”, and a query for the single word “city” must still match as well. Keeping all of them costs some index space but avoids missing any of these queries. A phrase query for “New York” now hits `new_york` directly, without proximity constraints. The question that follows is which n-grams are worth indexing. Adding every bi-gram in the corpus is prohibitive (it turns a 100k-token vocabulary into a 10-billion-pair vocabulary) and most of them are noise: “it is”, “of the”, “and to”. We need a scoring function that selects the small subset of bi-grams that mean something the individual tokens do not.

Naive frequency and the stop-word problem

The obvious approach is to count how often each bi-gram appears in the corpus and keep the most frequent ones. On any English text this immediately breaks:

of the	12,847
in the	9,203
to the	7,451
and the	6,988
for the	5,102
...	

The top of the list is dominated by pairs of stop words. Filtering these out helps but does not solve the problem: even after stripping stop-word pairs, high-frequency co-occurrences of common words (“said Holmes”, “young man”, “could see”) still bubble to the top and squeeze out the phrases we actually want. Frequency alone cannot tell us whether two words appear together because they mean something together or because they are both common.

Two better scoring functions capture that distinction: pointwise mutual information (PMI) and the likelihood ratio (LHR).

Pointwise mutual information

The intuition behind **pointwise mutual information** is straightforward. Two words that mean something together should co-occur more often than their individual frequencies would suggest under independence. In *A Study in Scarlet*, a corpus of about 43,200 tokens, “said” appears 207 times and “Holmes” 98 times. If the two were independent, we would expect the bi-gram “said Holmes” to occur about $207 \cdot 98/N \approx 0.5$ times. It actually occurs 12 times, roughly 25 times more than chance. That is a real signal, but a modest one: “said” precedes dozens of different names throughout the book.

Compare “Sherlock Holmes”. “Sherlock” appears 52 times, and every one of those 52 occurrences is followed by “Holmes”. Independence would predict about $52 \cdot 98/N \approx 0.1$ co-occurrences; the observed count of 52 is more than 400 times higher. “Sherlock” is effectively bound to “Holmes”.

Key Formula: Pointwise Mutual Information

$$\text{pmi}(t_1, t_2) = \log_2 \frac{p(t_1, t_2)}{p(t_1) \cdot p(t_2)} = \log_2 \frac{N \cdot \text{tf}(t_1, t_2)}{\text{tf}(t_1) \cdot \text{tf}(t_2)} \quad (3.2)$$

Here $\text{tf}(t)$ is the number of times term t occurs in the corpus, and $\text{tf}(t_1, t_2)$ is the co-occurrence count: how often the bi-gram appears, that is, how often t_2 immediately follows t_1 . Dividing each count by the corpus size N turns it into a probability: $p(t) = \text{tf}(t)/N$ for a single term and $p(t_1, t_2) = \text{tf}(t_1, t_2)/N$ for the pair. PMI is then the log-ratio of the observed joint probability to the joint probability that independence would predict; it is high when two tokens almost always appear together and rarely apart.

Two things help read the formula. First, $\log_2(N)$ is the same constant for every bi-gram, so ranking by PMI is the same as ranking by $\log_2 \text{tf}(t_1, t_2) - \log_2 \text{tf}(t_1) - \log_2 \text{tf}(t_2)$. Second, look at what maximizes the score. PMI is largest when all three counts equal 1: a word that occurs once, sitting next to another word that occurs once, giving $\text{pmi} = \log_2 N$. Even when the two words and their pair all occur n times together and never apart, the score is $\log_2(N/n) = \log_2 N - \log_2 n$, which only falls as n grows. So PMI rewards rarity, and this is the mirror image of the stop-word problem: instead of frequent function words dominating, two rare words that happen to land side by side by chance float to the top. In *A Study in Scarlet*, 199 different bi-grams reach the maximum score of $\log_2 N \approx 15.4$, every one of them a pair of words that each occur exactly once and only next to each other (“admired treated”, “airy cheerfully”, “ambitious title”). All of them outrank “Sherlock Holmes”. A single chance adjacency is no evidence of a real collocation; there is simply too little frequency to be confident the two words prefer each other. The cure is a minimum-frequency filter: require a bi-gram to occur at least, say, three times before scoring it at all, which drops the low-evidence rare pairs.

Example: PMI on the Sherlock Holmes corpus

Tokens are lowercased and reduced to alphabetic-only words, as in the normalization step of the previous section. Ranked by PMI with a minimum bi-gram frequency of 3 and a stop-word filter, the top of the list for *A Study in Scarlet* (about 43,200 tokens, 115 bi-grams surviving the filter) looks like this:

Rank	Bi-gram	$\text{tf}(t_1, t_2)$	$\text{tf}(t_1)$	$\text{tf}(t_2)$	PMI
1	audley court	3	3	4	13.40
2	torquay terrace	3	3	4	13.40
3	avenging angels	4	4	4	13.40
4	lauriston gardens	6	6	6	12.81
8	salt lake	10	11	10	11.94
11	scotland yard	6	8	9	11.81
...	...				
45	sherlock holmes	52	52	98	8.78

Even after the frequency filter, PMI's rare-word bias persists. The top three are tied at 13.40 because each reduces to $\log_2(N/4)$: "audley court" and "torquay terrace" pair a word occurring 3 times with one occurring 4 times, always together, and "avenging angels" pairs two words occurring 4 times each. All are rare pairs whose parts appear almost only together. "Sherlock Holmes", the defining phrase of the book, ranks only 45th of 115: because "Holmes" occurs 98 times, the larger denominator drags its PMI down to 8.78. The filter removed the $\text{tf}=1$ noise but did not fix the underlying preference for rarity.

Likelihood ratio

The likelihood ratio test attacks the same problem from a hypothesis-testing angle rather than an information-theoretic one. It asks: how much better is the "these words are dependent" hypothesis than the "these words are independent" hypothesis, given the observed counts?

Let $\text{tf}_1 = \text{tf}(t_1)$, $\text{tf}_2 = \text{tf}(t_2)$, and $\text{tf}_{12} = \text{tf}(t_1, t_2)$. Two hypotheses:

- H_1 (independence): the probability that t_2 follows any word is a single value p , whether the previous word is t_1 or not. The maximum-likelihood estimate is $p = \text{tf}_2/N$.
- H_2 (dependence): the probability that t_2 follows t_1 is $p_1 = \text{tf}_{12}/\text{tf}_1$, and the probability that t_2 follows any word other than t_1 is $p_2 = (\text{tf}_2 - \text{tf}_{12})/(N - \text{tf}_1)$, with $p_1 \neq p_2$.

Assuming a binomial distribution for the "next word is t_2 " events, the likelihood of each hypothesis is:

$$L(H_1) = b(\text{tf}_{12}; \text{tf}_1, p) \cdot b(\text{tf}_2 - \text{tf}_{12}; N - \text{tf}_1, p) \quad (3.3)$$

$$L(H_2) = b(\text{tf}_{12}; \text{tf}_1, p_1) \cdot b(\text{tf}_2 - \text{tf}_{12}; N - \text{tf}_1, p_2) \quad (3.4)$$

with $b(k; n, x) = \text{binom } nkx^k(1-x)^{n-k}$.

Key Formula: Log Likelihood Ratio

$$\log \lambda = \log \frac{L(H_1)}{L(H_2)} \quad (3.5)$$

The log-ratio of the likelihood under independence to the likelihood under dependence. Values near zero mean the two hypotheses fit the data equally well; strongly negative values mean dependence fits much better and the bi-gram is genuine. Bi-grams are usually ranked by $-2 \log \lambda$, which under H_1 follows a chi-squared distribution with one degree of freedom.

LHR is more robust than PMI on sparse data because it directly compares two probabilistic models rather than comparing observed to expected counts. It does not have PMI's rare-word bias, so infrequent bi-grams score less aggressively. The trade-off is that LHR ranks frequent grammatical pairs highly, because those pairs really are non-independent even though they are useless as phrases. The bi-gram "I am" is the clearest example: "am" occurs 41 times and 39 of them follow "I", so the dependence is overwhelming, yet "I am" is worthless as an index phrase. This is why LHR still needs a stop-word filter: without one, "of the", "to be", "had been", and "I am" crowd the top of the ranking alongside the genuine names.

Example: LHR on the Sherlock Holmes corpus

Ranked by $-2 \log \lambda$ on the same corpus, with the same minimum-frequency and stop-word filter as the PMI table:

Rank	Bi-gram	$\text{tf}(t_1, t_2)$	$\text{tf}(t_1)$	$\text{tf}(t_2)$	$-2 \log \lambda$
1	sherlock holmes	52	52	98	668
2	jefferson hope	34	37	56	458
3	john ferrier	29	39	62	352
4	brixton road	13	15	27	188

5	salt lake	10	11	10	181
9	scotland yard	6	8	9	98
11	baker street	6	6	29	89

With the stop-word filter in place, LHR puts the main characters and places of the novel on top: “Sherlock Holmes”, “Jefferson Hope”, “John Ferrier”, “Brixton Road”. Without the filter, the grammatical pairs discussed above (“of the”, “to be”, “I am”) would sit among them. The filter removes that whole class in one step.

PMI or LHR: which to use

The two measures rank the same corpus differently because they measure different things. PMI measures the *strength* of association: how much more often two words appear together than chance predicts, regardless of how often that is. LHR measures the *evidence* for association: how confidently the data rules out independence, which grows with frequency. “Sherlock Holmes” makes the contrast concrete: it ranks 45th of 115 under PMI, held down because “Holmes” is common, but first under LHR, which rewards the sheer weight of 52 co-occurrences.

Because they optimise different quantities, the two fail in opposite directions, and a genuine phrase can score well on one and poorly on the other:

- **Rare but tight** phrases occur a handful of times, always as a unit. They score high on PMI and low on LHR. In our results these are PMI’s top rows: “Lauriston Gardens” (the address of the murder), “Audley Court”, “chemical laboratory”. LHR buries them below its frequency-driven head, so PMI is what surfaces them.
- **Frequent but loose** pairs such as “young man” or “could see” occur often but are not real phrases. They score moderately on LHR but low on PMI, which correctly discounts them because the individual words are common.
- **Frequent and tight** phrases such as “Sherlock Holmes” or “Jefferson Hope” score high on both and need no adjudication.

No single measure is uniformly best. Some practical guidance:

- **For a phrase vocabulary that maximises recall** (finding as many genuine phrases as possible), take the top- k from both measures and union them: LHR contributes the well-attested head, PMI the rare-but-specific tail of named entities and technical terms that users actually search for. A union only adds candidates, so plan a downstream prune, whether a human review pass or a secondary filter, to drop the frequent-but-loose pairs that LHR lets through.
- **For a compact, high-precision list**, use LHR alone with the stop-word filter and a frequency floor. Its preference for well-attested pairs is an asset here, and you accept missing some rare phrases.
- **The filters matter more than the choice of measure.** A minimum-frequency floor removes PMI’s chance pairs and a stop-word filter removes LHR’s grammatical pairs; without both, neither measure produces a usable list. Scale the frequency floor to the corpus: 3 works for a single novel, but a large collection needs a higher floor to keep the vocabulary manageable.
- **An alternative to the union** is to use one measure to rank and the other to filter, for example rank by LHR but drop any pair whose PMI falls below a threshold. This keeps LHR’s ordering while using PMI to veto the frequent-but-loose pairs, trading some recall for precision.

Choosing a threshold and indexing strategy

Once a scoring function is chosen, we pick a threshold and add every bi-gram above it to the vocabulary. The threshold is not a precise cut-off; it trades index size against phrase coverage. Missing a bi-gram is not fatal: the document is still retrievable through its individual terms, so recall is unaffected. What we lose is the precision and ranking boost the phrase token would have given, which lets more incidental co-occurrences through as noise. For many use cases, such as fact-checking or broad information search, that extra noise is acceptable.

Building the bi-gram set with nltk (optional reading)

nltk provides collocation finders that do the ranking and filtering directly. The result feeds the bigrams set used above.

```
from nltk.collocations import BigramCollocationFinder, BigramAssocMeasures
from nltk.corpus import stopwords

finder = BigramCollocationFinder.from_words(tokens)
finder.apply_freq_filter(3) # require at least 3 co-occurrences

ignored = set(stopwords.words('english'))
finder.apply_word_filter(lambda w: len(w) < 3 or w.lower() in ignored)

# take the top-k scoring pairs and keep them as tuples
top = finder.score_ngrams(BigramAssocMeasures.likelihood_ratio)[:200]
bigrams = {pair for pair, score in top}
```

Applying the vocabulary changes how we tokenize, without changing the single scan through the text. Once the accepted bi-grams are known, we tokenize each document, and every query the same way, in one left-to-right pass. At each position we emit the single token, and we do one extra vocabulary lookup to check whether the current token together with the next one forms an accepted bi-gram; if it does, we emit the phrase token as well and move on to the next position, so overlapping phrases are still caught. Keeping the single tokens alongside the phrase is deliberate, as the chapter opener showed: a query for “Holmes” must still match a document that only contains “Sherlock Holmes”.

A compact implementation keeps two vocabularies: the ordinary single tokens, and a set of accepted token pairs stored as Python tuples.

```
bigrams = {("new", "york"), ("york", "city")} # accepted pairs from PMI/LHR (see code above)

def apply_phrases(tokens: list[str], bigrams: set[tuple[str, str]]) -> list[str]:
    result = []
    for i, tok in enumerate(tokens):
        result.append(tok) # always keep the single token
        if i + 1 < len(tokens) and (tok, tokens[i + 1]) in bigrams:
            result.append(f"{tok}_{tokens[i + 1]}") # add the phrase token
    return result

apply_phrases("the city of new york city".split(), bigrams)
# ['the', 'city', 'of', 'new', 'new_york', 'york', 'york_city', 'city']
```

The two overlapping phrases “new york” and “york city” are both emitted, next to the single tokens, exactly the mix the chapter opener described. The accepted-pair set itself comes from the collocation finder: rank with PMI or LHR, apply the frequency and stop-word filters, take the top-*k*, and store each surviving bi-gram as a tuple.

The construction extends to tri-grams and quad-grams: keep a set of triples, check a window of three tokens, and emit a three-word phrase when it matches. Returns diminish quickly, though. Any four-word sequence is rare enough that the scoring functions lose statistical power, and the extra vocabulary rarely earns its storage. Most production systems stop at bi-grams and handle the occasional longer phrase with proximity search at query time. nltk also provides TrigramCollocationFinder and QuadgramCollocationFinder with matching TrigramAssocMeasures and QuadgramAssocMeasures.

Compounds

A phrase glues two tokens together. A compound goes the other way: it hides several words inside a single token. German makes this productive: “Wolkenkratzer” (skyscraper) combines “Wolke” (cloud) and “Kratzer” (scratcher). “Rindfleischetikettierungsüberwachungsaufgabenübertragungsgesetz” (a Mecklenburg-Vorpommern law from 1999-2013 delegating the supervision of beef labelling) chains together seven concepts as one token. Finnish and Dutch have the same construction. Programming languages have their own version: `word_tokenize`, `assertEquals`, and `QueryParser` each pack a whole phrase into one identifier.

Compounds break retrieval in an asymmetric way. A user who types “Etikettierung Gesetz” cannot match a document indexed under the full compound. The reverse also fails: a query for “Rindfleischetikettierungsüberwachungsaufgabenübertragungsgesetz” only finds documents that spelled it exactly the same way, which is optimistic for a 63-letter word. The fix is to index both the compound and its parts, giving the retriever a chance to match either.

Endocentric vs exocentric compounds

Not every compound splits usefully. **Endocentric** compounds derive their meaning from their parts: “sunglasses” are a kind of glasses, “Abfalleimer” is a kind of Eimer (bucket), and splitting into parts is safe. **Exocentric** compounds do not: a “skyscraper” is not a kind of “scraper”, and the German “Wolkenkratzer” is not a kind of “Kratzer” (scratcher). Splitting an exocentric compound into parts adds tokens with the wrong semantics to the document, hurting precision. Whether the recall gain outweighs the precision loss depends on the collection: news search benefits from splitting, legal search often does not.

Splitting compounds

Compound splitting proceeds in two stages: generate candidate splits, then score them.

For **generating candidates**, we use language-specific rules. In English, hyphens and hyphenation syllables give the splits: “must-have” splits into “must” and “have”, “skyscraper” into “sky”, “scrap”, “er”. In German, syllable boundaries following German hyphenation rules produce candidates: “Wolkenkratzer” becomes (“wol”, “ken”, “krat”, “zer”), and every way of grouping consecutive syllables into words yields a candidate split: (wolken, kratzer), (wolken, krat, zer), (wol, kenkratzer), and so on. German also uses binding letters like “s” between compound parts (“Schiffahrtskapitän” needs to be tested both as “Schiffahrt-Kapitän” and “Schiffahrts-Kapitän”), so the candidate generator must try both variants.

For **scoring candidates**, we discard any split containing a piece that is not in the language’s dictionary, then score the survivors by how frequent their parts are. Crucially, the score is not a winner-take-all decision. Just as we indexed both “new york” and its parts, and both the long beef-labelling law and its constituents, we do not have to commit to a single decomposition of a compound: we can keep several plausible splits and index the union of their parts alongside the full compound. The score’s job is to rank plausibility and prune weak candidates, not necessarily to crown one winner.

Key Formula: Compound Split Score

$$\text{score}(S) = \frac{1}{|S|} \sum_{p_i \in S} \log \frac{\text{tf}(p_i)}{N} \quad (3.6)$$

The average log-frequency of the parts in split S , computed against a reference corpus of N tokens. A higher score means the split is into more common words, so it is a more plausible decomposition. Use it either to take the single best split, or to keep the top few plausible splits.

The intuition: the dictionary filter first removes any split containing a piece that is not a real word, so (“Wol”, “Kenkratzer”) is discarded outright because neither piece is a German word. Among the splits that survive, the score ranks plausibility by the frequency of the parts. We then either keep the single top split,

or, favouring recall, keep several plausible splits and index the union of their parts. The average is used rather than the sum so that splits with different numbers of parts are comparable.

Example: Splitting Autobahnausfahrt

Candidate splits of “Autobahnausfahrt” (motorway exit), scored by the average frequency of their parts. Frequencies are on the Zipf scale, where higher means more common and 0 means the word does not appear in the corpus:

Split	Parts (Zipf)	Average	Keep?
(Auto, Bahn, Ausfahrt)	5.3, 5.0, 3.7	4.67	yes
(Autobahn, Ausfahrt)	4.4, 3.7	4.05	yes
(Auto, Bahnausfahrt)	5.3, 0.0	•	discarded (“Bahnausfahrt” unseen)

Two things stand out. First, the score correctly throws out (Auto, Bahnausfahrt) because “Bahnausfahrt” never appears in the corpus. Second, plain average frequency ranks the finer split (Auto, Bahn, Ausfahrt) *above* the more natural (Autobahn, Ausfahrt), because “Auto” and “Bahn” are extremely common words. A frequency score alone tends to over-split into short common words; production splitters counteract this with a penalty for more or shorter parts, or a list of known compounds.

But we do not have to choose between the two surviving splits. We can index “Autobahnausfahrt” together with the union of their plausible parts, “Auto”, “Bahn”, “Autobahn”, and “Ausfahrt”, so a query for any of them matches. This over-indexing costs storage and lets some noise through, favouring recall; keeping only the single best split is the extra investment we make when a use case needs the precision.

Where this sits in modern retrieval

Phrase detection and compound splitting are both still active production techniques. Lucene ships two compound-word filters. The `HyphenationCompoundWordTokenFilter` is the two-stage method above in production form: a hyphenation grammar, the same pattern file that breaks words across lines at the end of a text line, proposes candidate break points in place of our syllable rules, and a dictionary keeps only the fragments that are real words. The matching sub-words are added to the token stream next to the original compound, exactly the union-indexing we described. Its sibling, the `DictionaryCompoundWordTokenFilter`, skips the hyphenation step and tests substrings directly against the word list, which is slower and produces more spurious fragments, so Elasticsearch recommends the hyphenation variant for German, Dutch, and other compounding languages. Solr exposes the same filters through the Redlink extension, and actively maintained dictionaries such as `uschindler/german-decompounder` provide the word lists these filters consume.

Dense retrieval learns the compound structure implicitly through sub-word tokenizers like BPE and WordPiece, but the learning is imperfect. A 2023 study (arXiv:2305.14214) showed that SentencePiece, the tokenizer most widely used with multilingual language models, still splits German compounds into semantically incoherent fragments a significant fraction of the time. Hybrid retrieval stacks that combine BM25 with dense embeddings therefore keep the explicit decompounder on the BM25 branch, letting the neural side learn what it can while the classical side handles the compounds the classical way.

Phrase detection has an analogous story. Bi-gram indexing appears in modern lexical retrievers wherever named entities matter: product-search stacks index brand-name bi-grams, patent search indexes technical compounds, and academic search indexes conference names. PMI and LHR remain the standard offline scoring functions for building those bi-gram vocabularies.

Hands-on: Phrases and Compounds

Extract PMI and LHR bi-grams from a text corpus, apply frequency and stop-word filters, and split a set of German compounds into their parts.

[Open notebook ->](#)

Includes pre-run results; you can read through or download and experiment.

The next section leaves the document-recall problem behind and turns to the query side of the two-way failure from the chapter opener. Given a natural-language query, how do we extract enough structure from it that the retriever can do more than bag-of-words matching?

Query Understanding

The first three sections closed the recall gap on the document side. A well-processed collection now has one token per concept, phrases identified, and compounds split. The chapter opener's other failure remains: the query "Bücher von Goethe" arrives as three tokens and the retriever has no idea that "Bücher" and "books" are the same concept, that "Goethe" names a person, or that the sentence is a request for works by a specific author. This section adds three tools that extract that kind of structure from free-text: word-relationship expansion, part-of-speech tagging, and named entity recognition. The section ends with the practical concern of spelling: a query that arrives misspelled needs to be repaired before any of these tools can help.

Word relationships: synonyms and homonyms

Stemming and lemmatization collapse different surface forms of the *same* word onto one token. They do nothing about the looser assumption we used so far: that one normalized token is independent of another normalized token. Every retrieval model in the book to this point has relied on it, treating tokens as unrelated dimensions with no notion that two of them might mean similar things or that one might mean two things. Real language breaks that assumption in two directions. A **synonym** is two different words for the same concept ("buy" and "purchase"); a **homonym** is one word carrying two concepts ("bank" the institution and "bank" the riverside). Lemmatization helps with neither, because "buy" and "purchase" reduce to unrelated stems and "bank" keeps a single stem whichever sense is meant. This section handles both with classical lexical tools; the semantic search chapter returns to the independence assumption in full and replaces it with learned representations that place related tokens near each other.

Synonyms break recall. A user searching a product catalog for "purchase history" misses documents indexed under "buy" or "buying", even though they hold exactly what the user wants, purely because the tokens differ. Two responses:

- **Synonym expansion (classical).** Attach a synonym list to each token. When the document contains "purchase", also index "buy" and "acquire". When the query contains "buy", also search for "purchase" and "acquire". The synonym lists come from curated resources (WordNet for English, thesaurus databases for other languages) or from domain-specific glossaries (medical terminologies, legal dictionaries).
- **Dense embeddings (modern).** Instead of maintaining synonym lists, learn a vector space where "buy" and "purchase" naturally end up near each other. This is the approach of the semantic search chapter and it subsumes the synonym problem, at the cost of an embedding model and a vector index.

Domain vocabularies show the same problem more sharply. The chapter opener's F1 query, "Who won the F1 race on the weekend?", needs to match documents that call the same event a "Grand Prix" and never use the tokens "F1" or "race" at all. WordNet does not capture this mapping because it is not a linguistic synonym but a domain shorthand. The fix is a domain-specific synonym list maintained by the search team: F1 maps to {Formula 1, Formula One, Grand Prix}, race maps to {Grand Prix, GP}. Product-search systems maintain similar lists for brand aliases ("iPhone" -> "Apple iPhone", "iPhones") and category synonyms ("laptop" -> "notebook").

Hand-maintained lists work well for the common, high-value terms a search team can anticipate, but they cannot keep up with the long tail: driver names the list has never seen, one-off event aliases, or code words specific to a community. This is where dense retrieval starts to pay for itself, because an embedding model learns which words appear in similar contexts without any manual list. Modern production stacks combine both: hand-crafted lists for the head, embeddings for the tail. The full argument comes back in the semantic search chapter.

Homonyms are the other direction, and the harder one. A query for "bank" cannot know whether the user means the financial institution or the riverside, and "lead" could be the metal or the verb meaning to guide. Two responses again:

- **Context from surrounding words (classical).** The words around a homonym usually fix its sense. Part-of-speech tagging is the simplest case: "lead" the noun (the metal) and "lead" the verb (to guide)

carry different tags, so tagging at index and query time separates them. Broader lexical context helps further: a text that also mentions “money”, “account”, or “loan” points to the financial “bank”, while “river”, “water”, or “shore” points to the riverside. This works whenever there is enough surrounding text to read.

- **Contextual embeddings (modern).** Sentence encoders and large language models represent a word by the company it keeps, so “bank” in “the river bank” and “bank” in “the savings bank” receive different vectors. They resolve the sense from context without any hand-written rules, which is why they handle homonyms far more reliably than lexical rules do. This is the approach developed in the semantic search chapter.

Both approaches lean on context, so both fail when the query is too short to supply any: a bare query “bank” gives the system nothing to disambiguate on. There the only options are to return results for both senses or to ask, which is what web search does with “did you mean” suggestions. Genuine ambiguity with no context is inherent to language, and no technique fully removes it.

Hypernyms and hyponyms

WordNet provides two more relationships that expand queries in a different direction. A **hypernym** is a broader category (“animal” is a hypernym of “cat”), and a **hyponym** is a narrower one (“cat” is a hyponym of “animal”, and “mammal” is both a hyponym of “animal” and a hypernym of “cat”). These relationships form a hierarchy, and every English noun in WordNet sits somewhere in that hierarchy.

Three retrieval uses:

- **Faceted search.** A user searching for “animals” is offered “cats”, “dogs”, “birds” as sub-facets that drill down into the hypernym tree. If the initial query returns too few results, the user is offered the parent facet (“mammals”) that broadens it. Amazon’s category tree and Wikipedia’s category system are both hypernym hierarchies dressed up as navigation UIs.
- **Query expansion with weighted hyponyms.** A search for “cat” is silently expanded to include hyponyms like “kitten”, “tabby”, “siamese”, each with a lower weight than the original term. Documents about specific cat breeds surface for a general “cat” query.
- **Relevance ranking with hypernym paths.** Instead of expanding the query, the scoring function is adjusted so a document mentioning “labrador” contributes something to a query for “dog” even without an explicit expansion.

```
from nltk.corpus import wordnet

for synset in wordnet.synsets('cat', pos='n')[:3]:
    print(synset.name(), '->', synset.definition())
    print('  hypernyms:', [h.name() for h in synset.hypernyms()])
    print('  hyponyms: ', [h.name() for h in synset.hyponyms()][:3]])

# cat.n.01 -> feline mammal usually having thick soft fur and no ability to roar
#  hypernyms: ['feline.n.01']
#  hyponyms:  ['domestic_cat.n.01', 'wildcat.n.03']
```

WordNet covers English; multilingual equivalents such as Open Multilingual WordNet exist for around 30 languages but with less coverage than the English version.

Part-of-speech tagging

Sentences are made of words that fall into grammatical classes: noun, verb, adjective, adverb, pronoun, article, and so on. A **part-of-speech tagger** assigns one of these labels to each token in a sentence, using surrounding words to disambiguate cases where the same surface form belongs to different classes (“run” as a noun in “a good run” versus “run” as a verb in “I run daily”). Figure 3.1 shows a POS-tagged parse tree for a simple English sentence, with the POS tags at the leaves and phrase-level structure above them.

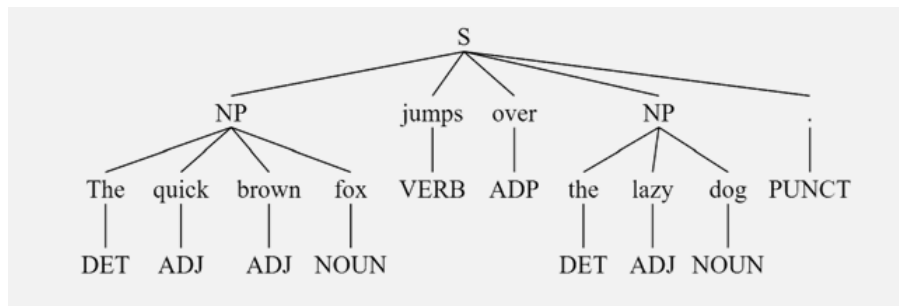


Figure 3.1: Constituency parse tree for “The quick brown fox jumps over the lazy dog.” POS tags (DET, ADJ, NOUN, VERB, ADP, PUNCT) appear at the leaves; phrase-level constituents such as NP (noun phrase) sit at intermediate nodes.

POS tagging supports three retrieval-side uses:

- **Selective stop-word filtering.** Not every occurrence of “it” is a pronoun. In the phrase “the IT department”, “IT” is a noun and should be kept; in “it is easy”, it should be dropped. POS-aware stop-word filtering keeps content-bearing occurrences and removes function-word ones. This is how the “IT security” case from the stop-word discussion in the previous section is handled correctly.
- **Lemmatizer disambiguation.** WordNet’s lemmatizer needs the POS tag: `wordnet.lemmatize("meeting", "n")` returns “meeting” while `wordnet.lemmatize("meeting", "v")` returns “meet”. Feeding the lemmatizer the wrong tag produces the wrong lemma.
- **Question-form analysis.** The query “Who is Albert Einstein?” has a WH-word (“who”), a copula verb (“is”), and a proper noun (“Albert Einstein”). Combining that structure with the recognition that “Albert Einstein” is a person’s name (see the next subsection) tells the retriever this is a person-lookup query, and the right response is not a keyword search but a lookup in a people database or a redirect to Wikipedia. The intent-routing section returns to this example.

Three families of taggers have been used historically:

- **Rule-based taggers** apply hand-written rules based on suffixes and surrounding words. Fast and simple, but each language needs its own rule set and rules cannot easily be extended.
- **Statistical (HMM) taggers** model the sentence as a sequence of hidden POS states emitting observed tokens. Transition and emission probabilities are learned from a POS-tagged corpus; decoding uses the Viterbi algorithm. Dominant approach from the 1990s to the mid-2010s.
- **Neural taggers** run a small transformer or LSTM over the token sequence and output a POS tag per token. This is the current default in spaCy and in most Hugging Face pipelines.

All three families reach around 97-99% accuracy on English news text. On informal genres (social media, product reviews) the neural taggers pull ahead because they handle out-of-vocabulary words better.

See Also

The Hidden Markov Model formulation used by statistical taggers is developed in the ML foundations appendix, alongside the Viterbi decoding algorithm. This section does not repeat the HMM math.

```

import spacy
nlp = spacy.load('en_core_web_sm')

doc = nlp("Who is Albert Einstein?")
for token in doc:
    print(f"{token.text:12s} {token.pos_:8s} {token.tag_:6s} {token.dep_}")

# Who      PRON      WP      nsubj
# is       AUX       VBZ     ROOT
# Albert   PROPN     NNP     compound
# Einstein PROPN     NNP     attr
# ?        PUNCT     .       punct
  
```

Two tag sets appear in practice. The `pos_` field uses the coarse Universal tag set (around 17 categories: NOUN, VERB, ADJ, ADV, PROPN, PRON, DET, ADP, ...). The `tag_` field uses the fine-grained Penn

Treebank tag set (around 45 categories: VBZ for third-person singular verb, NNP for proper noun, WP for WH-pronoun). The Universal tags are enough for retrieval-side filtering; the Penn tags are needed for building parse trees.

Named entity recognition

The tokens “Albert Einstein” in the previous example are not just two proper nouns; they name a specific person, and the retrieval system that recognizes them as such can act on that recognition. **Named entity recognition (NER)** classifies spans of tokens into entity types: person, location, organization, product, date, money, and so on. NER is a natural extension of POS tagging: it runs after tokenization and POS tagging (or jointly with them in modern neural pipelines) and outputs entity spans.

The approaches behind NER mirror the POS taggers above. The earliest systems were rule-based, combining gazetteers (lists of known people, places, and organizations) with hand-written patterns such as a capitalized word following a title like “Dr.” or preceding a suffix like “Inc.”. Statistical sequence models replaced them: a conditional random field (CRF) labels each token from features of the token and its neighbours, trained on annotated corpora such as CoNLL-2003. Modern systems fine-tune a transformer for token classification, tagging each sub-word with its entity type. This is the current state of the art and what spaCy’s larger models and Hugging Face NER pipelines use.

```
doc = nlp("Jack Higgins deposits £50,000 with BestBank in London.")
for ent in doc.ents:
    print(f"{ent.text:15s} {ent.label_}")

# Jack Higgins      PERSON
# £50,000           MONEY
# BestBank          ORG
# London            GPE
```

For query understanding, four categories dominate the retrieval-side use cases:

- **Person names.** “Who is Albert Einstein?” should trigger a person-lookup: prioritize Wikipedia, IMDb, MusicBrainz, and other person-oriented sources rather than running a keyword search across the whole web.
- **Time and date.** “Who won the F1 race last weekend?” contains a temporal expression. Retrieval should restrict to news articles from the past few days and rank recent items above older ones.
- **Locations.** “What to do in Basel?” should prioritize regional content and augment the results with a map. The location is a hard filter, not just another keyword.
- **Product brands.** “Where can I buy the latest iPhone?” should boost shopping and product-review pages, and can trigger a price-comparison widget.

Every NER category exposes a routing decision. The classifier and router are the subject of the next section; this section only extracts the entities themselves.

NER also generates candidate phrases automatically: consecutive tokens tagged as PERSON produce a bi-gram or tri-gram like “Albert Einstein” that should be indexed as a unit, exactly as in the previous section’s phrase-detection discussion. This is often faster and more targeted than PMI or LHR scoring over the whole corpus, because named entities are exactly the phrases that matter for entity-oriented search.

For tooling, spaCy and Stanza provide NER for many languages out of the box, nltk offers a lighter `ne_chunk`, and the Hugging Face token-classification pipeline exposes fine-tuned transformer models such as `dslim/bert-base-NER`. Beyond libraries, the major cloud providers offer NER as a managed API: Amazon Comprehend, Google Cloud Natural Language, and Azure AI Language all return typed entities (often alongside key phrases and relations) without training a model. This convenience is why they anchor many production document-processing pipelines, extracting parties and amounts from contracts, fields from invoices, or names and dates from scanned forms.

Chunking with rule-based grammars

For queries that do not fit neatly into a POS-plus-NER analysis, a lightweight **chunker** groups consecutive tokens matching a small grammar. The classic pattern is:

```
NP -> DET? ADJ* NOUN
```

which matches a noun phrase like “a red car” or “the lazy dog”. Chunking gives the retriever access to noun-phrase-level tokens without running a full parser. `nltk.RegexpParser` implements this cheaply:

```
grammar = r"NP: {<DT>?<JJ>*<NN>}"
chunker = nltk.RegexpParser(grammar)
tree = chunker.parse(nltk.pos_tag(nltk.word_tokenize("a red car")))
```

More elaborate grammars produce dependency parses that connect the noun phrase “red car” to the adjective “red” that modifies it. Stanford CoreNLP and spaCy’s dependency parser produce these for free once POS tags are available. For most retrieval scenarios noun-phrase chunking is enough.

Spell correction and “did you mean?”

Every query understanding pipeline has to handle typos. A user typing “Bücher von Goehte” (transposed letters) gets no results if the retriever demands an exact string match on “Goethe”, regardless of how good the rest of the pipeline is. The classical response is a two-part strategy: correct at both indexing time and query time.

- **At indexing time**, keep the original token but also add the auto-corrected form. A document containing the misspelling “Goehte” is indexed under both “Goehte” and “Goethe”, so users find it whether they type the correct or the incorrect spelling.
- **At query time**, run the query through the spell-checker. If a token is not in the dictionary, offer corrections and either search all of them silently or ask “Did you mean ‘Goethe’?”. Modern search engines do both: they auto-run the corrected query and let the user click back to the original spelling if the correction was wrong.

Spell correction is especially tricky for names. “Britney” is often misspelled as “Britny”, “Brittney”, or “Britnee”, which argues for folding them all back to “Britney”. The catch is that several of these forms are themselves legitimate names that other people genuinely carry, so a single variant can be at once a typo of one name and the correct spelling of another. A spell-checker that rewrites every variant to one canonical “Britney” then merges distinct people and loses anyone who is really spelled the other way. This is the “did you mean Britney, or Britnie?” problem. The pragmatic compromise is to keep the original spelling in the index and, at query time, expand to the common variants rather than silently rewriting to a single form.

Two standard algorithms underpin classical spell correction:

- **Edit distance** (Damerau-Levenshtein). Score candidate corrections by the minimum number of insertions, deletions, substitutions, and transpositions to reach the query. Widely used but slow to compute exhaustively; production systems restrict the candidate set with a phonetic prefilter or a small edit-radius trie.
- **Phonetic codes** (Soundex, Metaphone). Reduce each word to a compact code that captures its rough pronunciation. Different spellings of the same-sounding word share a code and are candidates for one another. Essential for name variants that no edit-distance approach can handle.

Query expansion and query rewriting

Several of the transformations described so far share a family resemblance. Synonym mapping, hypernym expansion, spell correction, alias substitution, and temporal normalization all take the tokens the user typed and change them so the resulting query matches more of the collection. In the IR literature these transformations fall under two umbrella names:

- **Query expansion** adds tokens to the original query without removing the ones the user typed. Synonym expansion, weighted-hyponym expansion, and adding all common spelling variants of a

name to a query all fall here. The original query terms usually stay at full weight and the added terms come in at a lower weight, so a document that matches the user's exact words still ranks above one that matches only via expansion.

- **Query rewriting** transforms the query text itself, replacing or restructuring tokens. Spell correction ("Goethe" -> "Goethe"), alias substitution ("F1" -> "Formula 1"), temporal normalization ("last weekend" -> a date range), and compound splitting on the query side ("Wolkenkratzer" -> "Wolken", "Kratzer") are query rewrites. The rewritten query goes to the retriever; the original may or may not be preserved for logging.

The line between the two blurs in practice. A rewrite that adds tokens alongside the originals (F1 OR "Formula 1" OR "Grand Prix") is an expansion in disguise. An expansion that later drops the original terms in favour of the added ones is a rewrite. What matters is that the retriever sees a different query from the one the user typed, and both directions are standard tools.

This chapter has been building both throughout:

- **Section 1** — language detection informs both expansion (which stop-word and synonym list to use) and rewriting (which stemmer to apply). Case, Unicode, and accent normalization are rewrites at the character level.
- **Section 2** — stemming and lemmatization are rewrites at the token level ("carried" -> "carri" for Porter, "carried" -> "carry" for WordNet).
- **Section 3** — compound splitting rewrites one token into several on both the document side and the query side.
- **Section 4** — synonym and hypernym mappings are expansions; spell correction is a rewrite.
- **Section 5** — temporal normalization and domain-alias substitution are rewrites applied during intent routing.

Two techniques not covered here belong in the same family and are worth naming so students recognize them elsewhere:

- **Pseudo-relevance feedback**, also called blind feedback. This is the relevance-feedback and query-expansion mechanism of the Binary Independence Model from [Probabilistic Ranking and BM25](#), run without any user judgements. Instead of asking the user which results are relevant, it assumes the top-*k* retrieved documents are relevant and applies the same term-discrimination weighting to choose expansion terms, the high-scoring terms like "woodland" in that chapter's worked example. It is a retrieval-model technique rather than a text-processing one, which is why it belongs with vector-space and BM25 ranking.
- **LLM-based query rewriting**. A language model reformulates the query. Common patterns include HyDE (write a plausible answer and search for that instead of the question), multi-query (generate several rewrites and union their results), and step-back prompting (generalize the query one level before searching). These belong to the retrieval-augmented generation chapter.

The chapter opener's F1 query is a plain example of query rewriting. "Who won the F1 race on the weekend?" is rewritten by dropping the WH-pronoun "who" and the verb "won" as non-content, expanding "F1" and "race" to "Formula 1" and "Grand Prix", and turning "the weekend" into a date range. What reaches the retriever is a scoped keyword query rather than a question, and the next section walks that rewrite through the full routing pipeline. Classical rewriting stops there: it can retrieve the right race-report page but cannot produce the short answer "Verstappen". That last step needs a reader or generator to extract the answer from the retrieved text which is covered in a later chapter (retrieval-augmented generation).

Where this sits in modern retrieval

Every classical technique in this section still runs somewhere in a modern search stack. Web search engines apply POS tagging, NER, and spell correction to every query before any retrieval or ranking. Product-search systems for e-commerce use NER heavily to extract brand names, category constraints, and numerical facets ("under \$50"). Enterprise search over document management systems relies on synonym expansion using domain-specific terminologies.

The one classical technique that dense embeddings largely subsume is synonym expansion: a well-trained embedding model puts “purchase” and “buy” near each other in vector space, so the query “purchase history” retrieves documents about “buying” without an explicit synonym list. Hypernym reasoning is partially learned but less reliably: an embedding model may or may not know that a labrador is a dog, depending on the training data. POS tagging, NER, and spell correction remain explicit steps even in fully neural pipelines because they produce structured outputs (tags, spans, corrected strings) that downstream systems can act on discretely. Large language models can substitute for all of them in a single prompt, but for high-throughput retrieval the classical stack is still cheaper by orders of magnitude.

Hands-on: Query Understanding

Run POS tagging and NER on a set of natural-language queries, extract candidate entities, and compare the spaCy and nltk outputs.

[Open notebook ->](#)

Includes pre-run results; you can read through or download and experiment.

The final section takes everything this section produces (language ID, POS tags, entities, corrections) and turns it into a routing decision: which backend should the query go to?

Intent Routing and Classification

Recall the second query from the chapter opener: “Bücher von Goethe”. By this point in the chapter, the pipeline has enough machinery to see it clearly. The tokenizer splits it into three tokens. The rule-based language detector recognizes German characters and function words. POS tagging identifies “Bücher” as a plural noun, “von” as a preposition, and “Goethe” as a proper noun. Named entity recognition promotes “Goethe” to a person entity. What has not been decided yet is what to do with all this structure. Should the query go to a book catalogue, a web index, an author database, or a general-purpose LLM? This section turns the extracted features into that routing decision.

The end-to-end query pipeline

A query enters the retrieval system as a raw string. Before any matching happens, it flows through the classical pipeline this chapter has been building:

```
raw query
-> tokenize (section 1)
-> normalize (case, Unicode) (section 1)
-> segment sentences (section 1, if multi-sentence)
-> detect language (section 1 rules + this section's classifier)
-> stop-word filter (section 2, language-specific)
-> stem or lemmatize (section 2, language-specific)
-> split compounds (section 3, if applicable)
-> POS-tag (section 4)
-> extract named entities (section 4)
-> spell-correct (section 4)
-> classify intent (this section)
-> route to backend (this section)
```

The order matters. Language detection has to come before any language-specific step; POS tagging has to come before selective stop-word filtering; NER has to come before intent classification because the entity types are input features to the classifier. What comes out at the end is a structured representation of the query that a downstream system can act on: a language, a set of entities, an intent label, and a corrected token list.

Two of these classification steps use the same underlying machinery. Language detection classifies a query into one of the languages the system knows; intent classification classifies a query into one of the backends the system supports. Both are text classification problems and both can be solved with the same Naive Bayes model. This section develops that model once, then applies it to both problems.

Naive Bayes for text classification

Naive Bayes chooses the most probable class given the observed features by applying Bayes’ theorem and assuming that features are conditionally independent given the class. The derivation lives in the ML foundations appendix; here we state only the decision rule.

For a feature vector $\mathbf{x} = (x_1, \dots, x_M)$ and classes $C_1, \dots, C_K$:

Key Formula: Naive Bayes Decision Rule

$$\hat{C} = \arg \max_k P(C_k) \cdot \prod_{j=1}^M P(x_j \text{ mid } C_k) \quad (3.7)$$

In log form, which is what implementations actually compute:

$$\hat{C} = \arg \max_k \log P(C_k) + \sum_{j=1}^M \log P(x_j \text{ mid } C_k) \quad (3.8)$$

The prior $P(C_k)$ captures the base rate of each class, and the likelihood $P(x_j \text{ mid } C_k)$ captures how well feature x_j fits class C_k . The full derivation from Bayes’ theorem is in the appendix.

Three things need to be learned from a labelled training corpus:

- **The prior $P(C_k)$:** fraction of training documents in class C_k . When the training data reflects real-world class frequencies, use it directly; when it is artificially balanced (equal examples per class) or when class balance is unknown, set a uniform prior $P(C_k) = 1/K$ and the priors drop out of the argmax.
- **The likelihood $P(x_j \text{ mid } C_k)$:** fraction of feature occurrences in class C_k that fall on token or feature x_j . This is a maximum-likelihood estimate over the training data.
- **A smoothing constant.** If x_j never appeared in the training data for class C_k , the maximum-likelihood estimate is zero, and one zero collapses the whole product to zero. Add-one (Laplace) smoothing avoids this by adding a small constant to every count.

The product in the decision rule is a direct consequence of the independence assumption: taking the features to be conditionally independent given the class turns the joint likelihood $P(\mathbf{x} \text{ mid } C_k)$ into the product $\prod_j P(x_j \text{ mid } C_k)$ of per-feature terms. This is what makes the estimation possible at all. In classical text classification the feature space has tens or hundreds of thousands of dimensions, so any particular full vector $\mathbf{x}$ almost never recurs in the training data, and $P(\mathbf{x} \text{ mid } C_k)$ could never be estimated directly. Each individual dimension x_j , by contrast, is observed many times across the corpus, so the per-feature factors $P(x_j \text{ mid } C_k)$ can be counted reliably. Independence trades a joint probability we cannot measure for a product of marginals we can.

The assumption is not literally true, since tokens in real text are not independent, but Naive Bayes still performs well on text because the errors it introduces tend to affect all classes similarly and rarely change which class wins the argmax. Its speed and small memory footprint are the reasons it stays in production stacks two decades after neural classifiers became viable.

Language detection as classification

Language detection uses character-based n-grams as features. For n from 1 to 5, count how many times each n-gram appears in the input text. This produces a bag-of-n-grams representation exactly like the bag-of-words representation used for topic classification, except that the tokens are short character strings instead of whole words.

Character n-grams work well as features because each language has characteristic short sequences that rarely appear in other languages:

- German trigrams: sch, cht, hen, und, der, die
- English trigrams: the, ing, and, ion, ent
- French trigrams: les, des, que, ent, oui
- Italian trigrams: che, non, per, ent

Some trigrams like ent appear in multiple languages but with different frequencies, which is enough for a Naive Bayes classifier to disambiguate.

Setting up the classifier:

- **Classes:** the target languages. lingua covers 70 languages; a domain-specific detector might cover only 5 or 10.
- **Features:** character n-grams from a per-language frequency profile built during training. Most of the discriminating signal sits in a modest set of frequent n-grams, so a frugal detector can store only those significant likelihoods and stay tiny. lingua in high accuracy mode keeps the full observed profile across five n-gram orders (unigram through fivegram): the English model holds about 380,000 n-grams (~3.3 MB), German about 470,000 (~4.2 MB). It trades size for robustness on short and ambiguous inputs rather than pruning to the discriminating core.
- **Prior:** uniform when the incoming text has unknown language distribution; empirical when the retrieval system knows its user base is 60% English, 30% German, 10% French.
- **Likelihood:** for the multinomial variant, $P(t_j \text{ mid } C_k) = n_{k,j} / \sum_l n_{k,l}$ where $n_{k,j}$ is the count of n-gram j in the training text for language k . With +1 smoothing, $P(t_j \text{ mid } C_k) = (n_{k,j} + 1) / (\sum_l n_{k,l} + V)$ where V is the vocabulary size.

The decision rule then takes the standard log-form:

$$\hat{C} = \arg \max_k \left(\log P(C_k) + \sum_j c_j \log P(t_j \text{ mid } C_k) \right) \quad (3.9)$$

where c_j is the observed count of n-gram j in the input.

Example: Detecting German for “Bücher von Goethe”

Extract the character trigrams, padding each word boundary with _ so that word-initial and word-final patterns become features in their own right (the classic Cavnar-Trenkle text-categorization convention):

bü, бүc, үch, che, her, er, _vo, von, on_, _go, goe, oet, eth, the, he_

For each candidate language $k \in \{\text{DE, EN, FR, IT}\}$, compute $\log P(C_k) + \sum \log P(t_j \text{ mid } C_k)$ over the 15 trigrams. German wins by a large margin because:

- _bü, бүc, үch are common in German and essentially absent from English, French, and Italian (ü is a diacritic that only German and a few other Latin-script languages use).
- _vo, von, on_ are common in German (“von” is the fourth most frequent German preposition).
- _go, goe, oet appear in German names (Goethe, Görlitz, Göttingen) at rates much higher than in English.

The boundary padding matters because languages differ in how words *begin* and *end*, not only in their interior sequences: _bü and _vo capture a distinctly German way of starting a word. *lingua* reaches the same verdict from unpadded n-grams taken within each word (its stored profiles carry no boundary marker), returning `Language.GERMAN` with a confidence of 0.96.

For very short queries with only one or two trigrams that discriminate between languages, confidence drops. *lingua* reports its confidence as a normalized posterior, and the retrieval system can fall back to a default language (English, or the user’s browser locale) when the top confidence is too close to the second-best.

```
from lingua import Language, LanguageDetectorBuilder

detector = LanguageDetectorBuilder.from_all_languages().build()
detector.detect_language_of("Bücher von Goethe")
# Language.GERMAN

# Restrict to expected languages and get confidence scores
euro = [Language.ENGLISH, Language.GERMAN, Language.FRENCH, Language.ITALIAN]
d2 = LanguageDetectorBuilder.from_languages(*euro).build()
d2.compute_language_confidence_values("Bücher von Goethe")
# GERMAN: 0.96, ENGLISH: 0.02, FRENCH: 0.01, ITALIAN: 0.01
```

langdetect is a lighter alternative: rule- and n-gram-based, 55 languages, ISO-code output.

```
from langdetect import detect
detect("Bücher von Goethe") # 'de'
detect("Livres de Goethe")  # 'fr'
detect("Books by Goethe")   # 'en'
```

Intent routing as classification

The same machinery routes the query to a backend. Once the language is known, an **intent classifier** decides which of the search system’s supported intents the query expresses:

- `book_search`: query looks for books (title, author, ISBN)
- `web_search`: general-purpose keyword search
- `image_search`: query looks for images

- `product_search`: query looks for shoppable products
- `people_search`: query looks for information about a person
- `news_search`: query has a temporal component or a current-events subject
- `map_search`: query looks for a location or directions
- `weather, calculator, definition, translation`: one-shot vertical intents

The training data is query logs annotated with the backend the user actually clicked into. Features are richer than for language detection because more of the pipeline's output can be used:

- The token bag after stemming and stop-word removal
- POS-tag counts (how many nouns, verbs, WH-words)
- NER labels present in the query (has-PERSON, has-LOCATION, has-DATE, has-MONEY)
- Question-form markers (starts with a WH-word, ends with a question mark)
- Detected language (as a categorical feature)

With these features, "Bücher von Goethe" scores highly on `book_search` because it contains a book-domain plural noun ("Bücher") and a PERSON entity ("Goethe") in an author-attribution construction. "What to do in Basel?" scores highly on `map_search` and `web_search` because it contains a WH-word, a LOCATION entity, and no product or person entity. "Who is Albert Einstein?" scores highly on `people_search` because of the WH-word combined with a PERSON entity.

The classifier itself is exactly the same Naive Bayes model as for language detection, just with different features and different classes. In practice, a modern production stack fine-tunes a small neural classifier on top of the same features because it captures conjunctions ("has-PERSON AND WH-word AND is-question") that Naive Bayes cannot express under the independence assumption. In a later chapter, we study the neural variants using LLM to make the routing decision (agentic RAG).

Naive Bayes prefers uniform priors for query classification

Query-log training data reflects the current backend distribution, not the ideal one. If 80% of past queries went to web search, an empirical prior will route new queries to web search 80% of the time regardless of their features, because the prior term overwhelms weak features. For intent routing, use a uniform prior and let the features drive the decision, or use empirical priors only after balancing the training data across intents.

End-to-end walkthrough

Putting all five sections of the chapter together on the running query:

```
Input:                "Bücher von Goethe"

Tokenize:             ['Bücher', 'von', 'Goethe']
Normalize (case):     ['bücher', 'von', 'goethe']
Detect language:      German (confidence 0.96)
Stop words:           drop 'von' -> ['bücher', 'goethe']
Stem (Snowball DE):  ['buch', 'goeth']
POS tag:              [(Bücher, NOUN), (Goethe, PROPN)]
NER:                  [(Goethe, PERSON)]
Spell-check:          no correction needed
Intent classify:       book_search (features: language=DE, PERSON present,
                                plural NOUN of book-domain lemma)
Route:                library catalog with filters
                        language = "de"
                        author   = "Goethe"
                        content  = *
```

The library catalog answers a structured query with all Goethe titles it holds. No general-purpose keyword match against "Bücher von Goethe" was ever needed.

A second walkthrough on the other chapter-opener query. This one combined a WH-question form, tokens that would not appear in the answer, and a domain-vocabulary gap.

```

Input: "Who won the F1 race on the weekend?"

Tokenize: ['Who', 'won', 'the', 'F1', 'race', 'on',
           'the', 'weekend', '?']
Normalize (case): ['who', 'won', 'the', 'f1', 'race', 'on',
                  'the', 'weekend']
Detect language: English
POS tag: [(who, WH-PRON), (won, VERB), (the, DET),
          (F1, PROPN), (race, NOUN), (on, ADP),
          (the, DET), (weekend, NOUN)]
NER: [(F1, EVENT), (the weekend, DATE)]
Selective stop-word filter: drop 'the', 'on'
                           keep 'who' as a classification feature
                           (WH-word signals question form)
Stem/lemma (English): ['win', 'F1', 'race', 'weekend']
Synonym expansion (domain): F1    -> {Formula 1, Grand Prix}
                           race   -> {Grand Prix, GP}
Temporal normalization: the weekend -> [Sat, Sun of the past weekend]
                           -> ['2026-08-08', '2026-08-09']
Intent classify: news_search
                 features: WH-word + EVENT entity +
                           DATE entity + motorsport terms
                 features against 'question' subclass:
                           WH-word + won + who
Route: news index with
       query = "Formula 1" OR "Grand Prix" OR "GP"
       date  = [2026-08-08, 2026-08-09]
       sort  = date desc

```

The classical pipeline turned three tokens of question form and one date reference into a scoped keyword query the news index can answer: a Boolean OR over motorsport aliases, a date filter for the past weekend, and a sort by date. Each of the three gaps named in the chapter opener has a specific step above that addresses it. The missing “who” is absorbed into the classification features rather than searched as a keyword. The missing “weekend” becomes a temporal range that a race-results page dated in that range will match. The missing “race” is replaced by the domain-synonym expansion.

That is as far as this chapter takes the F1 example. What the news backend returns is a ranked list of race-result pages, not the two-word answer “Verstappen won” that the user actually requested. Two later chapters carry the example the rest of the way:

- The **semantic search** chapter shows how a dense-retrieval index closes the domain-vocabulary gap without a hand-maintained synonym list, catching the driver names, sponsor terms, and circuit aliases the classical stack never saw.
- The **retrieval-augmented generation** chapter runs a language model over the top-ranked page and returns the extracted name as the answer, turning a list of results into a direct answer.

The classical pipeline of this chapter is still there in both cases. It prepares the query and prunes the search space so the more expensive downstream steps only see the pages that matter. The router itself does not do the retrieval; it picks a backend and passes the structured query to it. Everything upstream in this chapter was building the structure that the router now dispatches on.

Where this sits in modern retrieval

Classical intent routing runs in every large search stack in 2025. Google Search, Amazon Search, and enterprise search products all have an intent-classification stage before the retriever. The classifier is rarely still Naive Bayes: production systems have moved to gradient-boosted trees or small transformers because the feature interactions matter and the classifier is on the query-latency critical path only in a few-millisecond budget. Naive Bayes remains the pedagogical baseline and a common first pass in cost-sensitive deployments (mobile, embedded).

Large language models can perform the whole pipeline in one prompt: extract entities, detect the language, classify the intent, and generate the structured backend query. This is a live area of production adoption, especially for open-ended assistant systems. The trade-offs against classical pipelines are latency (an LLM call is 100-1000 milliseconds versus sub-millisecond for the classical stack), cost (per-query LLM cost is measurable, per-query classical cost is not), and confidence quantification (Naive Bayes gives calibrated posteriors, LLMs do not). Production stacks in 2025 usually run the classical pipeline first, escalate to LLMs for the queries where the classical stack's confidence is low, and reserve LLMs for high-value verticals like agentic search and complex reformulation.

Two forward references pick up where this section leaves off:

- The text classification chapter covers deep-learning text classification with TextCNN and transformer-based classifiers, using sentiment analysis as its running example. The Naive Bayes classifier of this section is the baseline those methods are compared against.
- The semantic-search chapter covers embedding-based intent classification: instead of extracting features from the query and running Naive Bayes, embed the query into a vector space and compare it to prototype vectors for each intent. Modern hybrid systems combine both approaches.

Hands-on: Intent Routing

Train a Naive Bayes language detector on a small multi-language corpus, then extend it into an intent classifier over a set of example queries, and walk a query through the full routing pipeline.

[Open notebook ->](#)

Includes pre-run results; you can read through or download and experiment.

The next page summarizes the chapter and points to the material that follows it in the book.

Summary

Method Comparison

Stemmers and lemmatizers side by side.

Method	Approach	Correctness	Speed	Multi-lingual	Needs dictionary
Porter	Rule-based suffix stripping (English)	pseudo-stem	fast	English only	no
Lancaster	Aggressive rule-based (English)	pseudo-stem, over-stems short words	fastest	English only	no
Snowball	Rule-based framework	pseudo-stem	fast	20+ languages	no
WordNet	Dictionary + POS lemmatizer	linguistic lemma	slow	English only (multilingual variants weaker)	yes
spaCy	Neural POS tagger + dictionary lemmatizer	linguistic lemma	medium	many languages	yes

Use a rule-based stemmer when performance dominates or when no dictionary is available for the language. Use a dictionary-based lemmatizer when accuracy on strongly-inflected languages matters (German case-inflected nouns, English strong verbs like “went -> go”) or when the downstream task needs linguistic base forms rather than arbitrary reductions. Snowball is the go-to compromise for multi-language corpora when only rule-based options are practical.

Key Takeaways

1. Classical retrieval fails in two directions: one concept has many surface forms on the document side (recall gap), and free-text queries carry structure that the retriever ignores (query understanding gap). Advanced text processing addresses both.
2. Tokenization is the first stage and the largest source of quality issues. A naive regex tokenizer breaks on possessives, numbers, abbreviations, and punctuation. Modern nltk and spaCy tokenizers handle these but still need retrieval-oriented cleanup.
3. Normalization decides which surface forms collapse to the same token. Case folding is nearly always applied. Unicode normalization is applied silently. Accent folding trades recall for precision and depends on the scenario.
4. Sentence segmentation is a small but essential step. Every RAG chunking strategy, every POS tagger, and every query-analysis pipeline depends on being able to find sentence boundaries reliably.
5. Stop-word removal is a size-versus-recall trade-off. Modern BM25-based systems handle stop words gracefully through IDF and can afford to keep them in the index. Legacy vector-space systems remove them aggressively. Phrase queries and titles like Stephen King’s “It” break under aggressive removal.
6. Stemming is a fast pseudo-linguistic reduction; lemmatization is a slower dictionary-based one that produces the true linguistic base form. Choose based on speed budget, language complexity, and whether downstream steps need real words.
7. Phrases and compounds sit on opposite sides of the word-boundary problem. Bi-grams like “New York” pack multiple tokens into one concept and need explicit phrase indexing; PMI and LHR are the classical selectors for which bi-grams are worth adding. Compounds like “Bücherregal” do the reverse, hiding multiple concepts in one token, and are split into their parts by log-frequency scoring, though only endocentric compounds split safely (a “Bücherregal” is a kind of “Regal”; a “Wolkenkratzer” is not a kind of “Kratzer”, so splitting it hurts precision).

8. Query understanding turns free text into structured features: language, POS tags, named entities, corrected spelling. Together they let the retriever route the query to the right backend and generate structured queries rather than doing bag-of-words matching.
9. A Naive Bayes classifier applied to hand-engineered features is enough to build both a language detector and an intent classifier. Modern production stacks use neural classifiers for accuracy but keep Naive Bayes as the baseline and as the fast path for cost-sensitive deployments.
10. Every classical technique in this chapter still runs in production 2025 search stacks. Dense retrieval subsumes some of them (synonym expansion) implicitly, but hybrid retrieval combines both branches and needs the classical stack on the lexical side.

Key Formulas

$$\text{pmi}(t_1, t_2) = \log_2 \frac{N \cdot \text{tf}(t_1, t_2)}{\text{tf}(t_1) \cdot \text{tf}(t_2)} \quad (3.10)$$

Pointwise Mutual Information: high when two tokens almost always appear together and rarely apart. Biased toward rare-word bi-grams; requires a minimum-frequency filter.

$$\log \lambda = \log \frac{L(H_1)}{L(H_2)} \quad (3.11)$$

Log Likelihood Ratio: log-ratio of the probability under independence to the probability under dependence. Robust on sparse data; ranks by $-2 \log \lambda$.

$$\text{score}(S) = \frac{1}{|S|} \sum_{p_i \in S} \log \frac{\text{tf}(p_i)}{N} \quad (3.12)$$

Compound Split Score: average log-frequency of the parts in split S . Choose the split with the highest score.

$$\hat{C} = \arg \max_k \left(\log P(C_k) + \sum_{j=1}^M \log P(x_j \text{ mid } C_k) \right) \quad (3.13)$$

Naive Bayes Maximum A Posteriori: pick the class that maximizes the log-prior plus the sum of log-likelihoods. Used for language detection over character n-grams and for intent classification over feature vectors combining tokens, POS tags, and NER labels.

Exam focus

- The two failure directions classical retrieval hits, and which chapter technique addresses each
- Why possessives, abbreviations, and multi-token numbers break the naive regex tokenizer
- The difference between stemming (pseudo-stem, rule-based) and lemmatization (real lemma, dictionary-based), and when to choose each
- Why aggressive stop-word removal breaks queries like “The Who” and “IT security”, and how BM25’s IDF weighting reduces the need for aggressive removal
- How PMI and LHR rank the same bi-grams differently and why both are useful
- Why endocentric compounds split usefully and exocentric ones do not
- The three retrieval-side uses of POS tagging (stop-word filtering, lemmatizer disambiguation, question-form analysis)
- How language detection and intent classification are the same Naive Bayes machinery with different features and classes
- Where in modern hybrid retrieval each classical step still runs, and where dense embeddings subsume it

Self-Check Questions

1. (Understand) The naive tokenizer produces ['Watson' , 's'] from “Watson’s”. Under what retrieval scenario is this desirable, and under what scenario is it a problem? What information is permanently lost?
2. (Understand) Why does BM25 tolerate stop words that appear in the index better than a raw vector-space TF-IDF model does?

3. (Analyze) Given the *A Study in Scarlet* corpus where “said” occurs 207 times, “Holmes” 98 times, “Sherlock” 52 times, “Sherlock Holmes” 52 times, and “said Holmes” 12 times, compute PMI for the two bi-grams “Sherlock Holmes” and “said Holmes”. Explain why the ranking matches (or does not match) what a human would call “the more meaningful phrase”. Assume a corpus of $N = 43,200$ tokens.
4. (Analyze) Explain why aggressive accent folding hurts a legal-document retrieval system for German but helps a general-purpose web search engine over the same language.
5. (Analyze) A retrieval system indexes “Wolkenkratzer” only as itself (no compound splitting). A user queries “Wolke”. A different user queries “Kratzer”. Which query, if any, retrieves documents about skyscrapers, and why? Would splitting help both users equally? Would you split this specific compound?
6. (Evaluate) A production search team proposes replacing the entire classical query pipeline (tokenization, stemming, POS, NER, intent classification) with a single call to a large language model that outputs a structured query directly. State two reasons the team might keep the classical pipeline anyway, and one query type where the LLM call is likely to win.
7. (Apply) A user queries “Livres de Molière”. Walk through the end-to-end pipeline from section 5, listing the output of each step and the final routing decision. Assume the search stack supports French.

Test Your Knowledge

[Take the Chapter 3 Quiz ->](#)

Further Reading

- Porter, M. F. (1980). **An Algorithm for Suffix Stripping**. *Program*, 14(3), 130-137. [Author copy](#). The original Porter Stemmer paper, still the reference for how to build a rule-based English stemmer and the ancestor of every rule-based stemmer that followed.
- Paice, C. D. (1990). **Another Stemmer**. *ACM SIGIR Forum*, 24(3), 56-61. Introduced the Lancaster stemmer and the aggressive suffix-stripping strategy that trades precision for speed.
- Manning, C. D., & Schütze, H. (1999). **Foundations of Statistical Natural Language Processing**, Chapter 5: Collocations. *MIT Press*. The standard reference for PMI, likelihood-ratio testing, and other statistical collocation-scoring methods; source of the LHR formulation used in this chapter.
- Kiss, T., & Strunk, J. (2006). **Unsupervised Multilingual Sentence Boundary Detection**. *Computational Linguistics*, 32(4), 485-525. [Free PDF](#). The Punkt algorithm, still the sentence-segmentation default in NLTK almost two decades later.
- Koehn, P., & Knight, K. (2003). **Empirical Methods for Compound Splitting**. *Proceedings of the 10th Conference of the European Chapter of the ACL*, 187-193. [Free PDF](#). The frequency-based compound-splitting method this chapter uses, evaluated for machine translation but applicable directly to retrieval.
- Minixhofer, B., Pfeiffer, J., & Vulić, I. (2023). **CompoundPiece: Evaluating and Improving Decomounding Performance of Language Models**. [arXiv:2305.14214](#). Shows that modern subword tokenizers (SentencePiece, BPE) still decompound German poorly, motivating why explicit decomponers remain in hybrid retrieval stacks.

Implementations and tools

- **NLTK** — Python toolkit for classical NLP; ships all the stemmers, tokenizers, POS taggers, and collocation finders used in this chapter.
- **spaCy** — Industrial-strength NLP library with neural taggers and lemmatizers for many languages.
- **WordNet** — Lexical database for English synonyms, hypernyms, and hyponyms.
- **Snowball** — Multi-language stemmer framework, source of the German and French stemmers used in section 2.
- **lingua-language-detector** — Character-n-gram Naive Bayes language detector covering 70+ languages.
- **Elasticsearch decompounder documentation** — Production compound-splitter configuration for German, Dutch, and other compounding languages.

The classical tools in this chapter sit at the end of a seventy-year research arc. The optional note below traces that history from 1950s symbolic AI through statistical corpus methods to today's neural models, for readers who want the wider context.

From ELIZA to Transformers: a short history of NLP (optional reading)

The classical techniques in this chapter descend from a much longer research programme that runs from 1950s symbolic AI through statistical corpus-based methods to today's neural language models. Not exam-relevant, but useful for placing the tools in context and understanding where the field went next.

Symbolic era (1950s-1980s). Rule-based systems dominated NLP. Joseph Weizenbaum's ELIZA (1966) simulated a Rogerian psychotherapist by matching keyword patterns and substituting into templates (Figure 3.2); users routinely believed it understood them. Terry Winograd's SHRDLU (1971) parsed natural-language commands in a blocks-world simulator (Figure 3.3), resolving pronouns ("it") and detecting ambiguity ("which pyramid?") explicitly. Both systems worked spectacularly in their narrow domains and failed the moment inputs strayed outside those domains. The lesson was that natural language cannot be fully formalized with hand-written rules.

```
Welcome to
EEEEEE LL   IIII ZZZZZZ AAAAA
EE   LL   II   ZZ  AA  AA
EEEEEE LL   II   ZZZ  AAAAAA
EE   LL   II   ZZ  AA  AA
EEEEEE LLLLLL IIII ZZZZZZ AA  AA

Eliza is a mock Rogerian psychotherapist.
The original program was described by Joseph Weizenbaum in 1966.
This implementation by Norbert Landsteiner 2005.

ELIZA: Is something troubling you ?
YOU:  Men are all alike.
ELIZA: What is the connection, do you suppose ?
YOU:  They're always bugging us about something or other.
ELIZA: Can you think of a specific example ?
YOU:  Well, my boyfriend made me come here.
ELIZA: Is it important to you that your boyfriend made you come here ?
YOU:  He says I'm depressed much of the time.
ELIZA: I am sorry to hear that you are depressed.
YOU:  It's true. I am unhappy.
ELIZA: Can you explain what made you unhappy ?
YOU:
```

Figure 3.2: A terminal session with ELIZA (Landsteiner's 2005 reimplement of Weizenbaum's 1966 program). Simple pattern matching, no world model, and users still developed emotional attachments.

This is an example of a **SHRDLU** session:

Person: Pick up a big red block.

Computer: OK.

Person: Grasp the pyramid.

Computer: I DON'T UNDERSTAND WHICH PYRAMID YOU MEAN.

Person (changing their mind): Find a block which is taller than the one you are holding and put it into the box.

Computer: BY "IT", I ASSUME YOU MEAN THE BLOCK WHICH IS TALLER THAN THE ONE I AM HOLDING.

Computer: OK.

Person: What does the box contain?

Computer: THE BLUE PYRAMID AND THE BLUE BLOCK.

Person: What is the pyramid supported by?

Computer: THE BOX.

Person: How many blocks are not in the box?

Computer: FOUR OF THEM.

Figure 3.3: A SHRDLU session (Winograd, 1971), tracking pronoun references across turns and reporting ambiguity explicitly when a referring expression is underspecified.

Statistical era (1980s-2010s). Hidden Markov Models drove advances in POS tagging, speech recognition, and named entity recognition. N-gram language models powered early machine translation and text generation. Annotated corpora such as Brown and the Penn Treebank provided the labelled data these methods required. The statistical bet paid off: data-driven models beat hand-crafted rules on almost every task. The limits were the size of the labelled datasets, heavy feature engineering, and no ability to capture long-range meaning.

Neural era (2010s-present). Bengio's 2003 neural language model represented words as continuous vectors. Word2Vec (Mikolov et al., 2013) made word embeddings practical at scale. RNNs, then LSTMs and GRUs, handled sequence dependencies. In 2017 the Transformer replaced recurrence with self-attention, enabling parallel processing of full sequences and much longer contexts. The techniques in this chapter still power the lexical-retrieval side of modern hybrid systems; the neural side is covered in the semantic-search chapter.